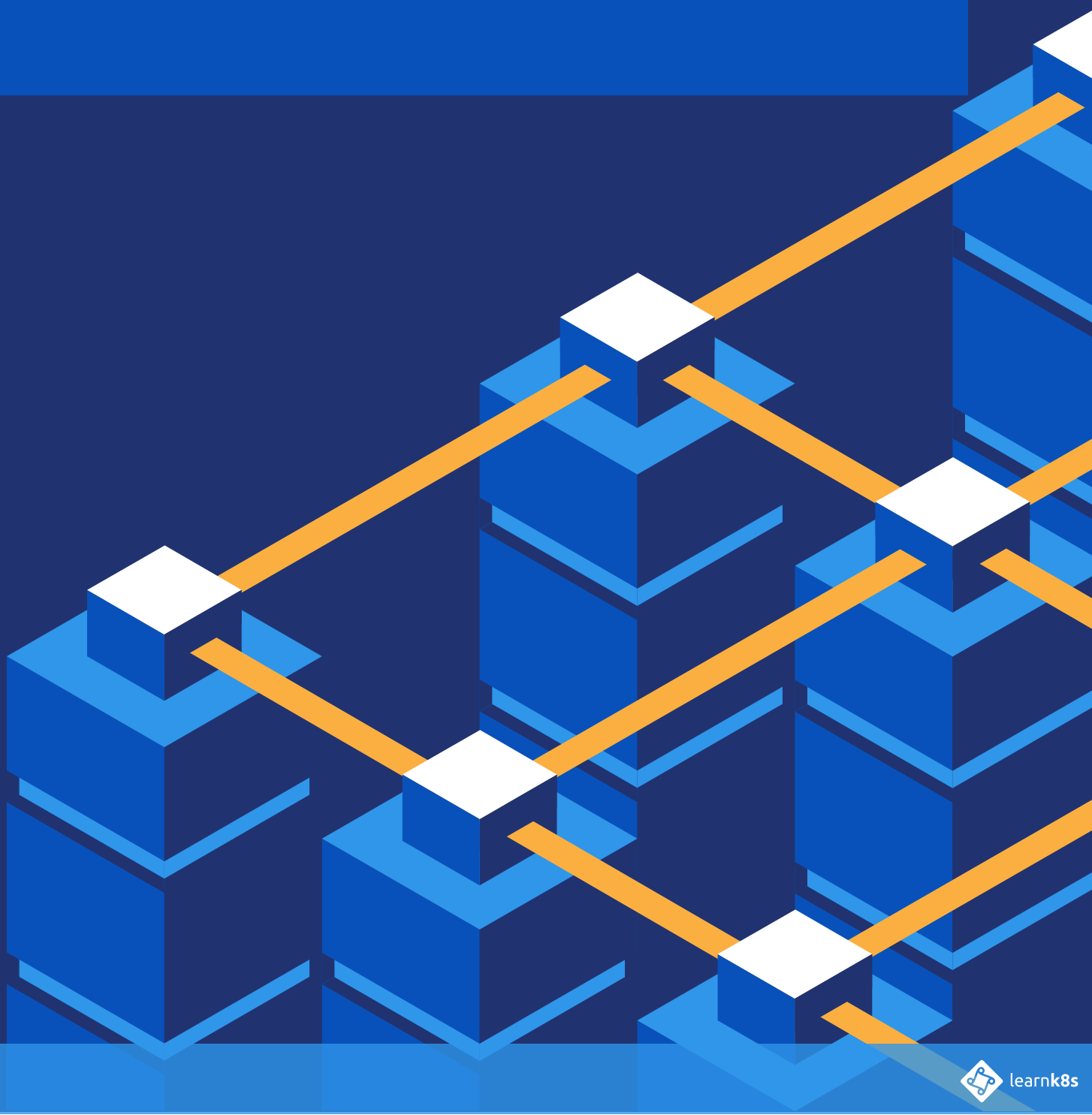
KUBERNETES ADVANCED NETWORKING



learnk8s

Table of contents

1. Introduction **6**

Kubernetes networking 7

Pod networking and CNI plugins 10

2. Pod networking **13**

The Kubernetes Pod networking model 16

Network namespaces 21

Host network, Pod network, and Pod subnets 29

3. The Container Network Interface (CNI) **35**

What is the CNI? 36

What are CNI plugins? 39

The ADD operation 41

The DEL operation 47

Input and output 50

The network configuration (NetConf) 52

The environment variables 55

The response 57

CNI plugin chaining 59

IPAM plugins 62

Example CNI plugins 65

4. Lab 1/4 — Designing the CNI plugin 68

The 'bridge' CNI plugin 69

Tracing a packet 76

Inter-node communication 81

5. Lab 2/4 — Creating the cluster 94

Network planning 96

Setting up the GCP command-line tool 99

Launching the infrastructure 102

Installing Kubernetes 110

Inspecting the cluster 117

6. Lab 3/4 — Implementing the CNI plugin 119

CNI plugin overview 120

Invoking the IPAM plugin 124, 137

Doing the one-time setup 124
Doing the Pod-specific setup 125
Returning the response 126, 164
Preliminary notes 127
Programming language 127
CNI version 128
Input parameters 129
The boilerplate code 130
Configuring the output 132
Reading the input 134
Invoking the IPAM plugin 124, 137
The one-time setup 141
The Pod-specific setup 152
Creating a veth pair 155
Setting up the host network interface 157
Setting up the Pod network interface 159
Creating a default route in the Pod network namespace 162

Returning the response 126, 164

The DEL operation 167

The VERSION operation 168

7. Lab 4/4 — Installing and testing the CNI plugin 170

Installing the CNI plugin 171

Testing the CNI plugin 178

Pods have IP addresses 180

Pods can communicate to Pods on the same node 181

Pods can communicate to Pods on different nodes 182

Pods can communicate to processes on the nodes 183

Processes on the nodes can communicate to Pods 185

Pods can communicate to destinations outside the cluster 188

All tests passed! 189

Cleaning up 190

Chapter 1

Introduction

*Welcome to the first part of the **Advanced networking** course series.*

*The first course is **Pod networking and CNI plugins**.*

Kubernetes networking

Networking in a Kubernetes is an important but quite complex topic.

The complexity is due to the fact that Kubernetes has multiple networking subsystems that all address different problems.

To truly understand networking in Kubernetes, you have to be able to conceptually separate these subsystems and understand each one's purpose, scope, and concepts.

The following is an overview of the most important networking subsystems in Kubernetes:

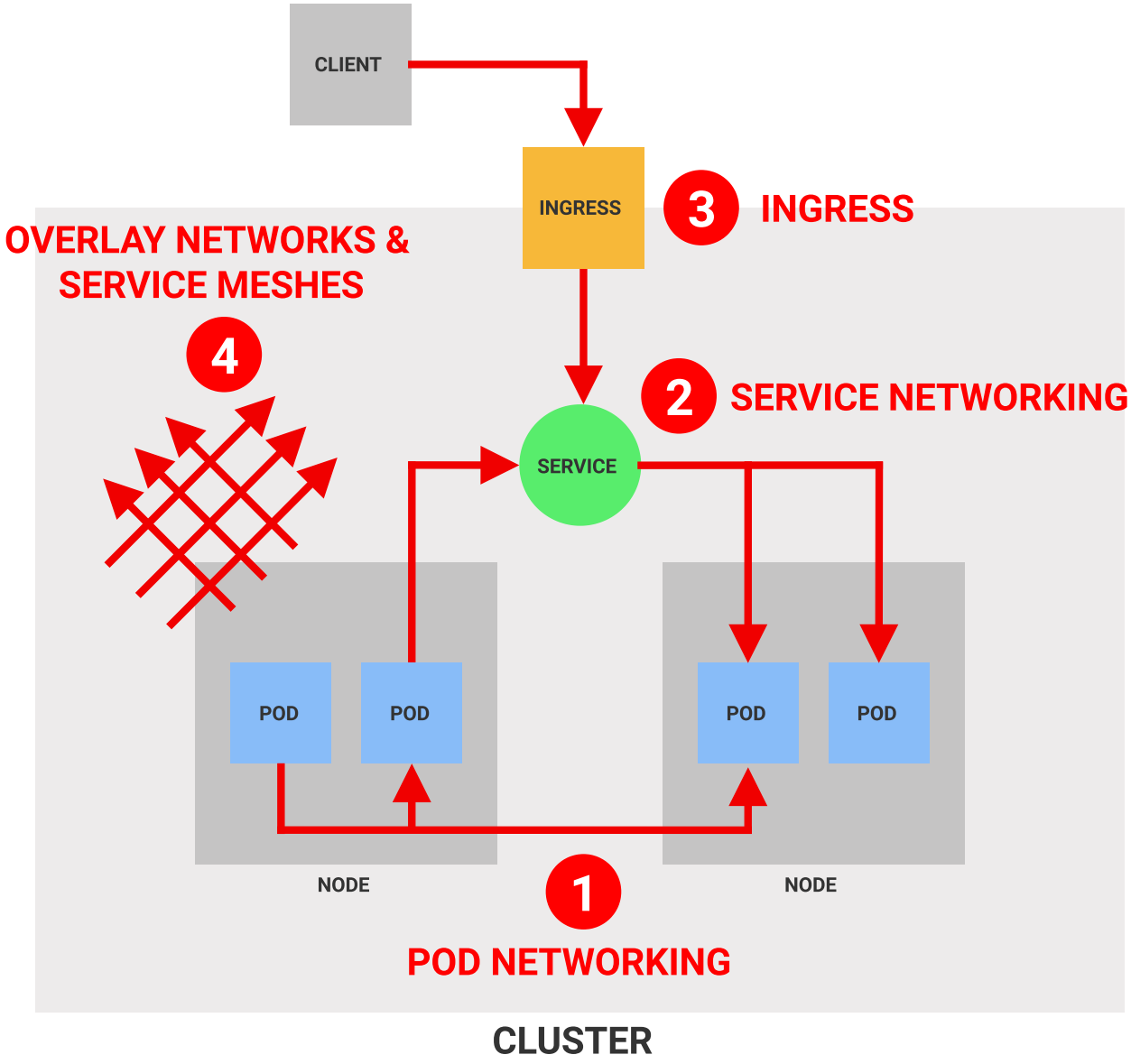


Fig. Kubernetes networking overview

The above illustration four networking subsystems in Kubernetes:

1. **Pod networking:** communication between Pods
2. **Service networking:** communication to Pods via the

2. Service abstraction
3. **Ingress**: communication to Services from outside the cluster
4. **Overlay networks & service meshes**: custom abstract networks on top of the default networks

Each of these items is its own subsystem with its own problem space, solutions, technologies, and concepts.

However, these subsystems also interrelate and build on each other.

For example, Pod networking is the basis for Service networking, and Ingresses depend on Service networking.

All this can make it difficult to stay on top of the Kubernetes networking space — and there are so many buzzwords like CNI, kubenet, kube-proxy, kube-router, iptables, IPVS, Cilium, Flannel, Calico, Weave Net, Traefik, Envoy, and Istio, and more.

The *Advanced networking* course series covers all of these subsystems in detail.

In each course, you will get hands-on with the main concepts of the corresponding networking topic.

For example, in the *Pod networking and CNI plugins*, you will not just learn about CNI plugins, but you will *build* your own CNI plugin — in *Overlay networks & service meshes*, you will not just learn about service meshes, you will *build* your own service mesh.

The goal of this is to provide you with a deep understanding of the fundamental *concepts* and *mechanisms* of each networking subsystem.

If you have that fundamental understanding, you can then fan out and explore specific solutions to the corresponding networking problems.

Let's start with the first course!

Pod networking and CNI plugins

In this course, you will learn about the most fundamental of all Kubernetes networking aspects — Pod networking.

Pod networking consists of the establishment of communication paths to and from the Pods in your cluster.

It enables the Pods in your cluster to communicate with each other.

But in a more general sense, Pod networking is what enables Pods to communicate at all — with other Pods, with Kubernetes components inside the cluster (such as the API server or kubelet), and with users outside the cluster.

In Kubernetes, Pod networking is implemented by components called **CNI plugins**.

A CNI plugin is a third-party component that can be

installed in a Kubernetes cluster and that creates all the necessary network infrastructure that Pods need to communicate.

There are many different ways to do this, and each CNI plugin represents a specific Pod networking solution.

The interface between Kubernetes and the CNI plugins is the **Container Network Interface (CNI)**, which is where the name CNI plugin comes from.

Through this standard interface, it is possible to install many different CNI plugins — you choose a CNI plugin depending on the type Pod networking that you want in your cluster.

Examples of existing CNI plugins are Cilium, Weave Net, Calico, Flannel, Multus CNI, Nuage CNI, Contiv CNI, TungstenFabric, Amazon VPC CNI, Azure CNI, and there are many more.

These CNI plugins all have different characteristics, but they all have in common that they create the necessary communication infrastructure for the Pods in your cluster.

In this course, you will build your own CNI plugin.

By the end of the course, you will have an implementation of a CNI plugin and you will install it on a cluster and verify that it really enables your Pods to communicate.

By building your own CNI plugin, you have to walk through all the steps that a CNI plugin has to take to achieve its task, which is the best way to learn about the

fundamental concepts of Pod networking and CNI plugins in general.

But before you start with the lab, you will first learn about Pod networking and the Container Network Interface (CNI) in more detail in the following sections.

Chapter 2

Pod networking

Pod networking enables Pods to communicate with other entities in the cluster over a network.

These entities may be other Pods as well as ordinary processes that run on the nodes of the cluster, including Kubernetes components like the kubelet or the API server.

> Kubernetes components running as ordinary processes on the cluster nodes are sometimes called "node agents".

Each Pod must be able to send network packets to other Pods and processes running on the nodes, as well as receiving network packets from these entities.

The following illustrates this:

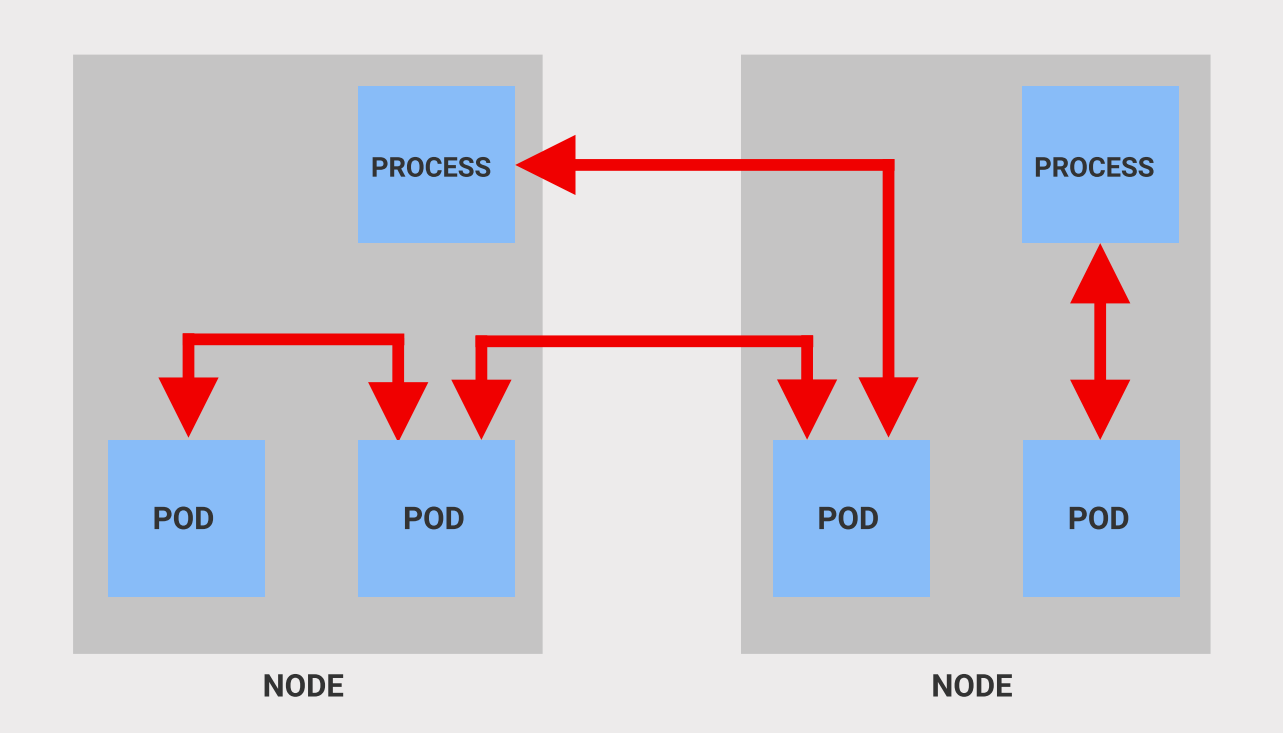


Fig. Pod networking

Kubernetes does not provide an implementation for these connectivities itself, but it leaves this task to third-party components called CNI plugins.

The job of a CNI plugin is to set things up so that these connections may occur.

Kubernetes invokes CNI plugins to do this task through the Container Network Interface (CNI) — which is where the name CNI plugin comes from.

> You will learn more about the CNI in the next section.

So, Kubernetes does not define the technicalities of how Pod networking is to be implemented.

However, what Kubernetes *does* define is a Pod networking *model*.

This model defines the basic requirements for Pod networking and the CNI plugins are responsible for implementing this model.

In this section, you will learn about the Pod networking model.

The Kubernetes Pod networking model

The most fundamental characteristic of the Kubernetes Pod networking model is as follows:

Each Pod has its own IP address.

That's right — each Pod has its own IP address, in the same way, that a node in a network has its own IP address.

The Pods use this IP address for all their network

communication.

Let's consider an example:

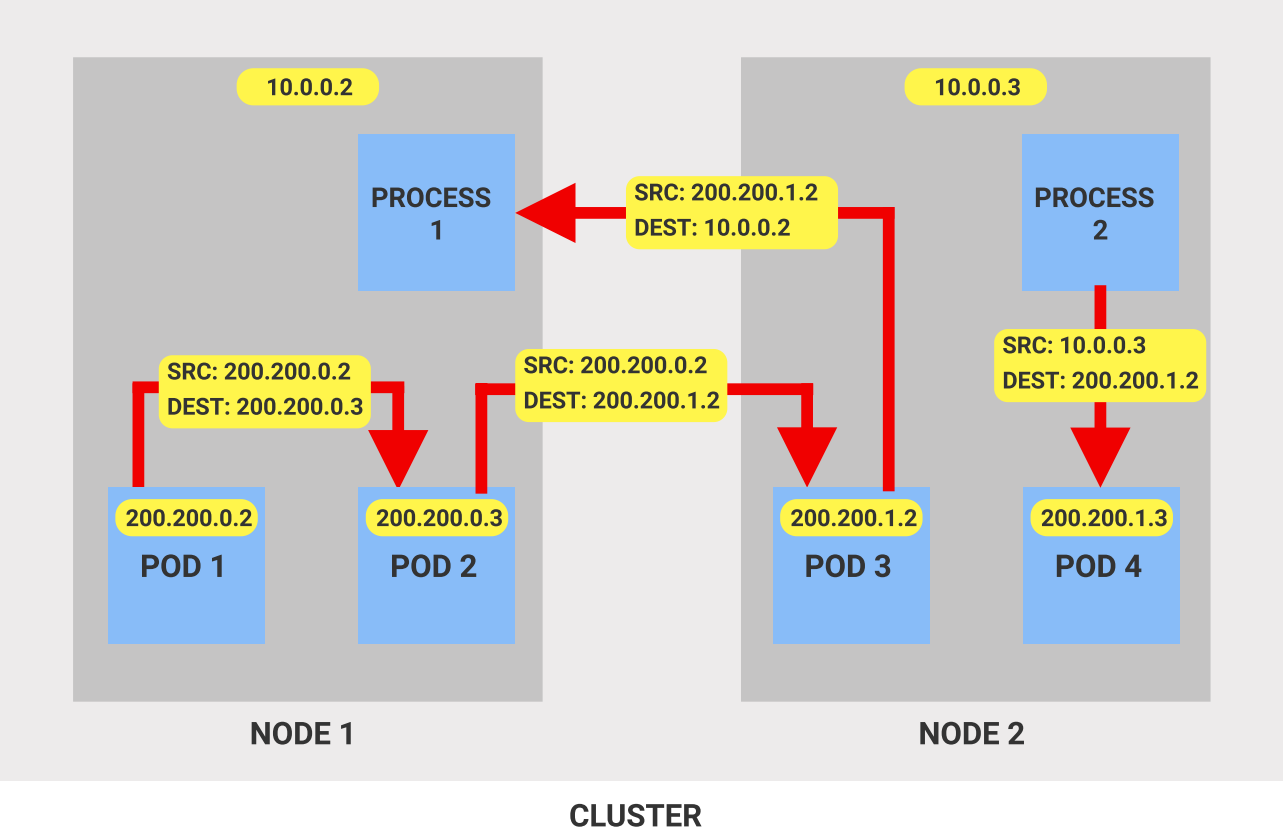


Fig. Pod networking

In the above example, there are four Pods on two nodes.

Each Pod has its own IP address — for example, Pod 1 has IP address 200.200.0.2.

Similarly, each node has its own IP address (as every node in a network) — for example, node 1 has IP address 10.0.0.2.

The Kubernetes Pod networking model defines, for example, that Pod 1 can send a network packet to Pod 2, and

that this packet has the IP address of Pod 1 (200.200.0.2) in the source field and the IP address of Pod 2 (200.200.0.3) in the destination field.

This applies to any pair of Pods, no matter if they run on the same node or on different nodes.

The processes running on the nodes that are *not* Pods (which may be node agents like the kubelet) don't have their own IP address, but they use the IP address of the node they're running on (like every ordinary process on a networked host).

The Kubernetes Pod networking model also defines that Pods must be able to reach these processes and vice versa.

For example, Pod 3 must be able to send a packet to process 1 running on node 1 with IP address 10.0.0.2, with a packet that has the IP address of Pod 3 (200.200.1.2) in the source field and the IP address of node 1 (10.0.0.2) in the destination field.

The same applies in the other direction for a process sending a packet to a Pod — the packet must have the IP address of the node that the process is running on in the source field and the IP address of the Pod in the destination field.

These connectivities are the essence of the Kubernetes Pod networking model.

And it is the job of a CNI plugin to implement these connectivities.

Kubernetes defines some addtional details for its Pod

networking model, which are as follows:

1. **The IP address that a Pod sees itself having is the same that other Pods see it having**
2. **Communication between Pods occurs without network address translation (NAT)**
3. **Communication between Pods and node processes occurs without NAT**

The first item states that the IP address of a Pod must be "real" and not just, for example, some mapping in another abstraction.

The second and third items state that the communication between Pods and other processes in the cluster must occur without network address translation (NAT), which means that the source and destination IP addresses of a packet must not be changed on the way from the sender to the receiver.

The effect of this is a network where each Pod acts similar to a host in a traditional flat network.

A Pod has its own IP address and it can send packets to the IP address of other Pods — just like a host can send packets to another host in a traditional network.

Furthermore, the node itself also has its own IP address and also acts like a host (it actually is a host in the traditional sense).

This results in a conceptual networking model where the different entities in a cluster communicate with each other like hosts in a traditional network.

The following shows an example:

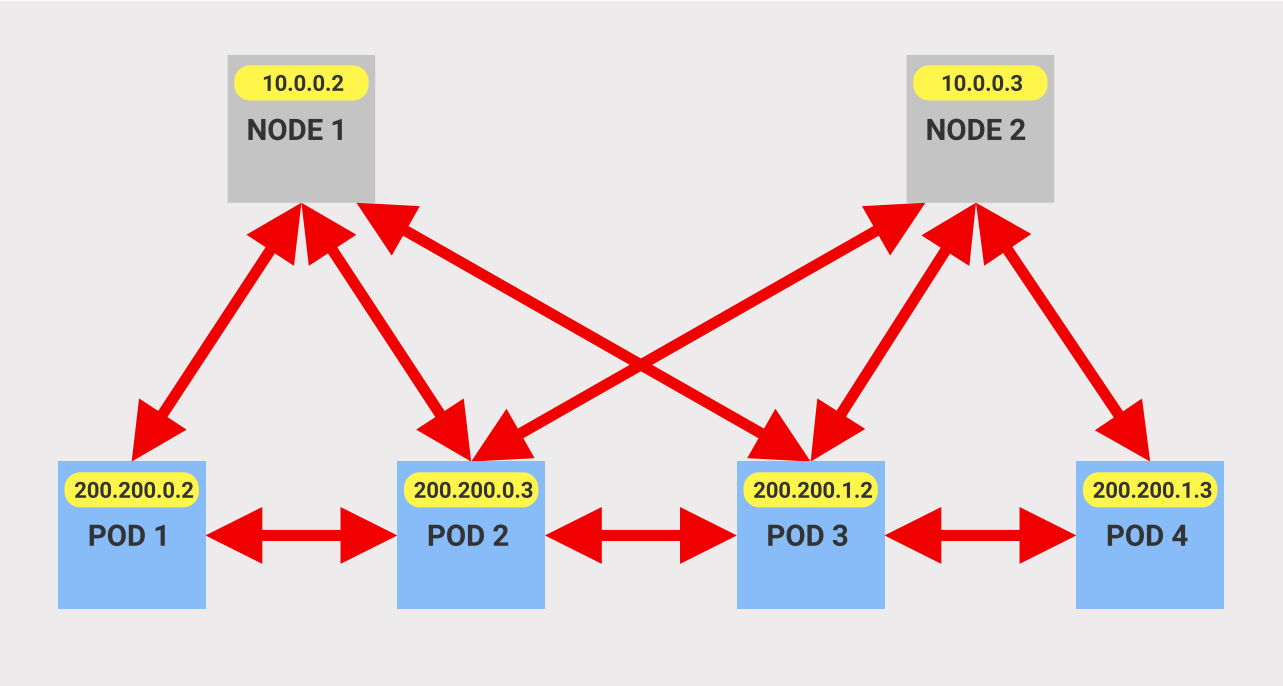


Fig. Pod networking

In the above example, each Pod and each node forms an independent logical entity that acts like a host and interacts with other entities like with other hosts.

Physically, Pod 1 and Pod 2 run on node 1 and Pod 3 and Pod 4 run on node 2 — however, for the logical entities, these physical conditions are *transparent*.

A Pod acts like a host and it sees other Pods as other hosts, no matter what node they're physically running on.

A Pod also sees the nodes themselves (including the node it's physically running on) as hosts that it can reach through their IP addresses.

The same applies to the nodes themselves which see Pods (and other nodes) as hosts reachable through their IP addresses.

Conceptually, you can think of the Pods and nodes of a Kubernetes cluster as a flat network of equal hosts that can communicate with each other through their IP addresses.

This is the Kubernetes Pod networking model.

You must be wondering how this is possible.

How can a Pod that physically runs on a node have its own IP address, and act like a separate host?

Enter network namespaces.

Network namespaces

Network namespaces are a Linux concept that belongs to the family of Linux namespaces.

In general, a Linux namespace abstracts and isolates a certain type of system-wide resources so that they're only visible to

the processes running in this namespace.

> There exist seven types of Linux namespaces (mount, IPC, network, UTS, PID, user, and cgroup) and network namespaces are one of them.

A network namespace abstracts and isolates the system-wide networking resources.
This includes the network interfaces, routing tables, packet filtering modules (firewall), and more.
This means that each network namespace can have its own version of these resources, and a process in a given network namespace sees only those resources that are in its network namespace.
The following shows an example:

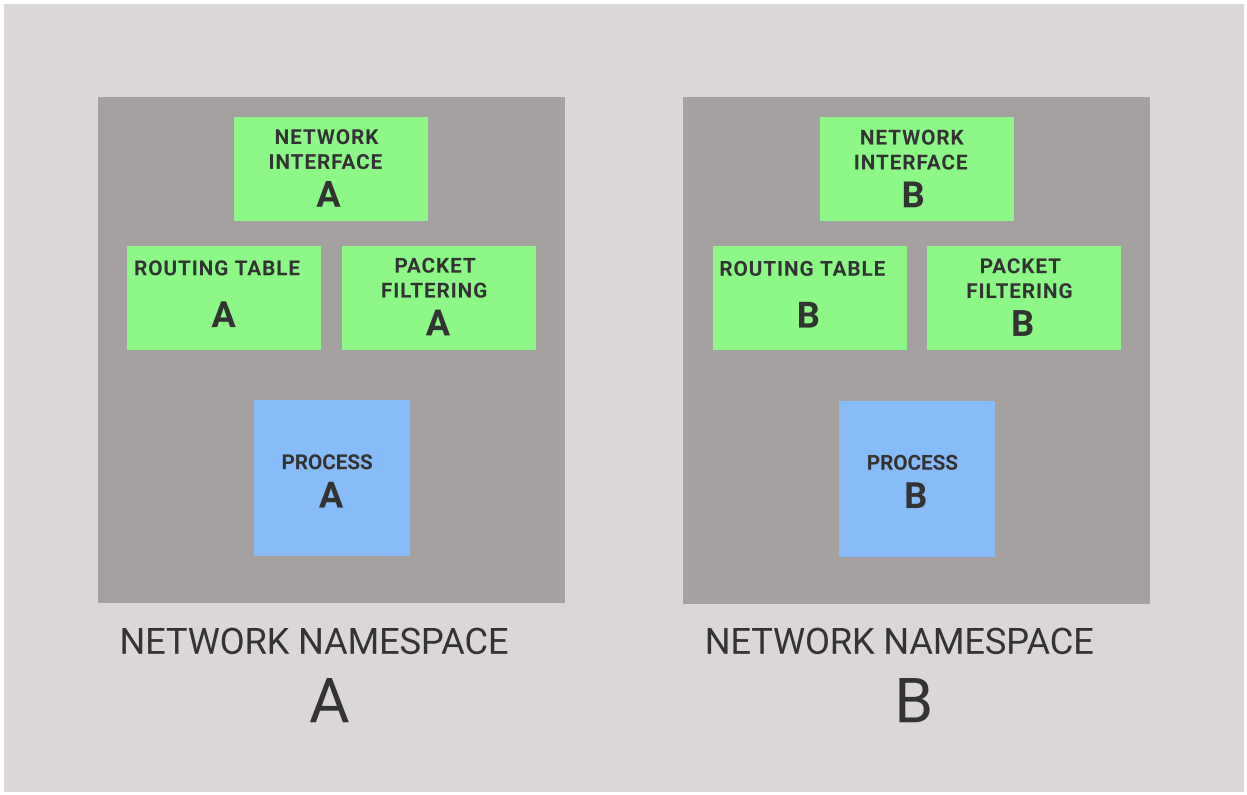


Fig. Network namespaces

In the above example, there are two network namespaces on the host and each has its own network interface, routing table, and packet filtering module.

Physically, there are two versions of each resource on the host, but for a process in either network namespace, it looks as if there's only one version of each resource on the host (the version in its network namespace).

For example, for process *A* it looks as if it's running on a

host with only one network interface (the one in network namespace *A*), and similarly for process *B* in network namespace *B*.

Thus, network namespaces can create multiple logical hosts (from a networking perspective) on the same physical host.

This is how Kubernetes Pods work.

In Kubernetes, each Pod runs in its own network namespace.

In particular, a Kubernetes node has a separate network namespace for each Pod that is running on it (in addition to the default network namespace that exists on every Linux system).

These Pod network namespaces contain a single main network interface with an assigned IP address.

The IP address of this network interface is the Pod's IP address and all the Pod's network communication goes through this network interface.

The following shows an example:

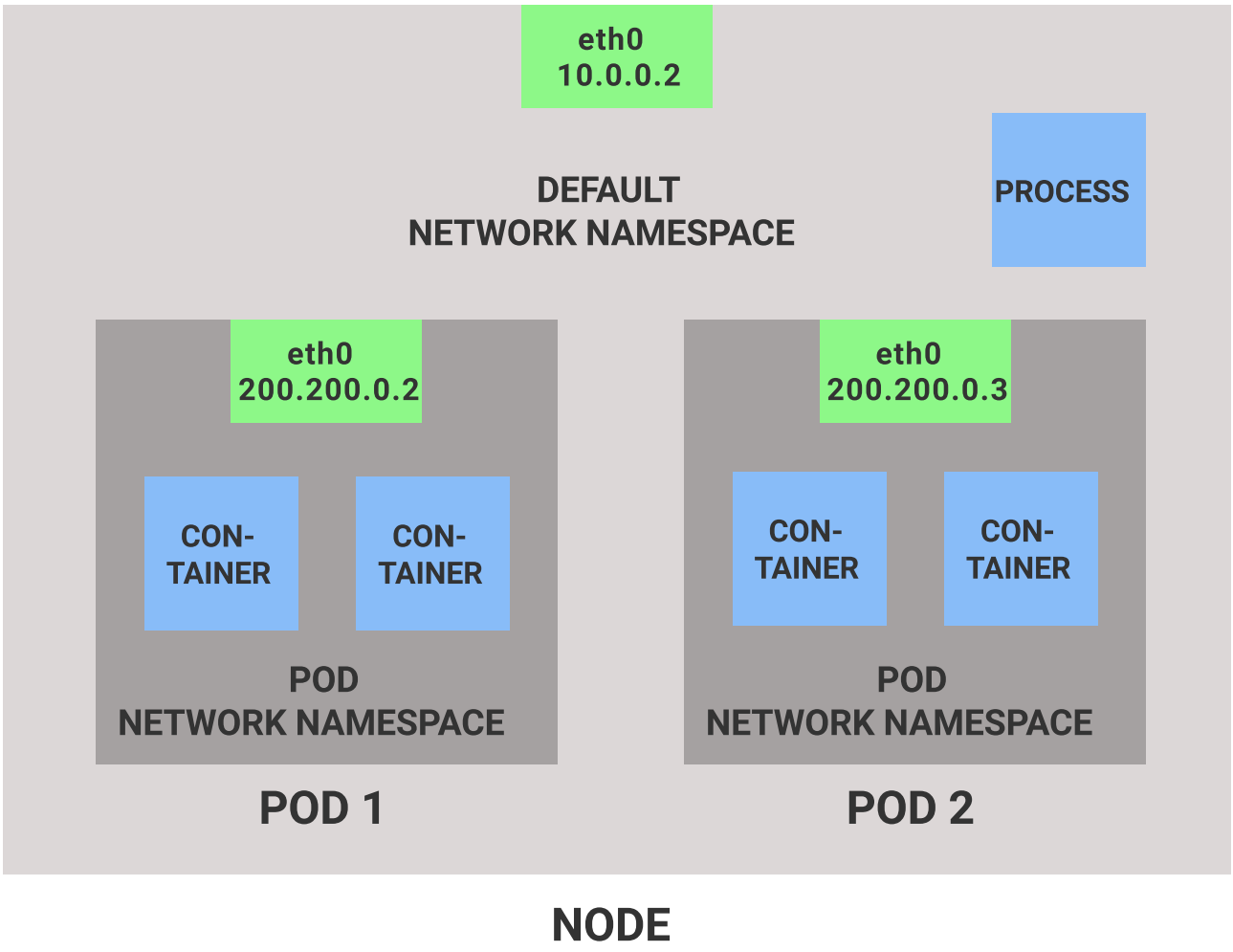


Fig. Network namespaces

The above example shows a node with two Pods — each Pod runs in its own network namespace, and each network namespace has an `eth0` network interface with an assigned IP address.

The network interface in the network namespace of Pod 1 has IP address 200.200.0.2, and the network interface in the network namespace of Pod 2 has IP address 200.200.0.3 —

these are the IP addresses of these two Pods.

For a container in Pod 1, it looks as if it's running on a host with a single network interface named `eth0` with IP address 200.200.0.2 — and the same applies for a container in Pod 2 with respect to IP address 200.200.0.3, respectively.

This is how Pods can have their own IP addresses and look like hosts.

Every process that is *not* in a Pod runs in the default network namespace of the node — this includes the mentioned node agents, like the kubelet.

The default network namespace also has a network interface with an IP address, which is most likely the physical network interface of the node.

The above example thus has three network namespaces (two Pod network namespaces and the default network namespace) and thus can be conceptualised as three logical hosts (that physically run on the same node):

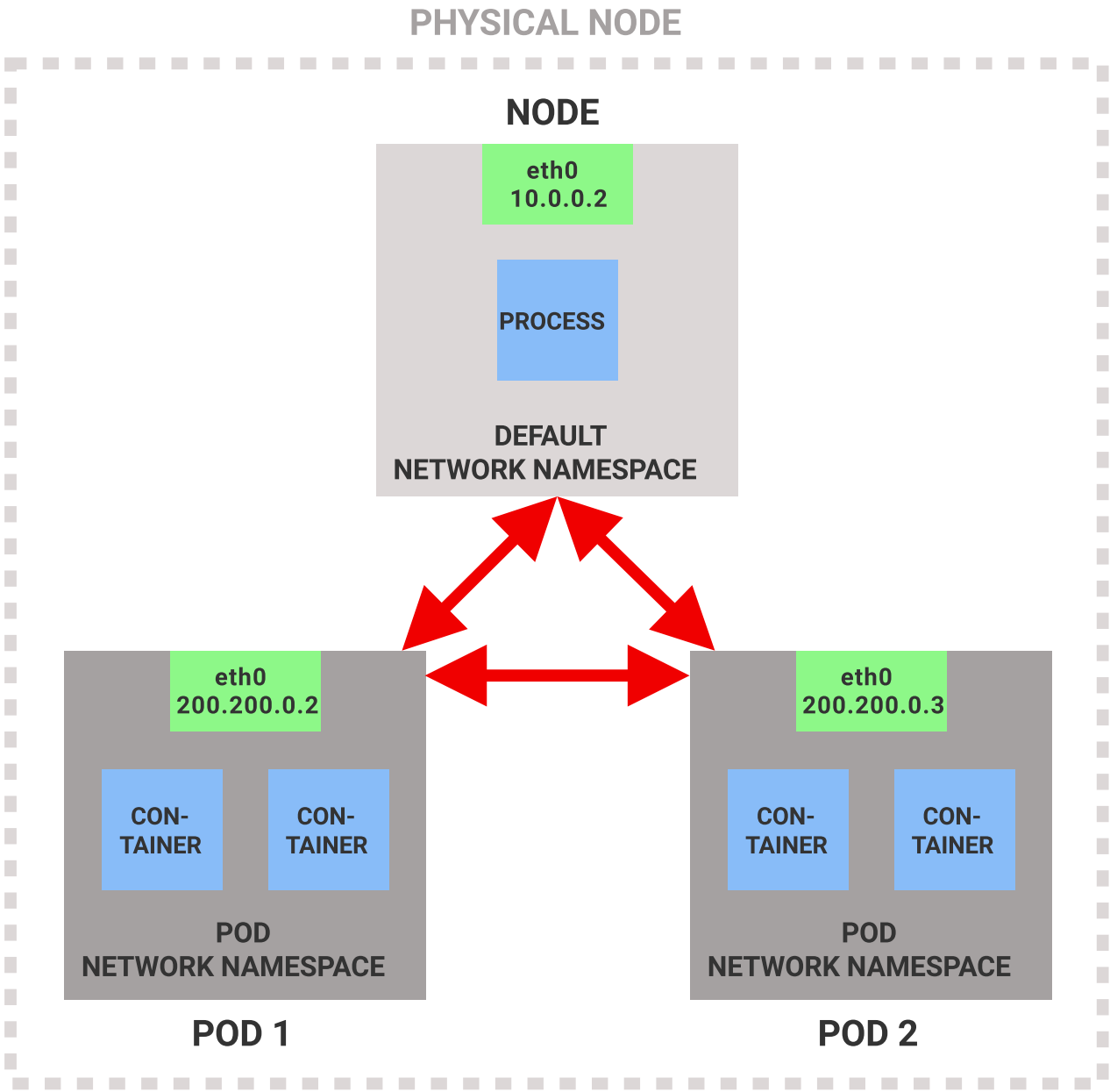


Fig. Network namespaces

The Kubernetes Pod networking model requires that all these entities be able to communicate with each other by using their IP addresses.

If you're wondering how the communication between these entities works, then you're asking the right question.

The implementation of these connectivities is the purpose of a CNI plugin.

This includes the creation of the network interface in each Pod network namespace (and assignment of an IP address to it).

Most CNI plugins use a virtual network interface for this purpose.

> After all, most nodes have only one or some few physical network interfaces, so it's not feasible to use a physical network interface for each Pod.

However, all these technicalities are up to the CNI plugin and different CNI plugins use different approaches.

You will get hands-on with this when you implement your own CNI plugin later in this course.

You should now have a basic understanding of the Kubernetes Pod networking model:

- Each Pod has an IP address
- Pods are modelled as independent hosts by the means of network namespaces
- All the Pods and nodes of a cluster form a network of logical hosts that can communicate with each other by

- using their IP addresses

The implementation of all this has to be provided by a CNI plugin.

Before proceeding to have a closer look at CNI plugins and the Container Network Interface (CNI), we should clear up some further important notions about Kubernetes networking.

Host network, Pod network, and Pod subnets

You have seen that there exist two types of entities that are modelled like hosts in a Kubernetes cluster — Pods and nodes.

These two groups of entities form two separate logical networks, which are called the **Pod network** and **host network**.

These two networks are distinguished because they typically use different non-overlapping IP address ranges.

The following shows an example:

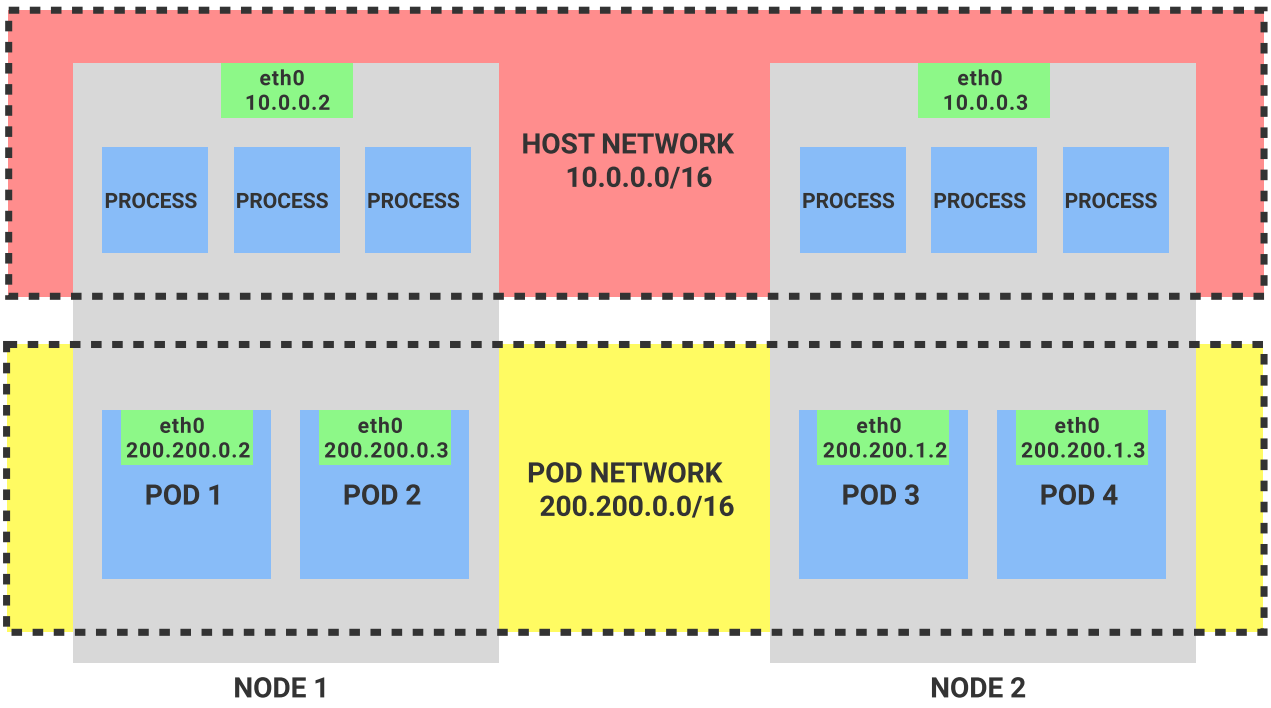


Fig. Host network and Pod network

In the above example, there are two nodes with four Pods running on them.

Consequently, the Pod network consists of four entities (the four Pods), and the host network consists of two entities (the two nodes).

As you can see, both of these networks use completely different IP address ranges.

- The host network has IP address range 10.0.0.0/16
- The Pod network has IP address range 200.200.0.0/16

This is very typical for Kubernetes cluster.

The IP address range of the host network is often dictated by the hardware that the cluster is running on.

For example, if the cluster is running in an existing network, then the IP addresses that each node can have are determined by the network.

On the other hand, the IP address range for the Pod network can be freely chosen when creating a Kubernetes cluster.

Since Pods usually use virtual network interfaces, there are no physical constraints as to what IP address a Pod can have.

In fact, the IP address range for the Pod network is a parameter that has to be supplied when creating a cluster.

In particular, the Pod network IP address range has to be supplied to the `--cluster-cidr` command-line flag of the controller manager.

If you use kubeadm to create a Kubernetes cluster, then the Pod network IP address range has to be supplied to the `--pod-network-cidr` command-line flag of kubeadm.

The important point to remember is that you can choose any IP address range you like for the Pod network.

The only point to consider is that the Pod and host network IP address ranges should *not* overlap, or you might end up with Pods and nodes having the same IP addresses, which will likely cause problems.

In general, the CNI plugin assigns each Pod an IP address out of the IP address range of the Pod network.

However, the Pod network is further divided into **Pod**

subnets.

In particular, each node in Kubernetes is assigned a Pod subnet, and all Pods on a node must have an IP address from the Pod subnet of that node.

> The allocation of Pod subnets to nodes is done by the controller manager during cluster creation. It's possible to turn this behaviour off by setting the --allocate-node-cidrs command-line flag of the controller manager to `false`. The Pod subnet IP address range of each node is saved in the `node.spec.podCIDR` field of the Node objects in Kubernetes.

Thus, the CNI plugin must assign each Pod an IP address from the Pod subnet of the node that the Pod is running on. The following shows an example:

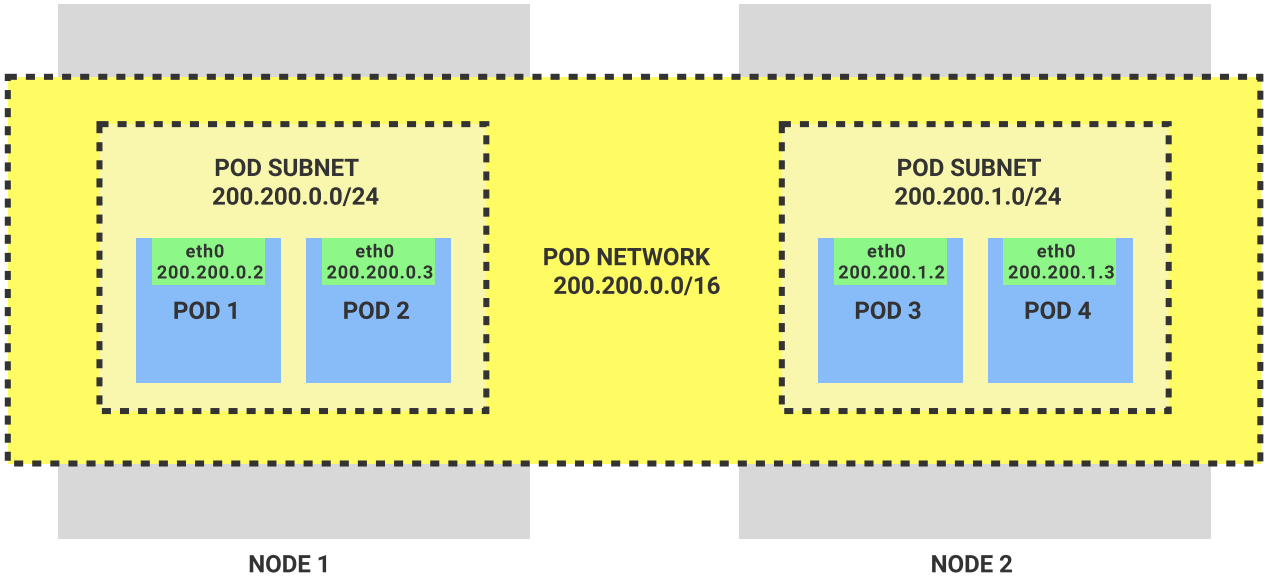


Fig. Host network and Pod network

In the above example, the Pod network IP address range is 200.200.0.0/16, and each node is assigned a Pod subnet:

- 200.200.0.0/24 for node 1
- 200.200.1.0/24 for node 2

The consequence of this is that all Pods on node 1 must have an IP address from the 200.200.0.0/24 range, for example, 200.200.0.2 and 200.200.0.3.

Similarly, all Pods on node 2 must have an IP address from the 200.200.1.0/24 range, for example, 200.200.1.2 and 200.200.1.3.

This organisation allows telling which node a Pod is running on by only looking at its IP address.

For example, if you encounter the Pod IP address 200.200.1.3, you know that this Pod is running on node 2 because node 2 is allocated the Pod subnet 200.200.1.0/24.

This is a crucial capability for CNI plugins to leverage to implement connectivity to Pods across cluster nodes.

> You will make use of this feature in your own CNI plugin too.

You should now have a comprehensive overview of the requirements posed by the Kubernetes Pod networking model.

The implementation of these requirements is the task of CNI plugins.

In the next section, you will learn in detail what CNI plugins are and how they work.

Chapter 3

The Container Network Interface (CNI)

In the previous section, you learned about the Kubernetes Pod networking model and its requirements.

The requirements of the Pod networking model have to be implemented by CNI plugins.

CNI plugins are called that way because they adhere to the Container Network Interface (CNI).

In this section, you will learn in detail what the CNI and CNI plugins are.

What is the CNI?

The Container Network Interface (CNI) is an interface specification maintained by the CNI project.

> The CNI project is an incubating Cloud Native Computing Foundation (CNCF) member. CNI is *not* specific to Kubernetes, but it is *used* by Kubernetes.

The goal of CNI is to provide a standard interface between container runtimes and container network implementations to allow different container network implementations to be plugged into container runtimes.

In a general sense, container runtimes are systems that manage the execution of containers, and a container network provides connectivity between these containers.

In the context of Kubernetes, Kubernetes is the container runtime, the Pod networking model is the container network, and CNI plugins are the container network implementations.

> In a Kubernetes context, the generic term "container", as used in the CNI specification, can be replaced with "Pod".

Kubernetes uses the CNI its interface of choice for Pod network implementations.

However, Kubernetes is not the only system that uses the CNI — other container runtimes use it too.

Conversely, there exist many container network implementations (CNI plugins) that can be plugged into any system that uses the CNI.

The following shows an overview:

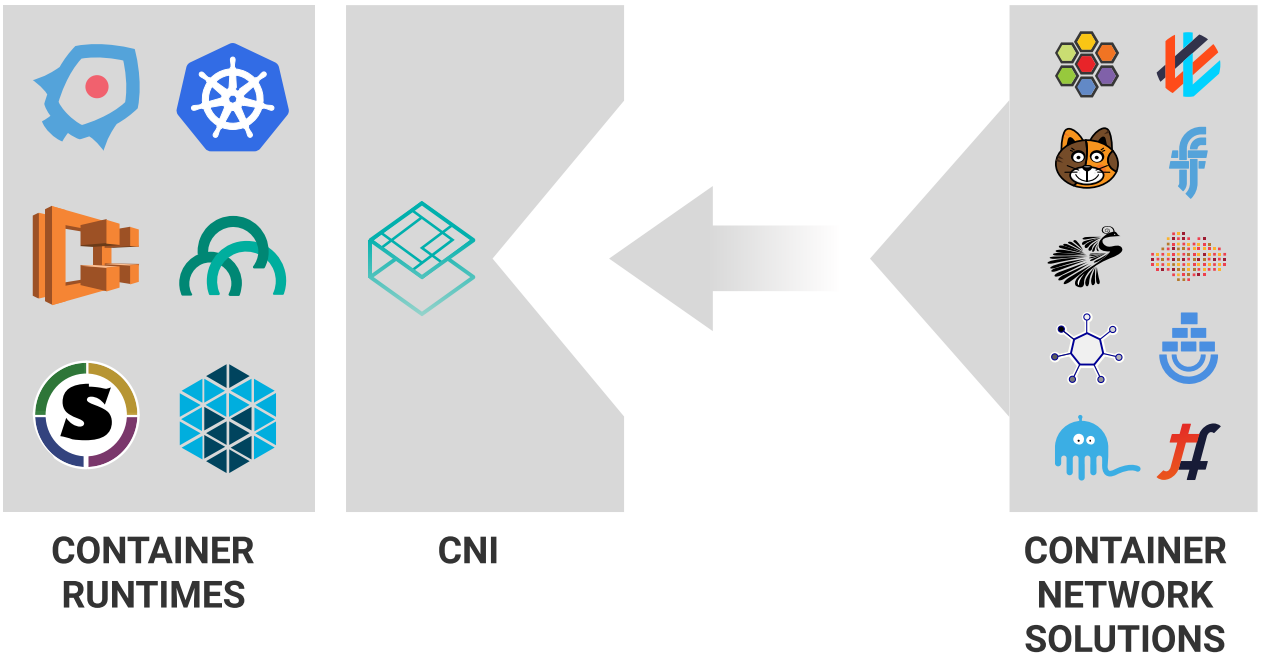


Fig. CNI

Container runtimes that use the CNI include:

- Kubernetes
- rkt
- Cloud Foundry
- Amazon ECS
- Apache Mesos
- Singularity

On the other hand, available container network implementations (CNI plugins) include:

- Cilium

- Weave Net
- Calico
- Flannel
- Multus CNI
- Nuage CNI
- DANM
- Contiv CNI
- Kube-OVN
- TungstenFabric

> You can find a list of CNI plugins in the CNI GitHub repository.

The CNI specifciation is available as a plain text file on the CNI GitHub repository.
You will tightly work with the CNI specification when you will develop your CNI plugin later.

What are CNI plugins?

The CNI specification defines a CNI plugin to be an executable file that may be written in any programming

language.
This file must be located local to the container runtime, and the container runtime invokes it during certain lifecycle events of a container.

> Again, in the context of Kubernetes, you can replace the "container" with "Pod".

These lifecycle events are:

1. When a container is created
2. When a container is deleted

When a container is created, the CNI plugin is expected to connect the container to the container network.
The CNI prescribes the use of a network namespace for the container (see the previous section).
In particular, the container runtime is expected to create a network namespace for the container, and the CNI plugin then has to wire up this network namespace with the container network.
When a container is deleted, the task of the CNI plugin is to release any resources that were allocated for this container.
These two cases are generally known as the ADD and DEL operations of a CNI plugin and they form the backbone of the interaction between a container runtime and a CNI

plugin.

In the following, let's walk through the generic procedure (not the implementation) that occurs when a container runtime invokes both the ADD and DEL operations of a CNI plugin.

Since you use Kubernetes, the explanations will be in the context of Kubernetes.

The ADD operation

The Kubernetes component that is responsible for creating and deleting Pods is the kubelet, hence, it is the kubelet that invokes CNI plugins.

The CNI plugin executable must be located on every node of the cluster in the `/opt/cni/bin` directory, by default.

The following describes what happens when the kubelet creates a new Pod:



Fig. 1 The kubelet gets the order to create a new Pod on its node.

Fig. 2 The kubelet creates a new network namespace and launches the default pause container in this network namespace.

Fig. 3 The kubelet executes the CNI plugin executable in `/opt/cni/bin`, passing it, among other parameters, a reference to the network namespace.

Fig. 4 The CNI plugin creates a network interface in the network namespace and assigns it an IP address. This will be the IP address of the Pod in the Pod network.

Fig. 5 The CNI plugin performs any other necessary network configurations to connect the network interface of the network namespace to the Pod network. This may include the creation of routes, packet filtering rules, or network interfaces in the Pod's network namespace, the node's default network namespace, or any infrastructure between the nodes. This task is very specific to each CNI plugin.

Fig. 6 The end result of the CNI plugin's work is that the Pod's network interface is correctly connected to the Pod network according to the Kubernetes networking requirements.

Now that the Pod can connect to the rest of the network is

the the job completed?
Almost.

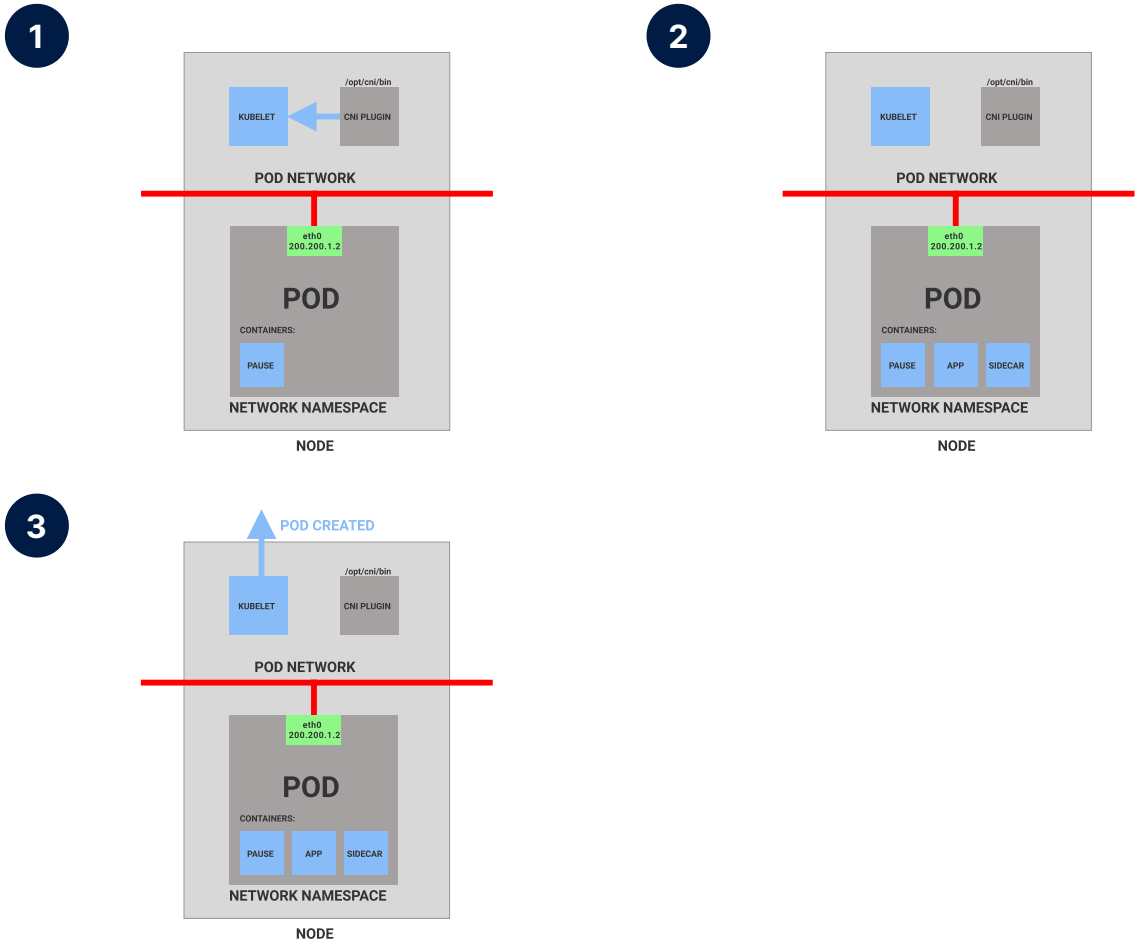


Fig. 1 The CNI plugin hands back control to the kubelet.

Fig. 2 The kubelet continues the creation of the Pod by launching the Pod's workload containers in the network namespace.

Fig. 3 The kubelet signalises the successfuly creation of the Pod. This may include writing the Pod's IP address to the the `pod.status.podIP` field of the Pod object in the Kubernetes object store.

When the kubelet creates a new Pod, it first creates a new empty network namespace, and it starts the ubiquitous pause container in that network namespace.

> The pause container is used to keep the network namespace alive when the workload containers of the Pod are not running.

The kubelet then executes the CNI plugin executable in the `/opt/cni/bin` directory by passing it a well-defined set of parameters (these parameters will be described below).

The parameters passed by the kubelet to the CNI plugin include a reference to the network namespace that the kubelet just created.

Now, it's the CNI plugin's job to wire up the network namespace to the Pod network.

To do so, the CNI plugin creates a network interface in the network namespace and allocates it an IP address from the

Pod network IP address range.

However, this is not enough — depending on the network implementation, the CNI plugin does many other tasks, such as creating routes, packet filtering rules, or further network devices.

These settings may be done in the Pod's network namespace, in the default network namespace of the node, or on any infrastructure between the nodes (such as gateway in a cloud network).

In the end, the network interface in the network namespace must be fully connected to the Pod network, so that the containers in the Pod can connect to any other Pod in the cluster (as well as node agents and the outside world), as specified by the Kubernetes network requirements.

At this point, the CNI plugin terminates and hands control back to the kubelet.

The kubelet continues the creation of the Pod by launching the workload containers in the network namespace.

When this is all done, the kubelet signalises the successful creation of the Pod to the rest of the Kubernetes control plane — this usually includes writing the IP address of the Pod into the `pod.status.podIP` field of the Pod object in the Kubernetes object store.

Next, let's see how the CNI plugin is used to delete existing Pods.

The DEL operation

The following describes what happens when the kubelet deletes a Pod:

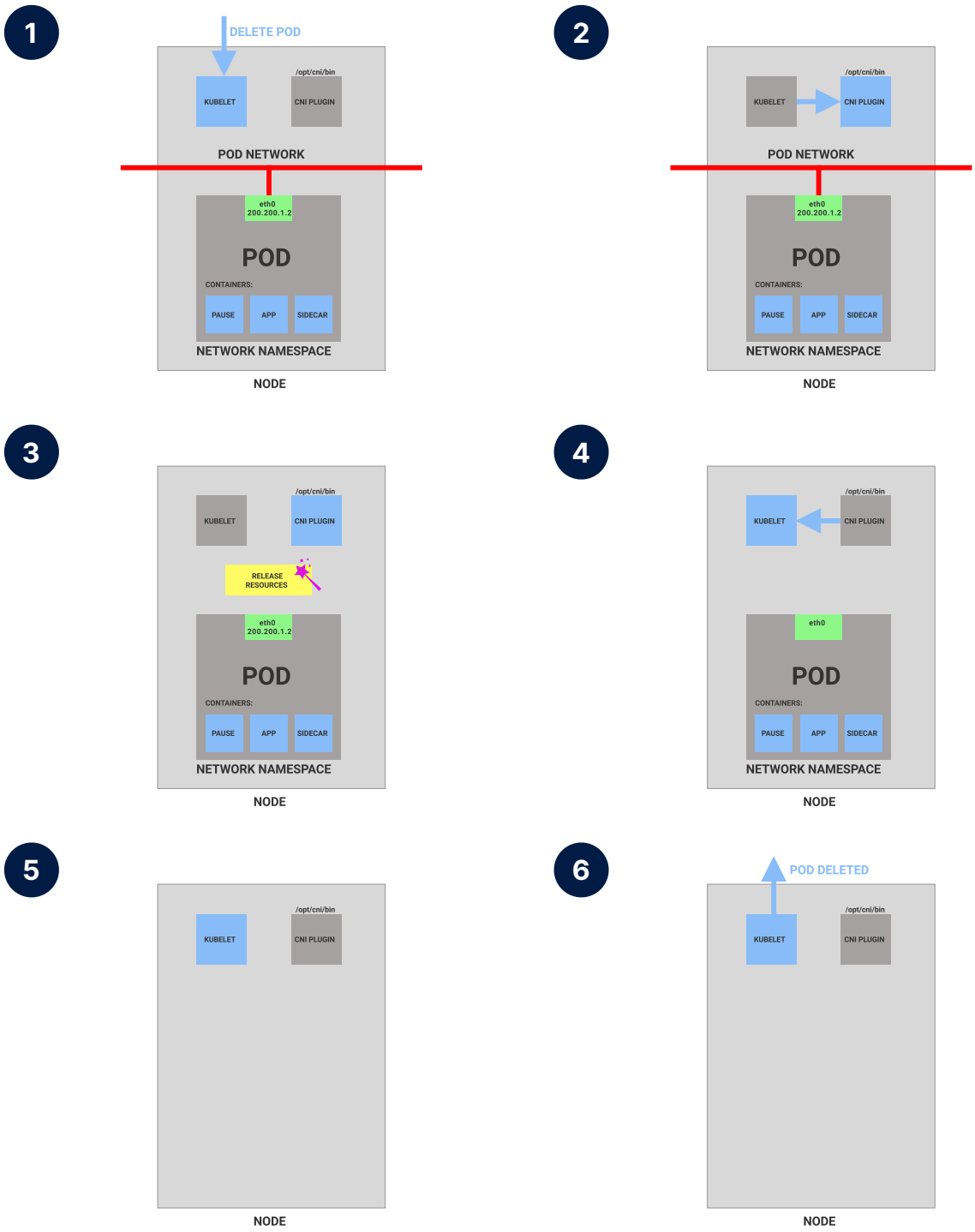


Fig. 1 The kubelet gets the order to delete an existing Pod.

Fig. 2 The kubelet executes the CNI plugin executable, passing it, among other parameters, the ID of the Pod to delete.

Fig. 3 The CNI plugin releases any resources that were allocated for the Pod, such as the Pod's IP address. .

Fig. 4 The CNI plugin terminates and hands control back to the kubelet.

Fig. 5 The kubelet kills all the containers of the Pod and deletes the network namespace. Deleting the network namespace automatically deletes all the networking resources in the network namespace, such as network interfaces, routing tables, and so on.

Fig. 6 The kubelet signalises the successful deletion of the Pod.

When the kubelet deletes a Pod, it first executes the CNI plugin by passing it, among other parameters, the ID of the Pod to delete.

The CNI plugin then releases any resources that are allocated for this Pod — this usually includes the Pod's IP address.

> Note that it is not necessary for the CNI plugin to delete the network resources that it created in the Pod's network namespace (such as the network interface) because these resources will be automatically deleted when the network namespace is deleted.

At this point, the CNI plugin terminates and control is handed back to the kubelet.

The kubelet now stops all the containers of the Pod and deletes the network namespace of the Pod.

If the deletion is successful, all the traces of the Pod are now removed.

The above has described the two basic operations of a CNI plugin from a bird's eye view.

Let's look next at the exact parameters that the kubelet passes to the CNI plugin.

Input and output

The CNI specification defines two types of input for a CNI plugin:

1. A set of environment variables
2. A JSON object (network configuration) streamed to `stdin`

The return value of the CNI plugin to the kubelet is a JSON object written to `stdout`.

Here's an overview:

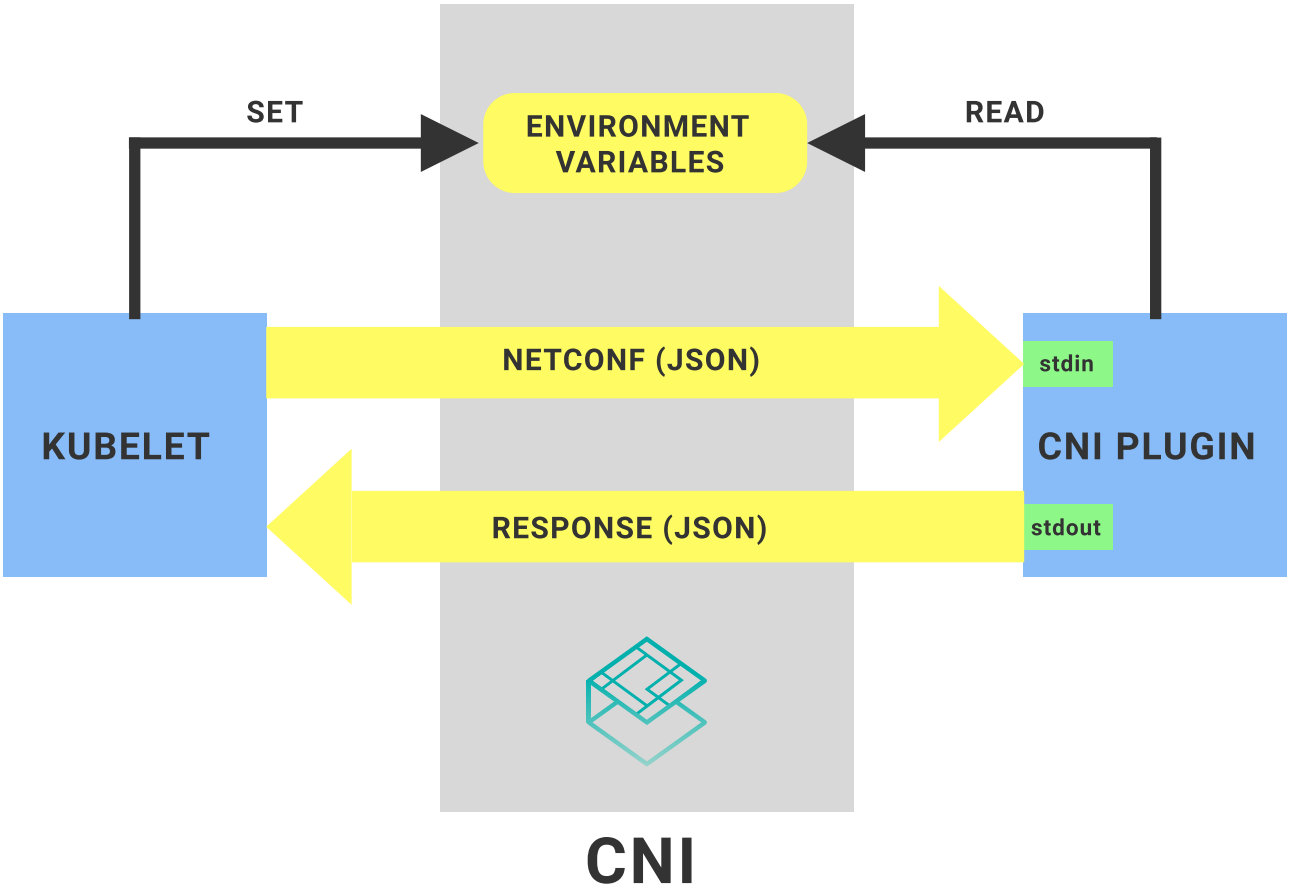


Fig. CNI

All these input and output parameters are well-defined in

the CNI specification, and we will look at them shortly.

But first, let's summarise the procedure of the kubelet invoking a CNI plugin (this applies to both the *create* and *delete* operations described above):

1. The kubelet sets the required environment variables
2. The kubelet executes the CNI Plugin executable and passes it the network configuration JSON to `stdin`
3. The CNI plugin reads both the environment variables and the network configuration JSON and does its job
4. The CNI plugin writes the response JSON to `stdout`
5. The kubelet reads the response JSON from the CNI plugin

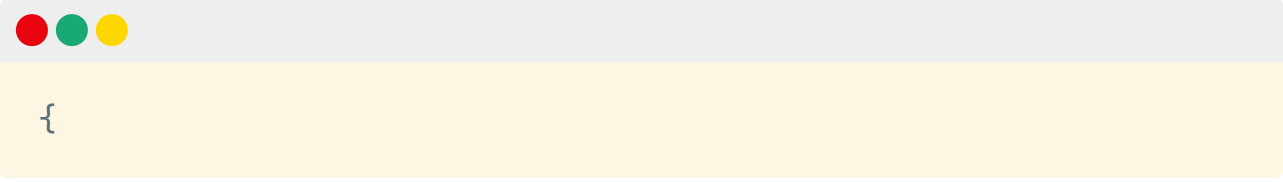
Now, let's look at each of these parameters in detail.

The network configuration (NetConf)

The network configuration (NetConf) JSON object that the kubelet passes to `stdin` of the CNI plugin is defined in the CNI specfication.

The purpose of the NetConf is to identify a CNI plugin and pass some basic static parameters to it.

A minimal example NetConf object looks like this:



```
  "cniVersion": "0.4.0",
  "name": "name-of-pod-network",
  "type": "name-of-cni-plugin"
}
```

The fields in this example are mandatory and must be present in every NetConf JSON object:

- `cniVersion` is the version of the CNI specification that this NetConf conforms to
- `name` is the name of the Pod network that the CNI plugin will attach any new Pods to
- `type` is the name of the CNI plugin executable

> The CNI specification allows multiple CNI plugins to be used at the same time, in which case a container/Pod will be attached to multiple container/Pod networks. For this reason, every container/Pod network must have a unique name, and this name is specified in the `name` field of the NetConf.

Beyond these mandatory fields, the CNI specification defines several optional NetConf fields.

The most important one of these optional fields is the `ipam` (IPAM stands for IP Address Management) field, whose meaning will be explained later in this section.

Beyond these optional fields, CNI plugins may accept custom fields in the NetConf.

The custom fields only have meaning to a specific CNI plugin that understands them, and they are not part of the CNI specification.

> When you will write your own CNI plugin in the next sections, you will use your own custom NetConf fields.

In Kubernetes, the NetConf has to be saved as a file on each node.

The default directory for NetConf files on a Kubernetes node is `/etc/cni/net.d`.

> This directory can be customised with the `--cni-conf-dir` kubelet flag.

Whenever the kubelet needs to invoke a CNI plugin, it checks this directory for a NetConf file, and it is actually from the `type` field of this NetConf file that the kubelet learns the name of the CNI plugin executable to execute.

Placing an appropriate NetConf file in `/etc/cni/net.d` of each node is an integral part of installing a CNI plugin.

The environment variables

The environment variables that the container runtime must set before invoking a CNI plugin are defined in the CNI specification.
In particular, these environment variables are:

- `CNI_COMMAND`
- `CNI_CONTAINERID`
- `CNI_NETNS`
- `CNI_IFNAME`
- `CNI_ARGS`
- `CNI_PATH`

Let's look at their meanings.
`CNI_COMMAND` contains the operation that the kubelet wants the CNI plugin to execute.
The CNI specification defines four operations for a CNI plugin to support: `ADD`, `DEL`, `CHECK`, and `VERSION`.

- The `ADD` and `DEL` operations correspond to the *create Pod* and *delete Pod* operations explained above
- The `CHECK` operation checks if a Pod is still attached to the Pod network correctly
- The `VERSION` operation returns the version of the CNI specification that this CNI plugin implements

In practice, this means, if the kubelet is creating a Pod, it will

set the `CNI_COMMAND` variable to `ADD`, if it is deleting a Pod, it will set it to `DEL`, and so on.

`CNI_CONTAINERID` contains an ID of the container (Pod, in the case of Kubernetes) that the present invocation of the CNI plugin applies to.

For example, when the kubelet deletes a Pod, it sets this variable to the ID of the Pod to delete, so that the CNI plugin knows which Pod this operation applies to.

`CNI_NETNS` contains a reference to the network namespace that the CNI plugin is supposed to work on.

For example, when the kubelet creates a Pod, it sets this variable to the path of the network namespace that it just created for the Pod.

> In Linux, network namespaces are referenced by a path of the form `/proc/<PID>/ns/net`, where `<PID>` is the process ID of the first process in the given network namespace. You can find details in the `namespaces (7)` man page.

`CNI_IFNAME` is the name of the network interface that the CNI plugin is supposed to create in the network namespace for a new Pod.

In Kubernetes, the default name for this network interface is `eth0`.

`CNI_ARGS` may contain additional custom arguments for the CNI plugin.

`CNI_PATH` contains a list of directories where CNI plugins are located.

Since in Kubernetes, the default directory for CNI plugin executables is `/opt/cni/bin` (customisable with the `--cni-bin-dir` kubelet flag), this variable will most of the time be set to `/opt/cni/bin`.

> You might ask, why does a CNI plugin need to know where CNI plugin executables are located? Well, CNI plugins can invoke other CNI plugins. This is called **CNI plugin chaining** and will be discussed later in this section.

So, now you know what parameters the kubelet passes to a CNI plugin.

To summarise, the kubelet sets the above environment variables, finds the NetConf file in `/etc/cni/net.d`, and executes the CNI plugin executable that it finds in `/opt/cni/bin` by streaming it the NetConf file to `stdin`.

At this point, the CNI plugin is running, and when it finishes, it must return a response to the kubelet.

Let's look at this response next.

The response

The response that the CNI plugin returns to the kubelet by writing it to `stdout` is defined in the CNI specification.

Its main purpose is to let the kubelet know the IP address that the CNI plugin set up for a new Pod.

This response is a JSON object with several mandatory and optional fields.

The mandatory fields are as follows:

```
{
  "cniVersion": "0.4.0",
  "interfaces": [],
  "ips": []
}
```

As you can expect, `cniVersion` is the version of the CNI specification that this response object conforms to.

The `interfaces` field is an array which lists all the network interfaces that the CNI plugin created in the Pod's network namespace.

Most of the time, this list contains only the single `eth0` interface that the CNI plugin created in a Pod's network namespace.

The `ips` field is also an array that contains the IP addresses corresponding to the network interfaces in the `interfaces` field.

Most of the time, this list consists only of the IP address that

the CNI plugin allocated for the `eth0` network interface of a new Pod.

By reading this response object, the kubelet learns everything it needs to know to finalise the creation of a Pod.

For example, the kubelet can now set the `pod.spec.podIP` field of the corresponding Pod object in etcd to advertise the IP address of the new Pod.

You should now have a good understanding of how the kubelet invokes a CNI plugin.

However, the CNI specification defines some additional features — one of these is CNI plugin chaining.

CNI plugin chaining

The kubelet (that is, the container runtime, in general) is not the only entity that can invoke CNI plugins.

CNI plugins themselves can invoke other CNI plugins.

This is called CNI plugin chaining.

The purpose of CNI plugin chaining is to allow for a modular architecture of CNI plugin functionality — you can define "helper plugins" that perform an isolated task, and other CNI plugins can then invoke these helper plugins.

The interface through which these CNI plugins are invoked

by other CNI plugins is the same as the one through which the kubelet calls any CNI plugin.

Here's an example:

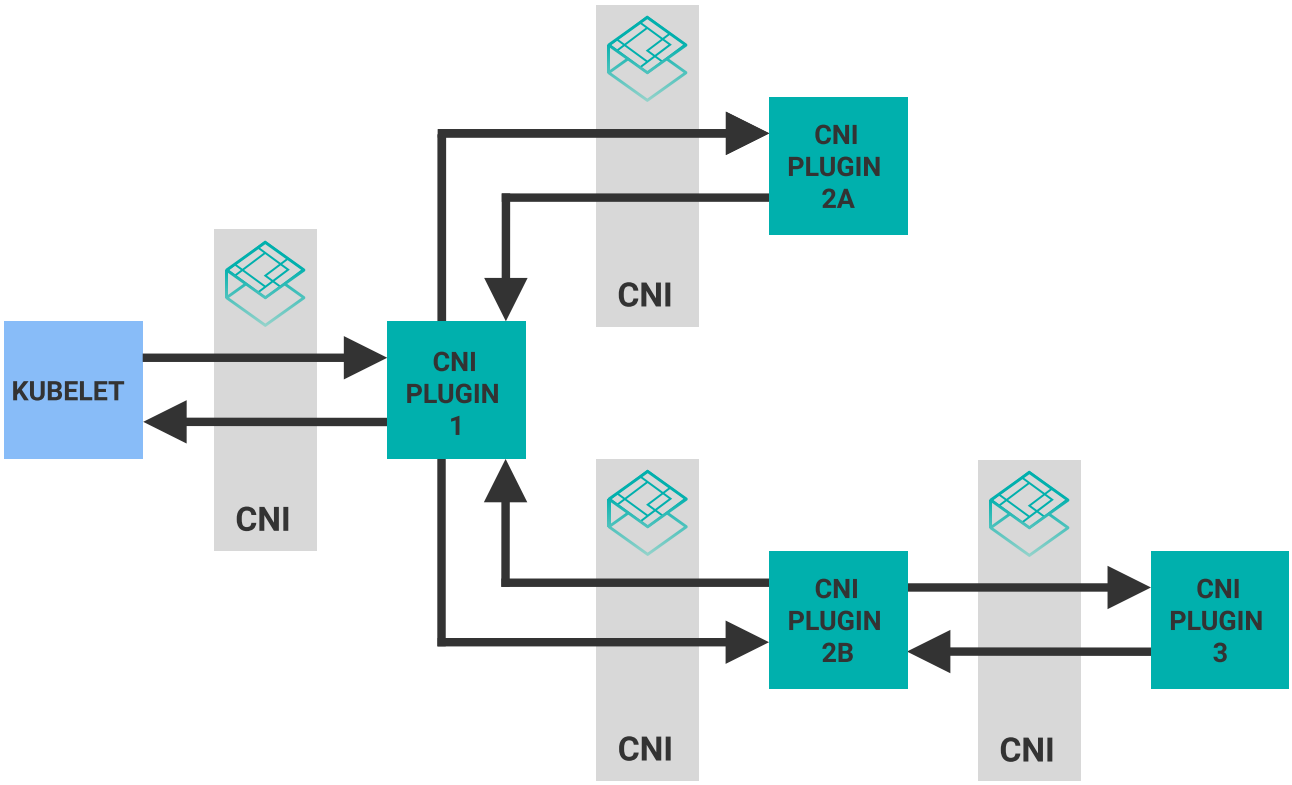


Fig. CNI plugin chaining

In the above example:

- The kubelet first invokes the main CNI plugin 1
- CNI plugin 1 then invokes a helper CNI plugin 2A and gets its response
- After that, CNI plugin 1 invokes another helper CNI plugin 2B

- CNI plugin 2B uses itself helper CNI plugin 3, which it invokes, gets its response, and finally returns its own response
- CNI plugin 1 now terminates too and returns its response to the kubelet

The above is just an example, and any other constellation is possible.

> CNI plugin chaining is the reason that the `CNI_PATH` environment variable is necessary. If a CNI plugin uses a helper CNI plugin, it must know the directory in which the executable of this helper plugin is located.

What's important to note is that the invocation of every CNI plugin happens through the same interface.

For example, CNI plugin 1 invokes CNI plugin 2A in the same way that it was itself invoked by the kubelet — namely, by providing environment variables and a NetConf and expecting an appropriate response as defined in the CNI specification.

> To call a helper CNI plugin, the calling CNI plugin can often reuse the parameters it got from its own caller. That is, CNI plugin 1 may just pass on the same NetConf and environment variables to CNI plugin 2A that it got from the kubelet. However, it is also allowed to make changes.

CNI plugin chaining allows to break out common sub-tasks as separate reusable CNI plugins that can be developed independently.

The benefits of this are a more modular architecture and higher code reusability.

One common sub-task that is very often implemented with CNI plugin chaining is IP address management (IPAM).

IPAM plugins

IP address management is the task of allocating IP addresses, keeping track of which IP addresses have been allocated, and releasing IP addresses.

IPAM is a necessary task for *every* CNI plugin.

For example, if a CNI plugin executes the `ADD` operation, it must choose a free IP address for the new Pod, and it must

"remember" across any future invocations that this IP address is in use to avoid allocating the same IP address twice.

Conversely, when the CNI plugin executes the `DEL` operation for the same Pod, it must remove its IP address from the list of allocated IP addresses so that the IP address is free again and may be allocated to another Pod.

This task is so common that most CNI plugins use a helper CNI plugin for it.

The CNI specification explicitly describes this type of helper CNI plugin and calls it IPAM plugin.

An IPAM plugin is usually invoked by a main CNI plugin for the sole task of allocating and maintaining IP addresses:

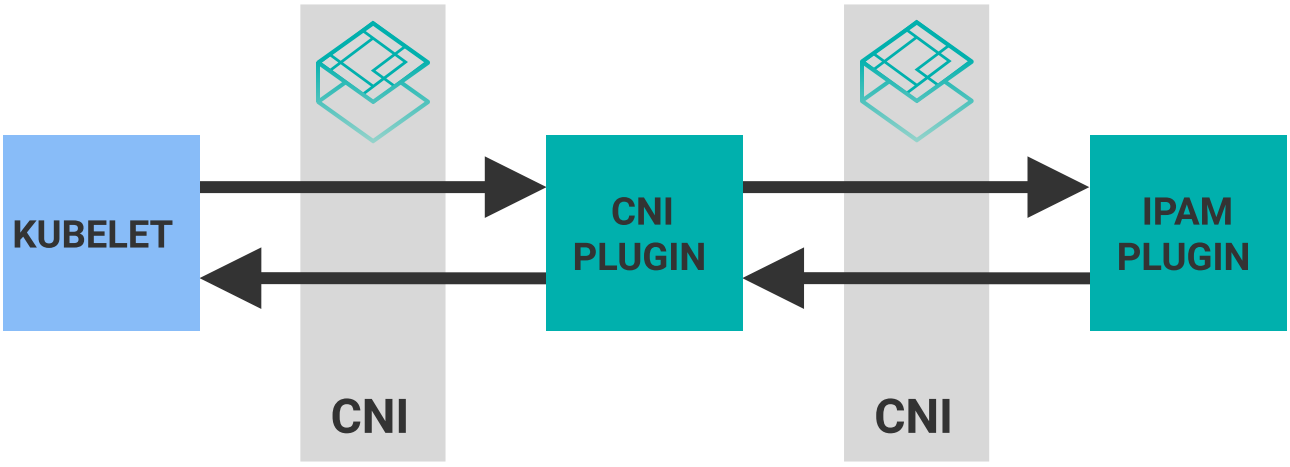


Fig. IPAM plugin

> The CNI plugin that you will write in a later section will use an IPAM plugin.

IPAM is such a common task that the CNI specification even defines a dedicated `ipam` field in the network configuration (NetConf) format.

The `ipam` field is optional and it allows to specify a specific IPAM plugin to use, as well as any custom parameters for this IPAM plugin.

An example NetConf with an `ipam` field looks like this:

```
{
    "cniVersion": "0.4.1",
    "name": "my-pod-network",
    "type": "my-cni-plugin",
    "ipam": {
        "type": "host-local",
        "subnet": "200.200.0.0/24"
    }
}
```

In this example, the `ipam.type` field is set to `host-local`, which is a hint to the main CNI plugin to use the `host-local` IPAM plugin.

Furthermore, the IP address range from which the IPAM plugin is supposed to allocate IP addresses is specified in the `ipam.subnet` field, which is a custom field understood by the `host-local` IPAM plugin.

This approach allows users to specify the IPAM plugin to use at runtime instead of it being hardcoded in the main CNI plugin — and the `ipam` field provides a common interface for users to provide IPAM-specific parameters.

Having said all that, here are some points to keep in mind about IPAM:

- An IPAM plugin is just a normal CNI helper plugin. It uses the same interface as any other CNI plugin and there's nothing special about it.
- The use of IPAM plugins and the `ipam` NetConf field are independent of each other. A CNI plugin may use an IPAM plugin without making use of the `ipam` NetConf field.
- The `ipam` field is just another field in the NetConf. A CNI plugin must understand and interpret the `ipam` field for it to have any effect. There are no intrinsic mechanisms associated with this field.

You should have now a good idea of how the CNI and CNI plugins work.

However, there is one last thing that you should know about the CNI project.

Example CNI plugins

The CNI project also provides a set of example CNI plugins in a separate GitHub repository.

What makes these CNI plugins important is that they are often used as building blocks for other CNI plugins by the means of CNI plugin chaining.

If you look at the list of reference CNI plugins in the project's GitHub repository, you can see that there are three groups of CNI plugins:

1. **Main CNI plugins:** `bridge`, `ipvlan`, etc.
2. **IPAM plugins:** `dhcp`, `host-local`, `static`
3. **Meta CNI plugins:** `tuning`, `firewall`, etc.

The main CNI plugins are those that are supposed to be invoked by the container runtime and do the bulk of the work.

IPAM plugins are helper CNI plugins that encapsulate a specific IP address management (IPAM) functionality, as described above.

Meta CNI plugins are CNI helper plugins that do miscellaneous other tasks.

What's important is that all these example CNI plugins are included by default in every Kubernetes distribution in `/opt/cni/bin` — this means that other CNI plugins can use them as CNI helper plugins by the means of CNI plugin chaining.

> You will use an IPAM plugin from the official example plugin as a CNI helper plugin in your own CNI plugin.

In this section, you learned what CNI plugins are and how they're used by Kubernetes.

In the previous section, you learned what the generic requirements of the Kubernetes Pod networking model are.

You should now be ready to become active and implement these Pod networking requirements with your own CNI plugin!

The next section will be the start of the lab that guides you through the design, implementation, installation, and testing of your own CNI plugin.

Let's get started!

Chapter 4

Lab 1/4 — Designing the CNI plugin

This is the start of a four-part lab that will guide you through the design, implementation, installation, and testing of your own CNI plugin.

- In the first part, you will layout the design of your CNI plugin
- In the second part, you will create the Kubernetes cluster on which you will install the CNI plugin
- In the third part, you will implement the CNI plugin
- In the fourth part, you will install and test the CNI plugin

By the end of this lab, you will have a fully functional Kubernetes cluster that runs your own CNI plugin.

Let's get started!

The 'bridge' CNI plugin

How do you go about writing a CNI plugin?

The best way is to look at an existing CNI plugin.

One of the most widely used CNI plugins is the `bridge` CNI plugin from the official example CNI plugins.

> All the official example CNI plugins are maintained in a dedicatd GitHub repository.

You will base the design of your CNI plugin on the `bridge` *CNI plugin, so let's see how it works.*

The `bridge` CNI plugin focuses on the connectivity between Pods on the same node and it uses a special network interface called a bridge to connect these Pods to each other.

A bridge, in the sense used here, is a virtual network interface on Linux that allows connecting multiple other network interfaces.

A message sent through a bridge is forwarded to all the connected network interfaces.

Let's see how the `bridge` CNI plugin uses a bridge to connect the Pods on the same node with an example:

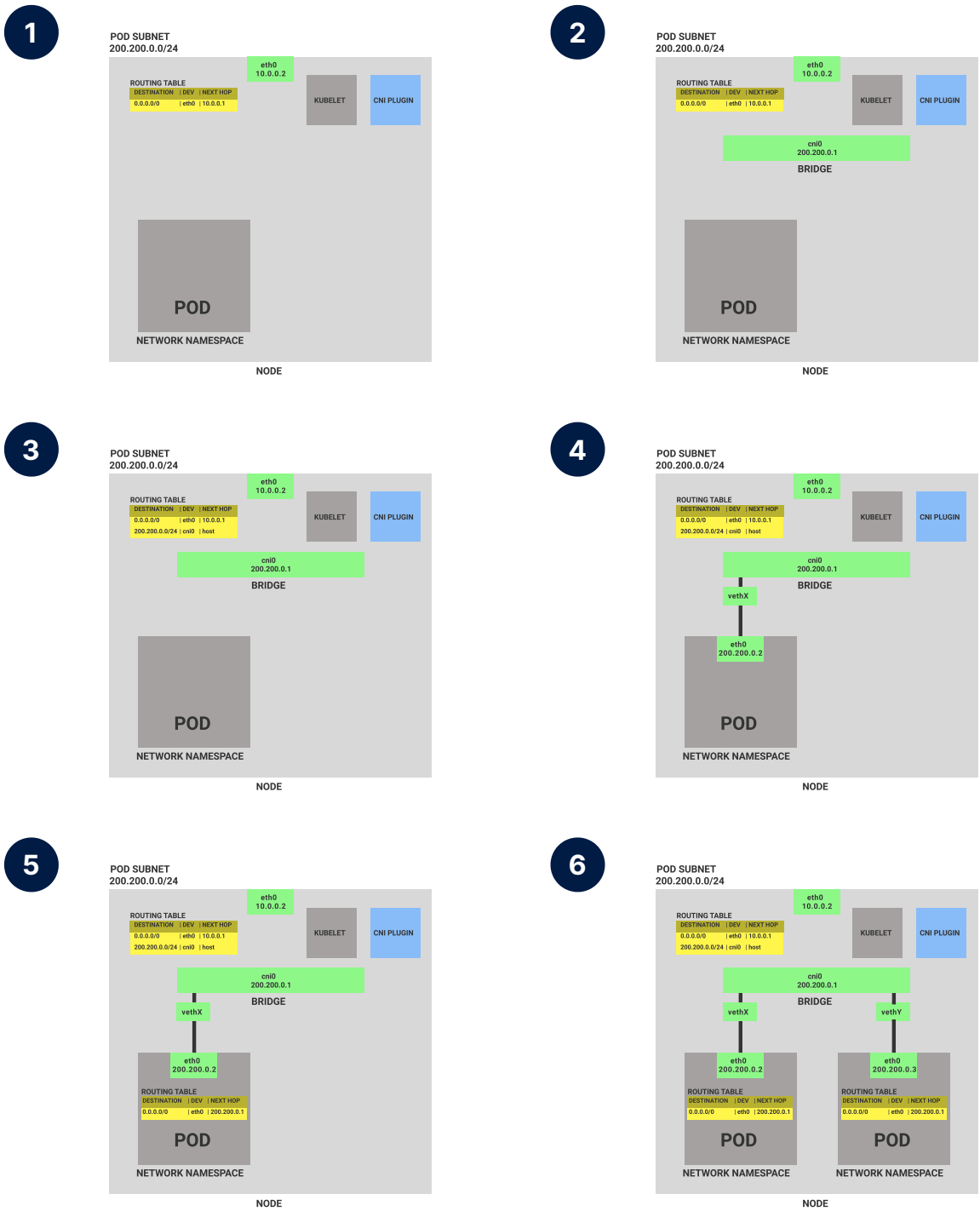


Fig. 1 The `bridge` CNI plugin starts running after the kubelet created a network namespace for a new Pod.

Fig. 2 The CNI plugin starts by creating a special network interface called a *bridge* in the default network namespace of the node and assigns it an IP address from the node's Pod subnet. The bridge is only created on the very first invocation of the CNI plugin.

Fig. 3 The CNI plugin also makes sure that there is a route in the node's default network namespace that sends out packets for a Pod on the node through the bridge network interface. From the point of view of the node, the bridge is the gateway to its Pod subnet.

Fig. 4 The CNI plugin creates an `eth0` network interface in the network namespace of the Pod, assigns it an IP address from the Pod subnet, and connects it to the bridge. The connection to the bridge is achieved by using a veth pair with one end of the pair in the Pod network namespace and the other in the node's default namespace.

Fig. 5 The CNI plugin creates a route in the Pod network namespace that forwards all packets to the bridge in the node's default network namespace. The bridge acts as the default gateway for all the Pods on the node.

Fig. 6 The CNI plugin repeats this procedure for any other Pod that is created.

In the above example, a new Pod is being created on a node which has allocated the 200.200.0.0/24 Pod subnet.

The kubelet already created a new network namespace for the Pod and now invokes the `bridge` CNI plugin.

When the `bridge` CNI plugin is invoked by the kubelet, it creates a bridge in the default network namespace of the node — in the example, it is called `cni0`.

The CNI plugin also assigns an IP address from the Pod subnet to this bridge — in the example, this is 200.200.0.1.

Furthermore, the CNI plugin creates a route in the default network namespace of the node.

This route applies to packets destined to the node's Pod subnet, and the target network interface is the bridge (`cni0`).

The effect is that all packets sent or forwarded by the default network namespace and destined to the Pod subnet are sent out through the bridge (to which the Pods will be connected).

At this point, you can look at the default network namespace of the node as a host with two network interfaces:

- The physical `eth0` interface connected to the host network
- The virtual `cni0` bridge connected to the Pod subnet

The routing logic is set up so that packets for a Pod are sent

out through the `cni0`, and all other packets are sent out through `eth0`.

> Note that the above steps are only performed on the very first invocation of the CNI plugin, that is, when the bridge hasn't been created yet.

With the bridge set up, the CNI plugin proceeds to set up the network namespace of the new Pod.

Like every CNI plugin, the `bridge` CNI plugin must create a network interface in the network namespace of the Pod — in the example, it is called `eth0`.

In the case of the `bridge` CNI plugin, this network interface must additionally be plugged into the bridge.

However, the bridge is in the node's default network namespace, and, in Linux, it is not possible to directly connect network interfaces across network namespaces.

To work around this issue, the `bridge` plugin uses another special virtual network interface called a veth pair.

A veth pair consists of two directly connected network interfaces — whatever is sent out through one end of the pair is immediately received by the other end.

Furthermore, and most importantly, the two ends of a veth pair may be in different network namespaces.

> The main purpose of veth pairs is actually to create connections between different network namespaces.

So, to connect the Pod to the bridge, the CNI plugin creates a veth pair and moves one end into the Pod's network namespace and the other into the node's default network namespace.

The end in the node's default network namespace is given a random name and connected to the bridge.

The end in the Pod network namespace is named `eth0` and assigned an IP address from the Pod subnet — in the example, this is 200.200.0.2.

Now the Pod is effectively connected to the bridge.

If the Pod sends a packet through its `eth0` interface, it is immediately received by the opposite end of the veth pair, and since the latter one is plugged into the bridge, the packet reaches the bridge.

The same applies in the other direction — if the bridge receives a packet, it reaches the Pod's `eth0` interface through the opposite end of the veth pair.

As the last step, the CNI plugin creates a route in the Pod's network namespace.

This route is a default route (that is, a route matching all packets) using the IP address of the bridge as the default gateway.

That's it for the setup of this Pod.

The CNI plugin repeats these steps (minus the setup of the bridge) for any other new Pods.

This is how the `bridge` CNI plugin works, and in your own CNI plugin, you will use the same approach to connect the Pods on a node to each other.

However, to do so, you should be sure that this approach really connects the Pods with each other.

So, let's verify that.

Tracing a packet

If the setup done by the `bridge` CNI plugin is correct, then Pods on the same node must be able to send packets to each other.

The following verifies this by playing through a scenario of a Pod sending a packet to another Pod:

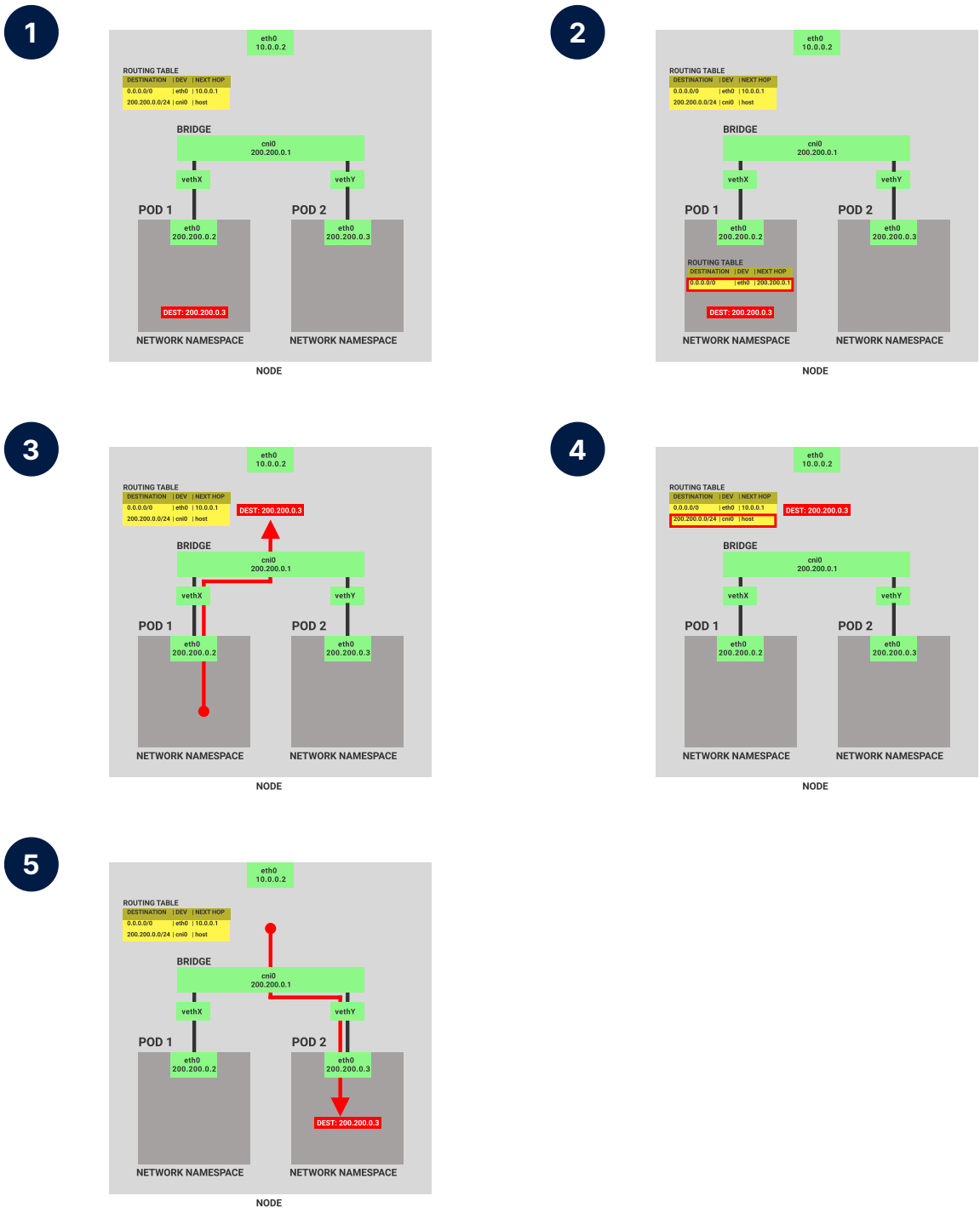


Fig. 1 Pod 1 with IP address 200.200.0.2 wants to send a packet to Pod 2 with IP address 200.200.0.3.

Fig. 2 To send the packet, Pod 1 checks its routing table and finds a default route 200.200.0.1, which is the bridge in the node's default network namespace.

Fig. 3 Pod 1 wraps the packet into a network frame, addresses it with the MAC address of 200.200.0.1 (the bridge), and sends it out through its `eth0` interface. The packet is received by the networking stack of the node's default network namespace through the `cni0` bridge interface.

Fig. 4 The network stack checks its routing table to decide what to do with the packet and finds a route that matches the 200.200.0.3 destination of the packet. The network interface of this route is `cni0` (the bridge) and the next hop is the final receiver.

Fig. 5 The network stack wraps the packet into a network frame, addresses it with the MAC address of 200.200.0.3, and sends it out through the bridge. Since Pod 2 is connected to the bridge, the packet is indeed received by Pod 2.

In the above scenario, two Pods are running on the same node:

- Pod 1 with IP address 200.200.0.2

- Pod 2 with IP address 200.200.0.3

Pod 1 wants to send a message to Pod 2, so it creates an IP packet with a destination IP address of 200.200.0.3.
To know where to send this packet, Pod 1 checks its routing table and finds a default route to 200.200.0.1, which is the IP address of the bridge.

> This is the route that was added by the CNI plugin.

So, Pod 1 wraps the IP packet into a network frame, addresses it with the MAC address of 200.200.0.1 (which it finds out through ARP), and sends it out through its `eth0` interface.
Since the Pod is connected to the bridge, the packet is received by the `cni0` interface of the default network namespace.
The network stack of the default namespace inspects the packet and, since it is intended for another destination (200.200.0.3), decides to forward it.
To know where to forward the packet to, the default network namespace checks its routing table and finds a route that matches the destination IP address of the packet.

> This is the route that was added by the CNI plugin.

The route uses the `cni0` network interface, and the next hop is the final receiver (that is, it is a directly-attached route).

So, the default network namespace wraps the packet into a network frame, addresses it with the MAC address of 200.200.0.3, and sends it out through the `cni0` bridge network interface.

Since a bridge sends packets to all connected network interfaces, the packet is also sent to the `eth0` interface of Pod 2.

That means Pod 2 receives the packet from Pod 1.

So, the design of the `bridge` CNI plugin seems to be correct — Pods on the same node can communicate with each other.

Note how all the settings made by the CNI plugin play role.

For example, the route created in the Pod network namespace instructs the Pod to use the default network namespace (via the `cni0` bridge interface) as the default gateway.

This default gateway function of the default network namespace is then configured with the route that the CNI plugin adds to the default network namespace.

You now know that communication between Pods on the same node works with this approach.

But what about Pods on different nodes?

Inter-node communication

The `bridge` CNI plugin actually doesn't address communication between Pods on different nodes at all.

If you use the `bridge` CNI plugin on a cluster with more than one node, then Pods on different nodes can't communicate with each other — this violates the Kubernetes Pod networking requirements.

That's a severe limitation of the `bridge` CNI plugin.

And you need to fix this in your own CNI plugin.

> The `bridge` CNI plugin still has its justification, for example, as a building block for other CNI plugins. A CNI plugin can invoke the `bridge` CNI plugin as a helper CNI plugin and then augment the `bridge` setup with its own settings. In fact, many real-world CNI plugins use the `bridge` CNI plugin under the hood.

So, how could you provide connectivity between Pods on different nodes?

With the the `bridge` CNI plugin, every packet sent by a Pod is first received by the default network namespace of the node that the Pod is running on (as explained above).

Each default network namespace is configured to forward packets to Pods on its own node (through the route that the CNI plugin created).

For example, if the default network namespace of node *A* receives a packet for Pod *A* (on node *A*), then it just forwards it to Pod *A*.

On the other hand, if the default network namespace of node *A* receives a packet for Pod *B* (on node *B*), then it doesn't know how to forward it to Pod *B*.

However, the default network namespace of node *B* knows how to forward packets to Pod *B*.

So, to solve the problem, the default network namespace of node *A* can send the packet to the default network namespace of node *B*.

When the default network namespace of node *B* receives the packet, it performs the same usual local forwarding to Pod *B* as it always does.

For a default network namespace on a node, it doesn't matter whether a packet to a local Pod arrives from outside the node (e.g. through the `eth0` interface) or from inside the node (e.g. through the `cni0` bridge interface) — it just forwards the packet to the local Pod.

Thus, the problem of sending packets between Pods on different nodes is reduced to sending packets between nodes.

And this can be configured with routing.

In particular, you need one route for each node that directs

packets for Pods *on* that node *to* that node.

Here's an example scenario with three nodes:

- Node 1: IP address 10.0.0.2, Pod subnet 200.200.0.0/24
- Node 2: IP address 10.0.0.3, Pod subnet 200.200.1.0/24
- Node 3: IP address 10.0.0.4, Pod subnet 200.200.2.0/24

The routes for providing inter-node Pod-to-Pod communication in the above scenario are as follows:

DESTINATION	NEXT HOP
200.200.0.0/24	10.0.0.2
200.200.1.0/24	10.0.0.3
200.200.2.0/24	10.0.0.4

Fig. Inter-node communication routes

As you can see, there is one route for each node that has the node's Pod subnet IP address range in the *Destination* field and the node's IP address in the *Next hop* field.

The effect is that any packet destined to a Pod on a given node is forwarded to the physical network interface of that node, which is in the default network namespace.

And, as mentioned, the default network namespace on that node will then locally deliver the packet to the destination Pod.

So, where should you install these routes?

One possibility is in the default gateway of the network that the nodes are part of.

The default gateway is the device that the nodes send all the packets for which they don't have any more specific routing rules to.

If all packets for Pods on different nodes end up in the default gateway, the default gateway can then forward them to the correct node.

Here is an example of this approach:

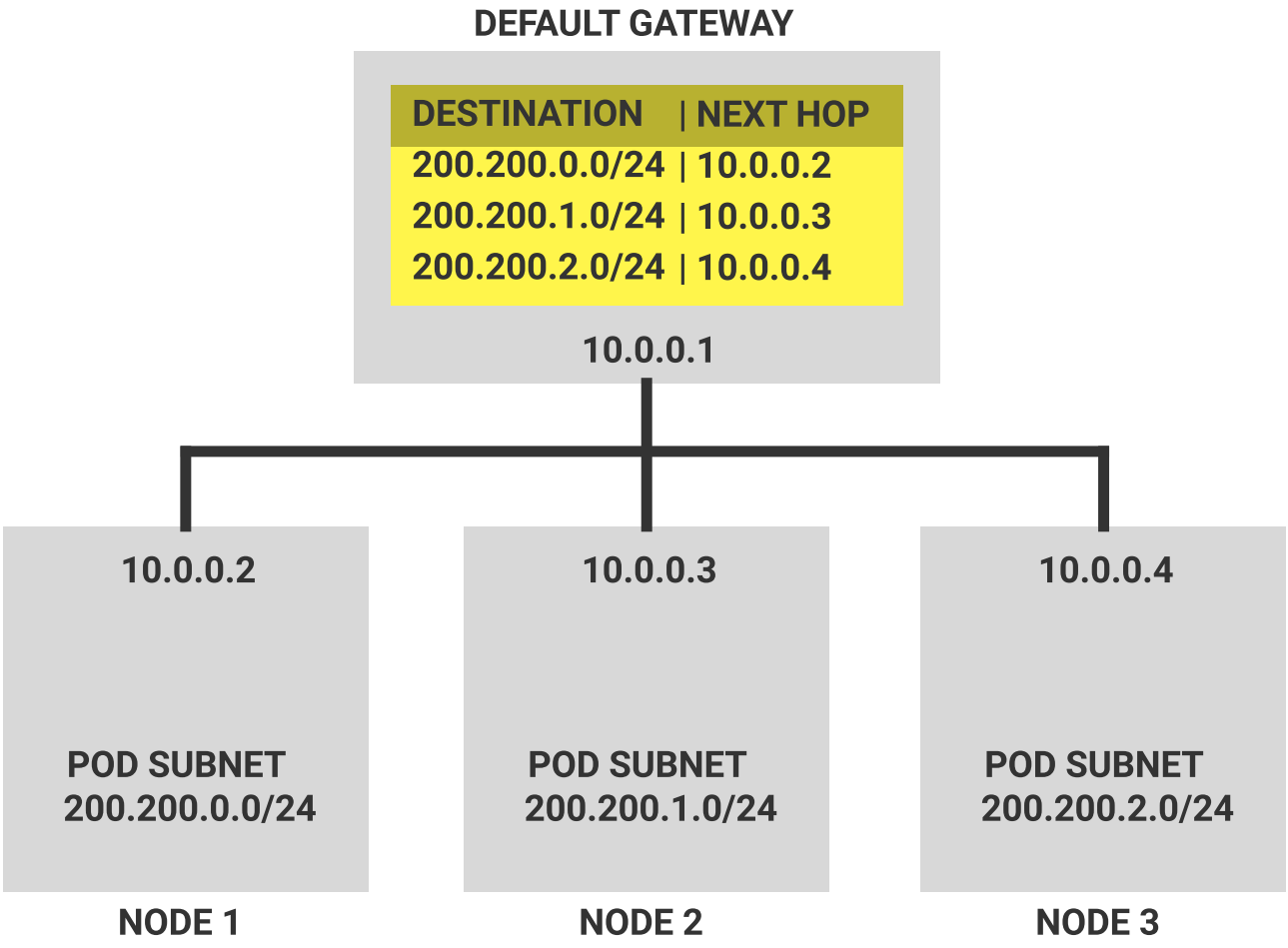


Fig. Inter-node communication routes

In the above example, if a Pod on node 1 wants to send a packet to a Pod on node 3 (say, 200.200.2.2), the packet is first sent to the default gateway (because node 1 doesn't have any specific routing rules for this destination IP address), and the default gateway then forwards it to 10.0.0.4 according to its routing rules, which is the network interface of node 3.

The solution with the default gateway is particularly suited

for the cloud where the nodes of a cloud network are often not physically connected to each other but always communicate through a gateway.

On a traditional network where the nodes are physically connected to each other, the routes could also be installed directly on each node — however, in that case, every node must only have the routes to all *other* nodes, but not the route to itself.

Since you will use your CNI plugin on a cluster on cloud infrastructure where the nodes are not physically connected to each other, you will use the default gateway approach.

To be sure that this really works, let's walk through an end-to-end example that combines the setup of the `bridge` CNI plugin with the approach to inter-node communication that you just figured out above:

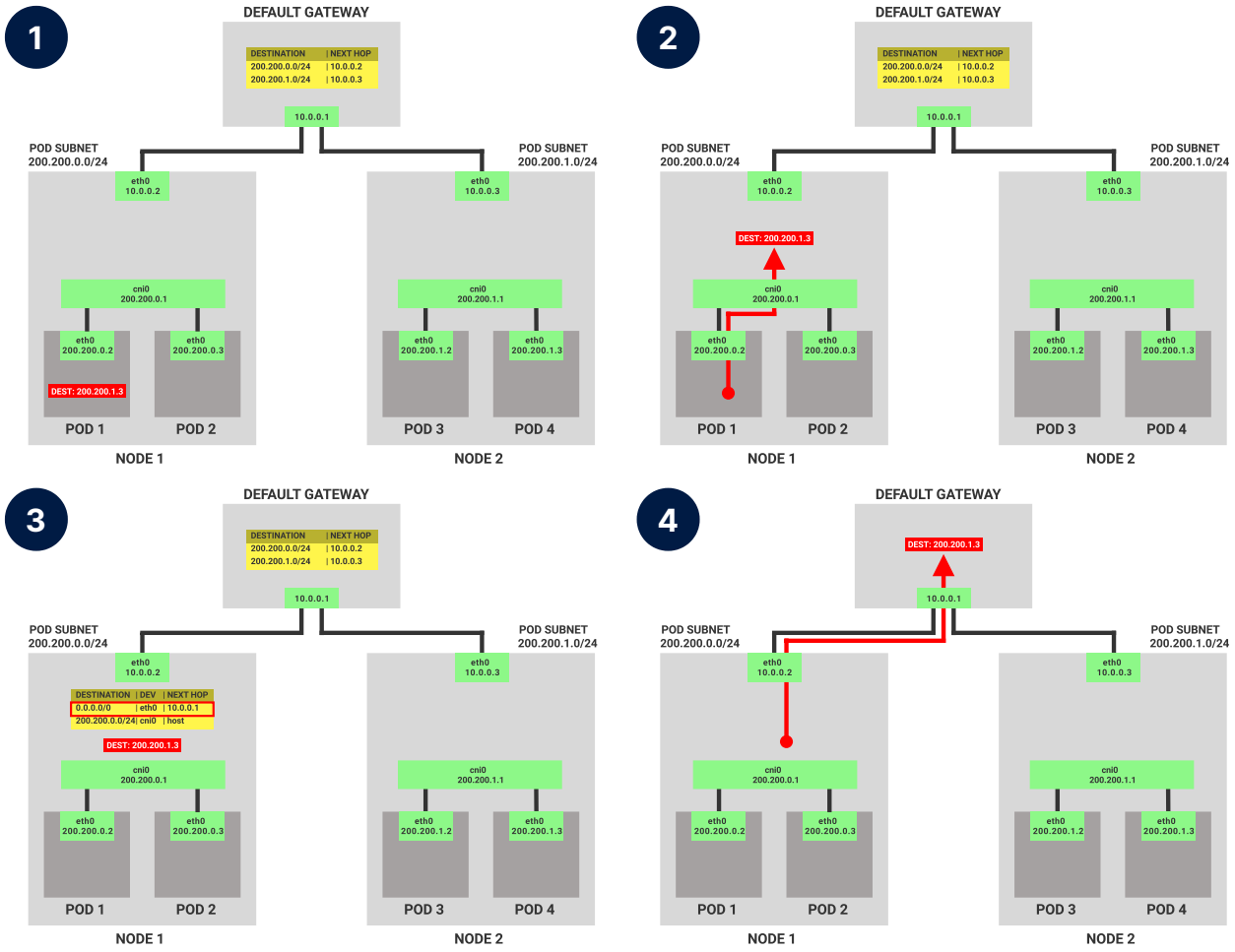


Fig. 1 In this scenario, Pod 1 with IP address 200.200.0.2 on node 1 wants to send a packet to Pod 4 with IP address 200.200.1.3 on node 2.

Fig. 2 As Pods have only a single default route, Pod 1 sends the packet to the `cni0` bridge interface with IP address 200.200.0.1 in the default network namespace.

Fig. 3 The network stack of the default network namespace receives the packet and checks its routing table to decide what to do with it. The only matching route is the default route, which means forwarding the packet to the default gateway.

Fig. 4 The default network namespace of node 1 forwards the packet to the default gateway with IP address 10.0.0.1.

Once the packet reaches the gateway, it can progress through the second leg of the journey.

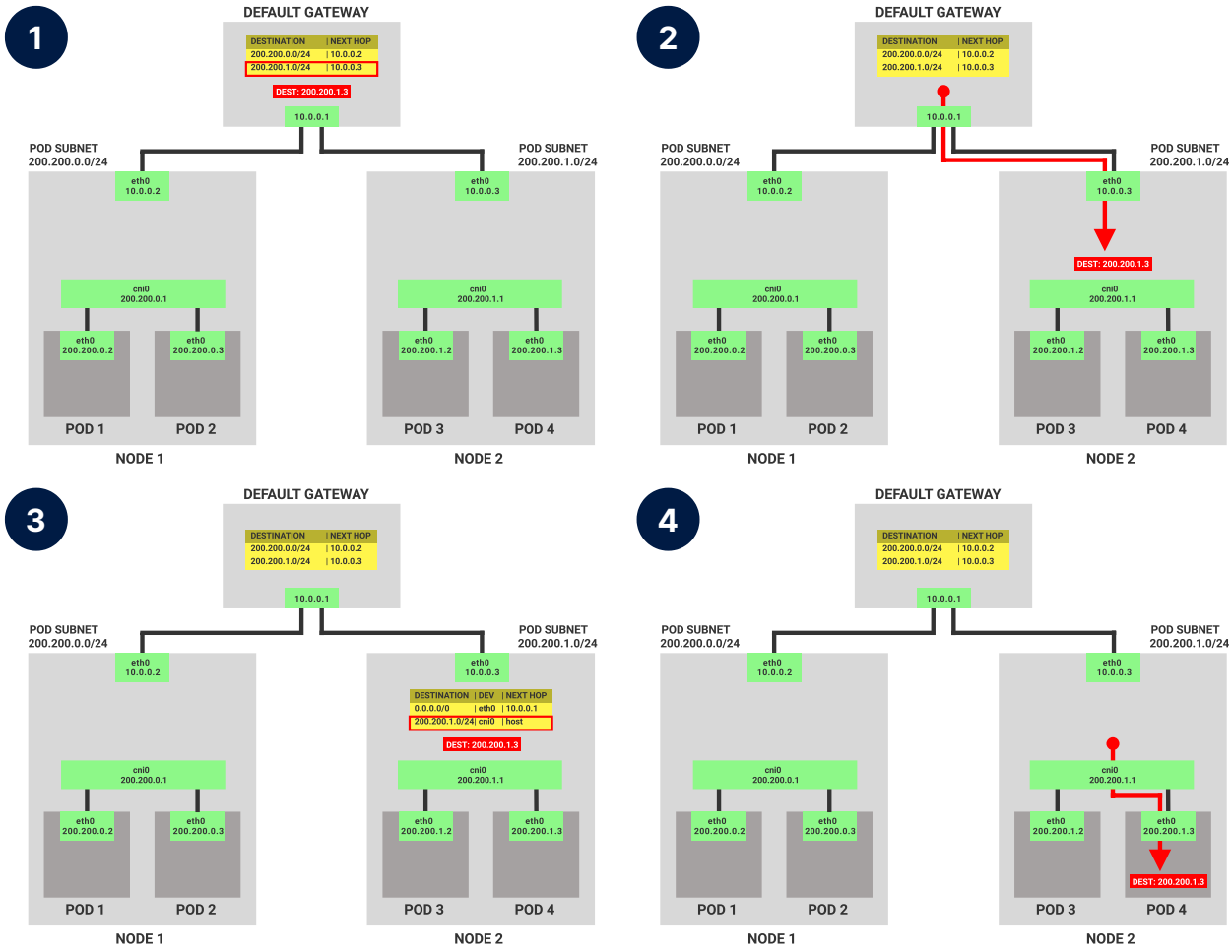


Fig. 1 The default gateway receives the packet and checks its routing table to decide what to do with it. It finds a matching route which instructs the packet to be forwarded to 10.0.0.3, which is the IP address of the `eth0` interface of node 2.

Fig. 2 The default gateway forwards the packet to 10.0.0.3, where it is received by the default network namespace of node 2.

Fig. 3 The default network namespace of node 2 checks its routing table to decide what to do with the packet. It finds a matching route that instructs the packet to be delivered via the `cni0` bridge interface directly to the destination Pod.

Fig. 4 The default network namespace of node 2 sends the packet out through the `cni0` bridge interface which causes it to be received by Pod 4's `eth0` interface. Pod 4 is receiving the packet from Pod 1!

In the above scenario, there are two nodes with four Pods:

- Node 1 has IP address 10.0.0.2 and Pod subnet 200.200.0.0/24 and runs Pod 1 (200.200.0.2) and Pod 2 (200.200.0.3)
- Node 2 has IP address 10.0.0.3 and Pod subnet 200.200.1.0/24 and runs Pod 1 (200.200.1.2) and Pod 2 (200.200.1.3)

The inter-node communication routes (as explained above) are installed in the default gateway of the network with IP address 10.0.0.1.

The example situation is that Pod 1 on node 1 wants to send a packet to Pod 4 on node 2.

To do so, Pod 1 addresses the packet with 200.200.1.3 (the

IP address of Pod 4) and sends it to 200.200.0.1, which is the `cni0` bridge interface in the node's default network namespace.

Remember from the explanations about the `bridge` *CNI plugin that Pods send all their packets to the bridge because they only have this single default route in their routing table — a Pod does _not* know whether the destination Pod is on the same or a different node._

The packet is received by the network stack of the default network namespace of node 1, which checks its routing table to decide what to do with the packet.

The only route that matches the packet is the default route of the default network namespace, which instructs to forward packets to 10.0.0.1, which is the default gateway of the network.

So, the default network namespace of node 1 sends the packet to the default gateway.

The default gateway receives the packet and checks its routing table to decide what to do with it.

It finds a matching route — namely the route that forwards packets destined to 200.200.1.0/24, which is the Pod subnet of node 2, to 10.0.0.3, which is the IP address of node 2.

Note that this is one of the routes that you installed in the default gateway to provide inter-node Pod-to-Pod communication.

So, the default gateway forwards the packet to 10.0.0.3,

which is the physical `eth0` network interface of node 2.

The packet is received by the default network namespace of node 2, which checks its routing table to decide what to do with it.

The packet matches with the route that instructs packets with a destination IP address in 200.200.1.0/24 (which is the Pod subnet of node 2) to be delivered directly to the receiver through the `cni0` bridge interface.

Note that this is the route installed by the `bridge` *CNI plugin.*

So, the default network namespace sends the packet out through the bridge interface.

Since the `eth0` network interface of Pod 4 is connected to the bridge, it receives the packet, and Pod 4 recognises it as te intended receiver of the packet.

Pod 4 successfully received the packet from Pod 1.

This means that the inter-node Pod-to-Pod communication scheme that you designed works!

The CNI plugin that you will develop will be a combination of the two techniques that you learned in this section:

- Intra-node Pod-to-Pod communication will be solved like in the `bridge` CNI plugin
- Inter-node Pod-to-Pod communication will be solved with the scheme just described above

You should now have a good idea of how your CNI plugin

will work.

In the next section, you will create the Kubernetes cluster on which you will use your CNI plugin.

Chapter 5

Lab 2/4 — Creating the cluster

In this part of the lab, you will create the Kubernetes cluster on which you will use your CNI plugin.

You will create this cluster from scratch on Google Cloud Platform (GCP) infrastructure with a tool called kubeadm.

> **Warning:** in this lab you will use GCP resources that cost about USD 0.27 per hour, which is is about USD 6.50 per day. If you don't want to spend any money, you can create a new GCP account, which gives you USD 300 credits that you can use for this lab. In any case, it's important that you delete all the resources when you're done with the lab to avoid any unnecessary costs. You can find the instructions for how to do this at the end of the lab.

Why creating a cluster yourself and not just using a managed Kubernetes service?

The reason is that clusters from managed Kubernetes services already have a CNI plugin preinstalled.

On the other hand, if you create a cluster with kubeadm, it has no CNI plugin installed, which is the perfect starting point for you to install your own CNI plugin.

This part of the lab consists of the following sections:

1. Network planning
2. Setting up the GCP command-line tool

3. Launching the infrastructure
4. Installing Kubernetes

By the end of this lab, you will have a Kubernetes cluster on GCP that you can access from your local machine with kubectl.

Let's get started!

Network planning

Before you launch any infrastructure, you need to have an idea about the network topology that you want to use for your cluster.

As you have learnt in a previous section, there are two logical networks in a Kubernetes cluster, the host network and the Pod network:

- The **host network** consists of all the physical nodes of the cluster
- The **Pod network** consists of all the Pods of the cluster

When you create a Kubernetes cluster, you need to define an IP address range for both of these networks.

These IP address ranges are completely independent of each other - they don't need to (and often shouldn't) overlap.

So, what should you choose?

The IP address range of the **host network** coincides with the GCP subnet in which you will create your cluster, and therefore you are constrained by GCP requirements.

On GCP, subnets must have *private IP address ranges*, which include 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16 as defined in IETF RFC 1918.

Given these constraints, you choose the following:

Host network: 10.0.0.0/16

> That means, your nodes will have IP addresses like 10.0.0.2, 10.0.0.3, and so on (10.0.0.1 is usually used for the default gateway of the subnet).

For the **Pod network**, you are completely free to choose any IP address range you like.

The only requirement is that, given the CNI plugin that you will build, it should *not* overlap with the host network IP address range.

In the previous examples of this course, 200.200.0.0/16 was usually used, and you will stick with this for your cluster, so:

Pod network: 200.200.0.0/16

> That means, your Pods will have IP addresses like 200.200.0.2, 200.200.0.3, and so on (200.200.0.1 will be used for the bridge in your specific CNI plugin).

These two parameters are very important and you will need them when you create the cluster in this section.

What about the Pod subnets that are assigned to all the nodes?

Do you have to define them too?

Fortunately not, because the Kubernetes controller manager will do this for you.

When you create your cluster, you will only supply the Pod network IP address to the controller manager, and the controller manager will allocate an appropriate subnet to each node.

Given that your Pod network IP address range is 200.200.0.0/16, the controller manager might assign 200.200.0.0/24 to node 1, 200.200.1.0/24 to node 2, and so on.

> The automatic assignment of Pod subnets to nodes can be controlled with the `--allocate-node-cidrs` command-line flag of the controller manager.

That's it for the network planning.
You're ready to jump into the GCP world!

Setting up the GCP command-line tool

In this section, you will set up the `gcloud` command-line tool that allows you to manage the GCP resources in your GCP account.

If you haven't yet `gcloud` installed, you can install it by installing the Google Cloud SDK according to the instructions in the GCP documentation.

Once the Google Cloud SDK is installed, you can initialise `gcloud` with the following command:

bash

```
$ gcloud init
```

This takes you through an interactive initialisation dialogue.

After you're done, you have to enable the Compute Engine API in your current GCP project:

bash

```
$ gcloud services enable compute.googleapis.com
```

Regardless of whether you just installed `gcloud` or you were already using it, it's important that you configure a default compute region and zone.
You can check if you have already configured a default region and zone with the following commands:

bash

```
$ gcloud config get-value compute/region
$ gcloud config get-value compute/zone
```

If this outputs a region and zone identifier, you're all set — if the output is empty, you can set a default region and zone wiht the following commands:

bash

```
$ gcloud config set compute/region europe-west6
$ gcloud config set compute/zone europe-west6-a
```

The region `europe-west6` and the zone `europe-west6-a` are taken as an example here — you can choose any region and zone you want (just make sure that the zone does actually belong to the region).

> You can find out all available regions and zones with the commands `gcloud compute regions list` and `gcloud compute zones list`.

Finally, you can verify that you have set up `gcloud` correctly by executing the following command:

```bash
$ gcloud compute instances list
```

If this command succeeds without an error, you're all set!

> As a pro tip, you may create a new GCP project specifically for this lab and set it as the default project. This will make it easier to delete all the resources after you're done with the lab because you can just delete the project. However, this is not mandatory and you can also use an existing GCP project that already has resources in it.

The next step is to launch the GCP resources that you will need for your cluster.

Launching the infrastructure

For running a Kubernetes cluster on GCP, you need to following types of resources:

- A virtual private cloud (VPC) network with a subnet
- A set of virtual machine (VM) instances
- A set of firewall rules

The functions of these resources are as follows:

- The **subnet** is the network in which your cluster will reside, and, in GCP, every subnet must be part of a **VPC network**.
- The **VM instances** will make up the nodes of your cluster — thus, you need one VM instance per node.
- The **firewall rules** are needed to allow certain types of traffic to your VM instances (by default, all incoming traffic is blocked)

> You can find a script named `infrastructure.sh` on the course's GitHub repository that automates the creation and deletion of all these GCP resources.

Let's start by creating the VPC network and subnet.

In your network planning above you decided to use the IP address range 10.0.0.0/16 for the host network of your cluster.

Since the subnet is the network where your VM instances, and thus your Kubernetes nodes, will reside, you have to use this IP address range for your subnet.

Here are the commands to do this:

```bash
$ gcloud compute networks create my-k8s-vpc --subnet-mode custom
$ gcloud compute networks subnets create my-k8s-subnet --network my-k8s-vpc --range 10.0.0.0/16
```

The first command above creates a VPC network named `my-k8s-vpc`.

The second command creates a subnet named `my-k8s-subnet` in this VPC network and it sets 10.0.0.0/16 as its IP address range.

Having set up the network, you can now create the VM instances.

Your cluster will be a simple one, consisting of a single master node and two worker nodes.

Consequently, you need to create three VM instances.

GCP provides a long list of different VM instances types, but for this lab, you will use:

- `n1-standard-2` for the master node
- `n1-standard-1` for the worker nodes

> The reason to use `n1-standard-2` for the master node is that Kubernetes requires a master node to have at least 2 CPUs, and `n1-standard-2` instances do have 2 CPUs whereas `n1-standard-1` instances do have only a single CPU.

Here is the command to create the `n1-standard-2` instance for the master node:

bash

```
$ gcloud compute instances create my-k8s-master \
    --machine-type n1-standard-2 \
    --subnet my-k8s-subnet \
    --image-family ubuntu-1804-lts \
    --image-project ubuntu-os-cloud \
    --can-ip-forward
```

And here the command to create the two `n1-standard-1` instances for the worker nodes.

bash

```
$ gcloud compute instances create my-k8s-worker-1 my-k8s-worker-2 \
    --machine-type n1-standard-1 \
    --subnet my-k8s-subnet \
    --image-family ubuntu-1804-lts \
    --image-project ubuntu-os-cloud \
    --can-ip-forward
```

After running these commands, you should have a VM

instance named `my-k8s-master` for the master node, and two VM instances named `my-k8s-worker-1` and `my-k8s-worker-2` for the worker nodes.

Note the following details about the above two commands:

- The `--subnet` flag causes these instances to be created in the subnet that you just created above
- The `--image-family` and `--image-project` flags set Ubuntu 18.04 as the operating systems running on the instances
- The `--can-ip-forward` flag allows the instances to send and receive packets with source or destination IP addresses that are *different* from their own IP address

> The `--can-ip-forward` flag is crucial for Kubernetes. A Pod has a different IP address than the node it's running on. Without the `--can-ip-forward` flag, a node would refuse to send a packet from a Pod to another node, because the packet has the Pod's IP address in the source field, which is different from the instance's IP address. For the same reason, an instance would refuse to receive a packet for one of its Pods.

You can verify that all the instances have been correctly created with the following command:

bash

```
$ gcloud compute instances list
NAME             ZONE            MACHINE_TYPE   PREEMPTIBLE  INTERNAL_IP  EXTERNAL_IP    STATUS
my-k8s-master    europe-west6-a  n1-standard-2               10.0.0.2     34.65.38.189   RUNNING
my-k8s-worker-1  europe-west6-a  n1-standard-1               10.0.0.4     34.65.192.69   RUNNING
my-k8s-worker-2  europe-west6-a  n1-standard-1               10.0.0.3     34.65.126.236  RUNNING
```

Note how each VM instance got an IP address from the 10.0.0.0/16 IP address range that you defined for your subnet.

These will be the IP addresses of the nodes in your Kubernetes cluster.

> The VM instances also have external IP addresses, however, they are only used for accessing the VM instances from outside the VPC network.

While your VM instances are now running, they cannot receive any traffic yet.

In GCP, you can think of all traffic to and from VM instances as having to pass through the default gateway of the subnet, and this default gateway applies firewall rules on the incoming and outgoing traffic.

The default firewall rules are to *block all incoming* and *allow all outgoing* traffic.

In practice, this means that your VM instances can *initiate* connections to everywhere, but they cannot *accept* connections from anywhere.

You have to change that by allowing your VM instances to accept those connections that are needed to create and operate a Kubernetes cluster.

On one hand, this includes certain types of traffic from *outside* the subnet, such as from your local machine:

- You need to be able to log in to the instances with **SSH** to install Kubernetes on them
- When Kubernetes is running, you need to be able to access the **API server** (for example, with kubectl)

Since SSH uses TCP port 22 and the Kubernetes API server uses TCP port 6443, these two types of traffic can be characterised as **TCP/22** and **TCP/6443**.

The following command creates a firewall rule that allows incoming TCP/22 and TCP/6443 traffic from any source:

bash

```
$ gcloud compute firewall-rules create my-k8s-ingress \
  --network my-k8s-vpc \
  --allow tcp:22,tcp:6443
```

After creating this firewall rule, you should be able to make TCP requests on ports 22 and 6443 to your VM instances (but any other types of connections will still be denied).

> The above firewall rule allows these types of traffic from everywhere, which includes the public Internet. This is okay since both SSH and the Kubernetes API server perform their own authentication.

In addition to traffic from *outside* the subnet, there are also some types of traffic from *inside* the subnet that should be allowed.

In particular, the individual nodes and Pods in your cluster will communicate with each other using various protocols and ports.

It's generally a safe strategy to allow *all* incoming traffic that originates from within the subnet.

So, how do you define the traffic that originates within the subnet?

Certainly if the source IP address is from the subnet IP address range, which in your case is 10.0.0.0/16.

But that's not enough.

Remember that Pods have their own IP addresses too.

A packet sent by a Pod has the Pod's IP address in the source field, and this packet must also be regarded as originating from within the subnet.

Since you selected 200.200.0.0/16 as the IP address range of the Pod network, the IP address ranges that define what's

internal to your subnet are **10.0.0.0/16** and **200.200.0.0/16**. The following command creates a firewall rule named `my-k8s-internal` that allows all incoming traffic that originates from within your subnet:

bash

```
$ gcloud compute firewall-rules create my-k8s-internal \
  --network my-k8s-vpc \
  --allow tcp,udp,icmp \
  --source-ranges 10.0.0.0/16,200.200.0.0/16
```

After creating the above firewall rule, your VM instances will accept all traffic from a source in one of the IP address ranges 10.0.0.0/16 and 200.200.0.0/16.

> If you wouldn't add the 200.200.0.0/16 IP address range, then all traffic from Pods (except the traffic allowed by the `my-k8s-ingress` rule) to different nodes of the cluster would be blocked.

That's it — that's all the GCP infrastructure you need to run a Kubernetes cluster.

The next step is to install Kubernetes.

Installing Kubernetes

In this step, you will install Kubernetes on the GCP infrastructure that you just created.

> You can find a script named `kubernetes.sh` on the course's GitHub repository that automates the installation and deinstallation of Kubernetes on your GCP infrastructure.

You will use kubeadm, which is a Kubernetes installer tool maintained by the Kubernetes project.

kubeadm runs directly on the nodes of the cluster, so you first have to install kubeadm on all of your VM instances.

You will do that by executing a small installer script on each VM instance.

The script looks as follows:

install-kubeadm.sh

```
#!/bin/bash

sudo apt-get update
sudo apt-get install -y docker.io apt-transport-https curl jq nmap
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
echo "deb https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubeadm
```

Go on and save this script in a file named `install-kubeadm.sh` on your local machine.

Then, you can use the following command to both upload and execute the script on all your VM instances:

```bash
$ for node in my-k8s-master my-k8s-worker-1 my-k8s-worker-2; do
  gcloud compute scp install-kubeadm.sh "$node":
  gcloud compute ssh "$node" --command "chmod +x install-kubeadm.sh"
  gcloud compute ssh "$node" --command "./install-kubeadm.sh"
done
```

After the command completes, you can verify that kubeadm has been correctly installed on all the VM instances with the following command:

```bash
$ for node in my-k8s-master my-k8s-worker-1 my-k8s-worker-2; do
  gcloud compute ssh "$node" --command "kubeadm version"
done
```

You have now kubeadm installed on all the VM instances, the next step is to use it to install Kubernetes.

But before you do that, you should retrieve the external IP address of your master node.

You will need this as a parameter for one of the kubeadm commands.

You can execute the following command to save the external

IP address of the master node in an environment variable:

```bash
$ MASTER_IP=$(gcloud compute instances describe my-k8s-master \
  --format='value(networkInterfaces[0].accessConfigs[0].natIP)')
```

Now, let's install Kubernetes!

kubeadm has two main commands:

- `kubeadm init` : run on the master node
- `kubeadm join` : run on the worker nodes

In the following, you will run a `kubeadm init` command on the master node, and a `kubeadm join` command on each worker nodes.

The `kubeadm init` command that you will execute looks like this (don't execute it yet):

```
kubeadm init --pod-network-cidr=200.200.0.0/16 --apiserver-cert-extra-sans="$MASTER_IP"
```

The most important argument of this command is the `--pod-network-cidr` flag.

It defines the desired IP address range of the Pod network in the cluster.

As you can see, you set this flag to 200.200.0.0/16, which is the Pod network IP address range that you defined in your

network planning.

The `--apiserver-cert-extra-sans` flag sets the master node's external IP address as an additional subject alternative name (SAN) in the Kubernetes API server certificate.

This is needed because when you will access the API server from your local machine, you will use the master node's *external* IP address rather than it's *internal* one, which is included by default in the API server certificate.

To execute the above `kubeadm init` command on the master node, you can use the following command:

bash

```
$ gcloud compute ssh my-k8s-master --command \
  "sudo kubeadm init --pod-network-cidr=200.200.0.0/16 --apiserver-cert-extra-sans=\"$MASTER_IP\""
```

> Please make sure that you have the `MASTER_IP` environment variable set to the external IP address of the master node, as explained above.

This command now installs the necessary Kubernetes components on the master node.

The very last part of the output should look somehow like this:

```
kubeadm join [IP]:6443 --token [TOKEN] --discovery-token-ca-cert-hash sha256:[HASH]
```

This is the `kubeadm join` command that you have to execute on all the worker nodes now.

To do so, copy the `kubeadm join` command *exactly as is* to your clipboard and insert it into the following two commands:

```bash
$ gcloud compute ssh root@my-k8s-worker-1 --command "<CMD>"
$ gcloud compute ssh root@my-k8s-worker-2 --command "<CMD>"
```

> Please replace `<CMD>` with the `kubeadm join` command from the output of `kubeadm init`.

These commands install the necessary Kubernetes components on the worker nodes and wire them up with the master node.

After these commands complete, you have a complete Kubernetes cluster.

That's how easy it is to install Kubernetes with kubeadm!

Your cluster is now up and running, but you want to be able to access it from your local machine.

To do so, you need a kubeconfig file.

Fortunately, kubeadm already created a kubeconfig file in `/etc/kubernetes/admin.conf` of the master node.

You can download this file to your local machine with the

following command:

```bash
$ gcloud compute scp root@my-k8s-master:/etc/kubernetes/admin.conf my-kubeconfig
```

The above command saves the kubeconfig file in a file named `my-kubeconfig` in your current working directory.

If you look into this kubeconfig file in the `clusters.cluster.server` section, you see that it uses the master node's internal IP address in the API server URL.

Since you're outside of the GCP subnet now, this won't work, and you have to replace it with the master node's external IP address that you saved in the `MASTER_IP` environment variable.

You can either make this change by hand, or use the following sed command:

```bash
$ sed -i -r "s#[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+:6443#$MASTER_IP:6443#" my-kubeconfig
```

Now, your kubeconfig file is ready to be used.

You should now be able to access your Kubernetes cluster.

Let's make the test:

```bash
```

```
$ kubectl get nodes --kubeconfig my-kubeconfig
NAME              STATUS     ROLES    AGE   VERSION
my-k8s-master     NotReady   master   36m   v1.17.0
my-k8s-worker-1   NotReady   <none>   32m   v1.17.0
my-k8s-worker-2   NotReady   <none>   31m   v1.17.0
```

Congratulations!

You just installed Kubernetes on your own GCP infrastructure and you're able to access the cluster from your local machine.

That's a great achievement!

One last thing that you should do is saving the kubeconfig file name in the `KUBECONFIG` :

bash

```
$ export KUBECONFIG=$(pwd)/my-kubeconfig
```

This allows you to access your cluster without having to specify the `--kubeconfig` flag in every kubectl command.

> Note that you should set this environment variable in every new terminal window in which you wish to work with your cluster.

Now that your cluster is running, let's have a closer look at it.

Inspecting the cluster

Is the cluster that you just created really complete?
Let's list the nodes again:

bash

```
$ kubectl get nodes
NAME              STATUS     ROLES    AGE   VERSION
my-k8s-master     NotReady   master   38m   v1.17.0
my-k8s-worker-1   NotReady   <none>   34m   v1.17.0
my-k8s-worker-2   NotReady   <none>   33m   v1.17.0
```

All the nodes are `NotReady`.

> Do not try to create any Pods now, or they will remain in the `ContainerCreating` status indefinitely.

So, why are the nodes `NotReady`?
To find out, check the details of one of the nodes:

bash

```
$ kubectl describe node my-k8s-worker-1
```

In the output, locate the *Conditions* section and therein the

`Ready` entry.

The `Ready` entry has a status of `False` with a reason of `KubeletNotReady`, and the associated message says something like:

```
runtime network not ready: NetworkReady=false reason: NetworkPluginNotReady
```

Bingo!

Your cluster doesn't yet have a CNI plugin installed.

As mentioned, kubeadm does *not* install a CNI plugin by default — it is up to you to do this.

What's happening now is that the kubelet tries to figure out the installed CNI plugin, can't find any and reports the `KubeletNotReady` status, which causes the entire node to be `NotReady`.

You're cluster isn't functional until you install a CNI plugin.

But that's actually good news for you because you anyway wanted to create and install your own CNI plugin.

You will do that in the next section.

Chapter 6

Lab 3/4 — Implementing the CNI plugin

In this section, you will implement the CNI plugin that you designed in the first part of the lab.

In the next section, you will then install this CNI plugin in the Kubernetes cluster that you created in the previous section.

For starters, let's recap how your CNI plugin works.

CNI plugin overview

As you know, in general, a CNI plugin is an executable file on each node of the cluster that is executed by the kubelet when a Pod is created or deleted.

When a Pod is created, the task of the CNI plugin is to connect the network namespace of the Pod to the Pod network — the network namespace is created by the kubelet, and the CNI plugin then performs all the necessary configurations to wire it up with the Pod network.

The following summarises how the CNI plugin that you designed in the first part of the lab solves the task of connecting Pods to the Pod network:

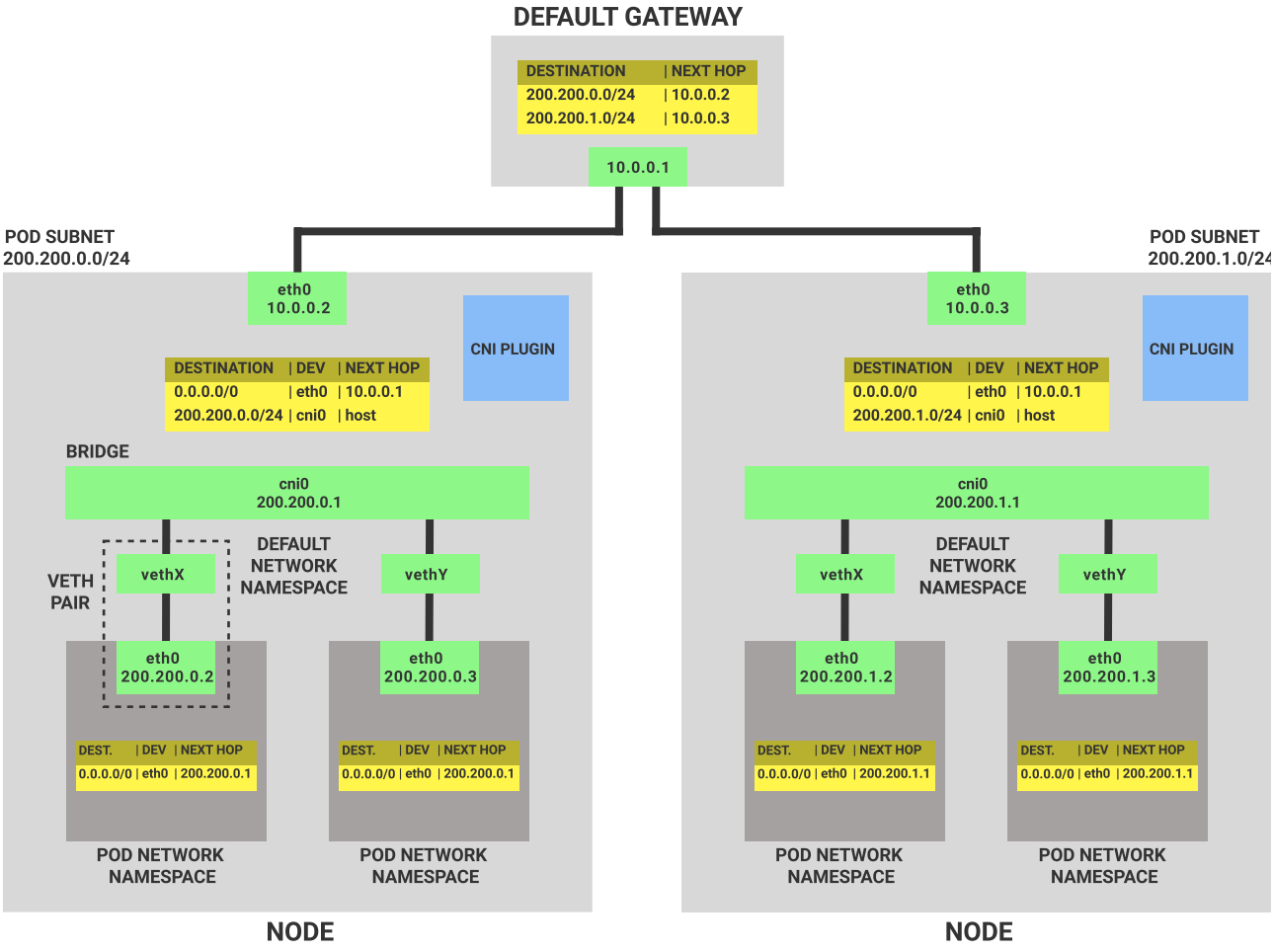


Fig. CNI plugin overview

At the heart of your CNI plugin design is a bridge network interface named `cni0` in the default network namespace of each node to which all the Pods of that node are connected through a veth pair.

The bridge works in IP mode, which means that it has an IP address and acts like an ordinary network interface of the default network namespace.

That means, everything sent from a Pod to the bridge is received and processed by the default network namespace, and everything sent by the default network namespace through the bridge is received by the Pods.

The bridge is configured as the default gateway of the Pods by the means of an appropriate route in each Pod's network namespace, and there is also a route in the default network namespace that directs all packets destined to the node's Pod subnet through the `cni0` bridge network interface.

All this enables *intra-node* Pod-to-Pod communication — when a Pod sends a packet to another Pod on the same node, the packet first travels via the bridge into the default network namespace, where it is then forwarded through the bridge again to the destination Pod.

To enable *inter-node* Pod-to-Pod communication, there are routes installed in the default gateway of the host network.

There's one route for each node in the default gateway — these routes forward packets destined to the Pod subnet of a node to the IP address of that node.

When a packet sent by a Pod to a Pod on a different node arrives like this on the destination node, it is then forwarded by the default network namespace of that node via its bridge to the destination Pod (in the same way that intra-node Pod-to-Pod packets are forwarded).

An important task of every CNI plugin is to allocate an IP address from the node's Pod subnet to the network interface

of a Pod network namespace.

This requires the CNI plugin to keep track of which IP addresses are already allocated and which are free — a task called IP address management (IPAM).

For your CNI plugin, you decided to not implement this task yourself, but to make use of a dedicated IPAM plugin — in particular, you will use the `host-local` IPAM plugin from the official example CNI plugins.

> An IPAM plugin is a non-standalone CNI plugin that selects and keeps track of IP addresses. An IPAM plugin cannot be used on its own, but it can be invoked as a helper CNI plugin by other CNI plugins.

Given that this is how your CNI plugin works, the following is a possible organisation of the tasks that your CNI plugin has to do:

1. Invoking the IPAM plugin
2. Doing the one-time setup
3. Doing the Pod-specific setup
4. Returning the response

Let's briefly discuss each of these tasks.

Invoking the IPAM plugin

Invoking the IPAM plugin is necessary for both Pod creation and Pod deletion — when a Pod is created, for selecting an IP address for this Pod, and when a Pod is deleted, for releasing the IP address of this Pod.

In the case of Pod creation, the IPAM plugin returns an IP address — the CNI plugin then uses this IP address later when it creates the main network interface in the Pod network namespace.

In general, invoking the IPAM plugin is one of the first tasks that a CNI plugin does.

> Note that an IPAM plugin does *not* do any network configurations by itself. It just returns and keeps track of IP addresses. It is up to the caller what to do with a returned IP address, such as assigning it to a network interface.

Doing the one-time setup

Certain configurations done by your CNI plugin have to be done only once during the lifetime of a node or cluster.

This includes the creation of the bridge and the inter-node communication routes.

There's only one bridge on each node, and thus the CNI

plugin has to create it only if it doesn't yet exist (which is typically during the very first invocation of the CNI plugin on a given node).

Furthermore, the inter-node communication routes in the default gateway exist only once per cluster, and thus they have to be created only by one single invocation of the CNI plugin across the entire cluster.

What's common to these tasks is that the CNI plugin must check whether they have already been done before and only do them if they haven't.

> As you will see later, the one-time-setup includes some additional packet filtering and network address translation (NAT) settings in the default network namespace of a node. You will learn more about this when you implement the CNI plugin.

Doing the Pod-specific setup

The remaining configurations done by your CNI plugin have to be carried out for every creation of a new Pod.

This includes most notably the creation of the veth pair which connects the Pod network namespace to the bridge.

One end of the veth pair (the host-end) has to be placed in the default network namespace and connected to the bridge.

The other end of the veth pair (the Pod-end) has to be

placed in the Pod network namespace and set up as the primary network interface of the Pod network namespace.
This includes giving it a standard name (`eth0`) and assigning it an IP address.

> This is the IP address that was previously selected by the IPAM plugin.

Finally, the CNI plugin also has to create a default route to the bridge in a new Pod network namespace so that the default network namespace acts as the default gateway for the Pods.

Returning the response

After the CNI plugin has done its job, it must return a response to the kubelet.
The structure of this response is prescribed by the CNI specification.
The primary information contained in this response is the name and the IP address of the network interface that the CNI plugin created in the Pod network namespace.
The kubelet will use this information to finalise the creation of the Pod — for example, it will write the returned IP address into the `pod.status.podIP` field of the corresponding Pod object in Kubernetes.

You should now have a basic idea of what you will implement.

Let's get started!

> Note that you can find the complete source code of the CNI plugin in the GitHub repository accompanying this course.

Preliminary notes

Before starting with the implementation, some preliminary notes about the programming language, CNI version, and input parameters are necessary.

Programming language

For the sake of this lab, you will write your CNI plugin in Bash.

Bash is a convenient choice for implementing a simple CNI plugin, because it allows to easily use common networking tools, such as `ip` and `iptables`.

However, you could also write this CNI plugin in any other programming language, such as Python, Java, or Go.

> You might consider translating this CNI plugin to your favourite programming language as an exercise for the future.

If you have to choose a programming language for writing a CNI plugin, you should also consider that in some languages there exist dedicated libraries for writing CNI plugins, such as the CNI library for Go.

CNI version

There exist multiple versions of the CNI specification, and you can find the latest one on the CNI GitHub repository.

For your particular CNI plugin, you will implement CNI version 0.3.1.

This is not the newest CNI version, but it is the newest version that is relatively simple to implement.

In particular, CNI version 0.3.1 defines only the three CNI operations, `ADD`, `DEL`, and `VERSION`, whereas the subsequent CNI version 0.4.0 additionally defines the `CHECK` operation.

The `ADD`, `DEL`, and `VERSION` operations are the most fundamental ones, and the `CHECK` operation is furthermore verbose to implement.

Thus, by using CNI version 0.3.1, you are freed from implementing the `CHECK` operation and can concentrate on

the core business with the other operations.

Input parameters

As you know, a CNI plugin takes two types of inputs, a set of environment variables and a JSON structure called network configuration (NetConf).
The environment variables are standardised, but the NetConf allows a CNI plugin to define custom input parameters.
Your CNI plugin will require some custom input parameters, which are:

- The IP address range of the **host network**
- The IP address range of the **Pod network**
- The IP address range of the node's **Pod subnet**

The CNI plugin will expect these parameters as custom fields named `myHostNetwork`, `myPodNetwork`, and `myPodSubnet` in the Netconf.
Consequently, an example NetConf for your CNI plugin looks like this:

```
{
  "cniVersion": "0.3.1",
  "name": "my-pod-network",
  "type": "my-cni-plugin",
  "myHostNetwork": "10.0.0.0/16",
  "myPodNetwork": "200.200.0.0/16",
```

```
  "myPodSubnet": "200.200.1.0/24"
}
```

You will have to create such a NetConf for each node when you will install the CNI plugin later.

Now, let's start coding!

The boilerplate code

Start by creating a new working directory for your CNI plugin:

bash

```
$ mkdir my-cni-plugin
$ cd my-cni-plugin
```

Your CNI plugin will be called `my-cni-plugin`, so create this file and make it executable right away:

bash

```
$ touch my-cni-plugin
$ chmod +x my-cni-plugin
```

You're ready to write some code!

Add the following boilerplate code to your `my-cni-plugin` file:

my-cni-plugin

```
#!/bin/bash

case "$CNI_COMMAND" in
  ADD)
    ;;
  DEL)
    ;;
  VERSION)
    ;;
esac
```

This is the basic structure of your CNI plugin implementation.

As you have learned previously, the CNI operation to perform is conveyed by the kubelet to the CNI plugin in the `CNI_COMMAND` environment variable, and CNI version 0.3.1 (which you are implementing) defines three CNI operations: `ADD`, `DEL`, and `VERSION`.

The above code checks the `CNI_COMMAND` variable and then executes one of three blocks, depending on the value of the variable.

The remainder of implementing your CNI plugin will, to the largest part, consist of filling in these blocks with the implementation of each CNI operation.

However, note that the `ADD` operation is by far the most complex one, and most of the following instructions will

refer to the `ADD` operation.
That's the scaffold of your CNI plugin — let's see what's next.

Configuring the output

Add the following code to the beginning of your CNI plugin file:

my-cni-plugin

```
#!/bin/bash

exec 3>&1
exec &>>/var/log/my-cni-plugin.log
```

The first line above creates a new file descriptor 3 and directs it to `stdout`.
The second line redirects the default file descriptors 1 and 2 from `stdout` and `stderr`,respectively, to a log file.
The effect of this is that anything written to the default file descriptors 1 and 2 ends up in the log file, and only what's written to file descriptor 3 goes to `stdout`.
Why is that necessary?
The reason is that the kubelet takes whatever the CNI

plugin writes to `stdout` as the response from the CNI plugin.

> Remember that a CNI plugin must write its response to `stdout`.

If some command of your CNI plugin inadvertently wrote something to `stdout`, the kubelet would interpret this as a response from your CNI plugin and would likely get confused about it.

With the above code, this can't happen, but you can still write the *real* response of your CNI plugin to `stdout` via file descriptor 3.

Just below the above code, add the following function:

my-cni-plugin

```
log() {
  echo -e "$(date): $*"
}
```

This is a log function that you will use a few times during your code to produce some descriptive log output that will help you understand what's going on.

Note that thanks to the above redirections of file descriptors 1 and 2, this function writes to the log file and not to `stdout`.

Having set up the output, let's turn to the input of your CNI plugin.

Reading the input

Start by adding the following code just below the code from above:

my-cni-plugin

```
netconf=$(cat /dev/stdin)
host_network=$(jq -r ".myHostNetwork" <<<"$netconf")
pod_network=$(jq -r ".myPodNetwork" <<<"$netconf")
pod_subnet=$(jq -r ".myPodSubnet" <<<"$netconf")
```

The first line of this code reads the content of `stdin` into a variable.

Since `stdin` must contain the network configuration NetConf, this sets the `netconf` variable to the NetConf passed by the kubelet to your CNI plugin.

The next lines extract certain values from this NetConf.

In particular, these are the values of the custom NetConf fields that you defined above:

- `myHostNetwork` : IP address range of the host network
- `myPodNetwork` : IP address range of the Pod network

- `myPodSubnet` : IP address range of the Pod subnet of this node

You will use these values various time later in the code.

> Note that the code uses `jq` to process the NetConf JSON. If you followed the previous instructions to create the cluster, then `jq` is installed on your nodes.

Just below this code, add the following line:

my-cni-plugin

```
ipam_netconf=$(jq ". += {ipam:{subnet:\"$pod_subnet\"}}" <<<"$netconf")
```

This line creates a copy of the NetConf JSON object with an added field.
The added field is the `ipam` field that is used by many IPAM plugins, including `host-local` .
The resulting JSON object that's saved in the `ipam_netconf` variable may look like this:

```
{
  "cniVersion": "0.3.1",
  "name": "my-pod-network",
  "type": "my-cni-plugin",
```

```
  "myHostNetwork": "10.0.0.0/16",
  "myPodNetwork": "200.200.0.0/16",
  "myPodSubnet": "200.200.1.0/24"
  "ipam": {
    "subnet": "200.200.1.0/24"
  }
}
```

Checking the `host-local` documentation, you can see that this is a valid NetConf for the `host-local` IPAM plugin.

As you can guess, you will use this NetConf when you will invoke the `host-local` IPAM plugin later in the code.

> Since you will invoke `host-local` in both the `ADD` and `DEL` operations, you prepare the NetConf at this point, so that you can reuse it later.

Finally, add the following line of code:

my-cni-plugin

```
log "CNI_COMMAND=$CNI_COMMAND, CNI_CONTAINERID=$CNI_CONTAINERID, \
     CNI_NETNS=$CNI_NETNS, CNI_IFNAME=$CNI_IFNAME, \
     CNI_PATH=$CNI_PATH\n$netconf"
```

This line uses the `log` function that you defined above to write a log entry to the log file.

The log entry includes the values of all CNI environment variables, as well as the NetConf — you will find this useful

when you later run the CNI plugin to see what's going on.
That's it for the preparatory work.
The next step is to turn to the actual CNI operations.

Invoking the IPAM plugin

You will now start implementing the `ADD` operation of your CNI plugin.

> All of the next few steps belong to the `ADD` operation. You will implement the `DEL` and `VERSION` operations in two separate steps at the very end.

One of the first things the `ADD` operation should do is invoking the IPAM plugin to determine the IP address for the new Pod.
To do so, add the following line to the `ADD` block of your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
```

```
ipam_response=$(/opt/cni/bin/host-local <<<"$ipam_netconf")
```

The above line executes `/opt/cni/bin/host-local` by passing it the content of the `ipam_netconf` variable to `stdin` and saving its `stdout` output in a variable named `ipam_response`.

Note the following about this command:

- The executed file is the `host-local` IPAM plugin
- The value in `ipam_netconf` passed to `stdin` is the NetConf that you prepared above
- The value saved in `ipam_response` is the response from the `host-local` IPAM plugin

> Kubernetes includes all the official example CNI plugins by default in `/opt/cni/bin` on each node. This is the reason that you can invoke `host-local` without having to install it first.

Please note that the `host-local` IPAM plugin uses the same interface as every other CNI plugin — that means, it expects a NetConf on `stdin`, a set of environment variables, and returns a response, as defined in the CNI specification.

This is because every CNI plugin uses the same interface, and an IPAM plugin is just a special-purpose CNI plugin.

But what about the environment variables?

You didn't set any of the CNI environment variables (such as `CNI_COMMAND`) before invoking `host-local`, so why does it still work?

The reason is that, on Linux, a sub-process inherits the environment variables from its parent process, which means that `host-local` automatically gets the same set of environment variables that your CNI plugin currently has.

So, if the CNI environment variables that you want to convey to a CNI helper plugin are exactly the same as those that your CNI plugin got from the kubelet, then there is no need for you to redefine them before invoking a CNI helper plugin.

> Since the `CNI_COMMAND` is currently set to `ADD`, the above code invokes the `ADD` operation of `host-local`.

Let's now turn to the response from the `host-local` *IPAM plugin.*

In general, the response from an IPAM plugin has the same format described in the CNI specification as the response from any other CNI plugin.

Here is a typical example response from the `host-local` IPAM plugin:

```
{
  "cniVersion": "0.3.1",
  "ips": [
    {
      "version": "4",
      "address": "200.200.0.2/24",
      "gateway": "200.200.0.1"
    }
  ],
  "dns": {}
}
```

The important bits in this response are the values of the `address` and `gateway` fields.

These are the IP addresses that `host-local` selected for the new Pod and the default gateway of the Pod subnet, respectively.

Add the following code to extract and save these values:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    pod_ip=$(jq -r '.ips[0].address' <<<"$ipam_response")
    bridge_ip=$(jq -r '.ips[0].gateway' <<<"$ipam_response")
```

The above code saves the IP address in the `address` field to a variable named `pod_ip` and the IP address in the `gateway` field to a variable named `bridge_ip`.

You will need these variables later when you do the actual network configurations.

In particular, you will assign the IP address in `pod_ip` to the network interface of the new Pod's network namespace, and you will use the IP address in `bridge_ip` as the IP address for the bridge in the default network namespace.

> Please note that an IPAM plugin does *not* assign IP addresses to any network interface, nor does it do any other network configurations. All that an IPAM plugin does is selecting an IP address for the new Pod (and for the default gateway of the Pod subnet, which is always the same) and returning it in a response as shown above. It is up to the caller of an IPAM plugin to do anything with these IP addresses.

Having invoked the IPAM plugin, the next step to do is the one-time setup.

The one-time setup

The one-time setup consists of those configurations that have to be done *only once*.

This is opposed to the Pod-specific setup which are those

configurations that have to be done *each time* that a Pod is created or deleted.

The one-time setup includes the creation of the bridge and inter-node communication routes in the default gateway of the host network.

To ensure that these resources are created only once, your CNI plugin should check whether they already exist, and only create them if they don't.

However, this introduces another issue — concurrency.

CNI plugins are invoked in parallel by the kubelet — that means, if multiple Pods are created at the same time, then multiple instances of the CNI plugin run at the same time.

As the check whether a resource exists and the creation of the resource are two separate steps, this may lead to race conditions resulting in attempts to create the same resource twice.

A possible way to prevent this is to make the entire one-time setup a critical section that can only be executed by at most one process at a time.

You can do that by adding the following code to your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    {
```

```
    flock 100

} 100>/tmp/my-cni-plugin.lock
```

The above is an idiomatic way to create a critical section in Bash.

Any code between the `flock` command and the closing curly brace is the critical section and can only be executed by a single process at a time — if another process reaches the `flock` command while a process still in the critical section, it waits until the other process leaves the critical section.

> Technically, `flock` works by acquiring a lock on the file pointed to by the file descriptor given to it as an argument. In your case, the file descriptor is 100 and the file is `/tmp/my-cni-plugin.lock`. When a process leaves the curly-braced block, the file descriptor is unset and thus the lock is released.

Having the concurrency issue out of the way, let's turn to the actual one-time setup.

You will start by creating the bridge and the corresponding route in the default network namespace:

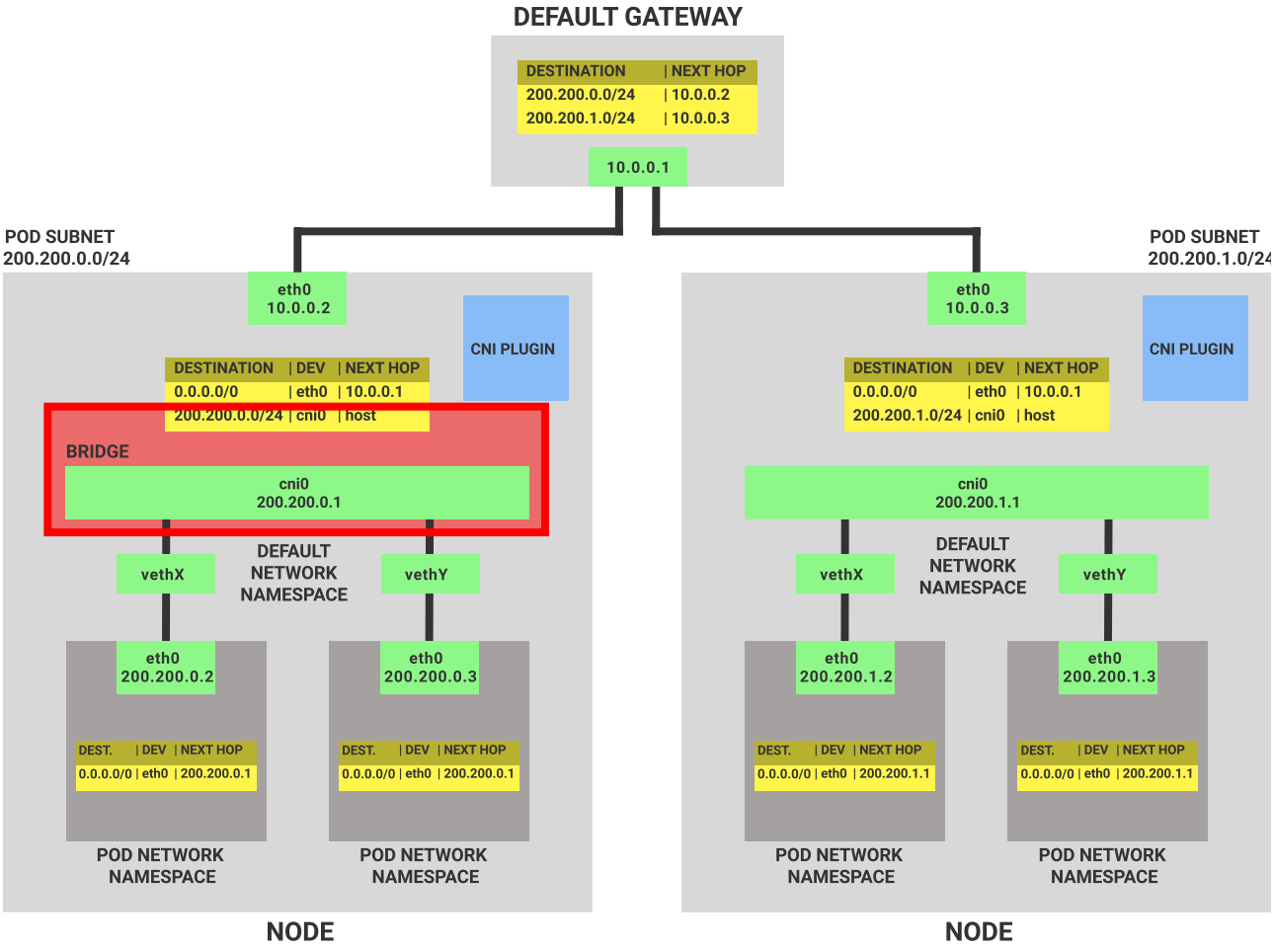


Fig. CNI plugin — creating the bridge

To do so, add the following code to the critical section that you just created above:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    {
      flock 100
```

```
        if ! ip link show cni0 &>/dev/null; then
          ip link add cni0 type bridge
          ip address add "$bridge_ip"/"${pod_subnet#*/}" dev cni0
          ip link set cni0 up
        fi
      } 100>/tmp/my-cni-plugin.lock
```

The above code checks whether the bridge already exists and creates it if it doesn't.

The `if` condition checks whether there exists a network interface named `cni0` — if yes, it evaluates to false, and if no, it evaluates to true.

> Remember that `cni0` is the name of the bridge in your CNI plugin design.

The body of the `if` statement contains three commands:

- The `ip link add` command creates the bridge
- The `ip address add` command assigns an IP address to the bridge
- The `ip link set` command enables the bridge

The IP address assigned to the bridge is composed of `bridge_ip`, which is the IP address that the `host-local` IPAM plugin returned in the `gateway` field, and the CIDR suffix of the Pod subnet, which in your cluster is 24 — an example IP address assigned to the bridge might be

200.200.0.1/24.

> Note that this is the first use of the result of the IPAM plugin.

The last command that enables the bridge has an important side-effect — it creates the route for the Pod subnet via the bridge in the default network namespace (see illustration above).
This is a default behaviour when enabling a network interface and it is made possible by the inclusion of the CIDR suffix in the IP address assigned to the bridge, which defines the size of the subnet to which the bridge provides access.

> The above code makes heavy use of the `ip` command and so will the code that follows. The `ip` command is a general-purpose networking tool that can manipulate network interfaces, routing tables, and more. It is generally regarded as a replacement for a whole set of older commands such as `ifconfig`, `route`, `netstat`, and `arp`.

The next configuration that's part of the one-time setup is the creation of the inter-node communication routes in the

default gateway of the host network:

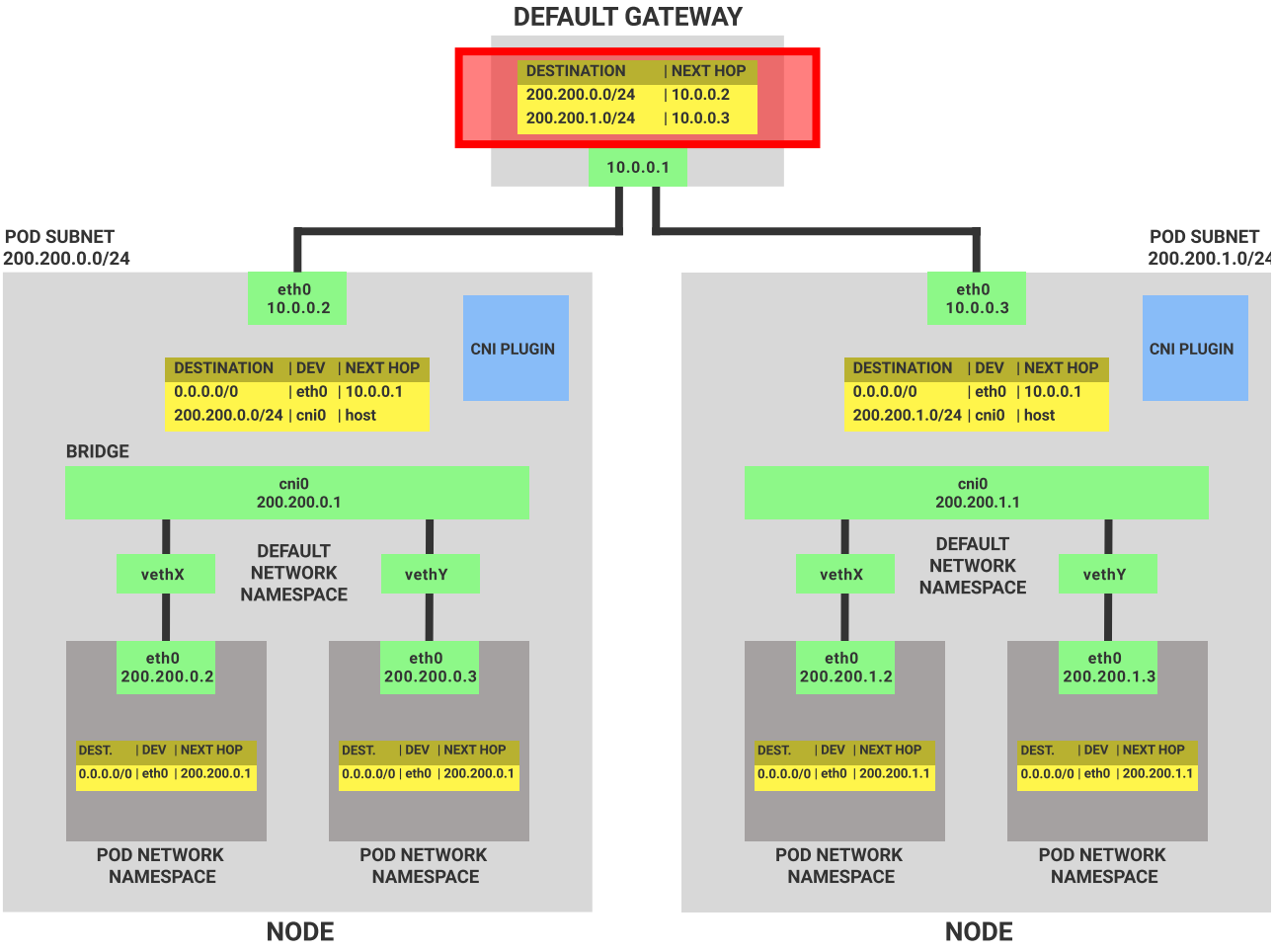


Fig. CNI plugin — creating inter-node routes

Here you are going to cheat a little.

Technically, it should be the CNI plugin that creates these routes.

However, your cluster runs on GCP, and creating these routes in a GCP VPC network has to be done through the GCP API (for example, with `gcloud`).

While it's possible for your CNI plugin to do this, it has two disadvantages: first, it ties the CNI plugin to GCP (that is, you can't use it on different types of infrastructure), and, second, it requires you to grant permissions to access the GCP API to your VM instances, which complicates the deployment of the CNI plugin.

To circumvent these issues, you will simply create these routes manually in the cluster.

Since these routes need to be created only once during the entire lifetime of a cluster, this means that the CNI plugin never has to bother with them, but the Pod network in your cluster still works as expected.

To create the routes, execute the following command:

bash

```
$ for node in my-k8s-master my-k8s-worker-1 my-k8s-worker-2; do
  node_ip=$(kubectl get node "$node" -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}')
  pod_subnet=$(kubectl get node "$node" -o jsonpath='{.spec.podCIDR}')
  gcloud compute routes create "$node" --network=my-k8s-vpc --destination-range="$pod_subnet" \
    --next-hop-address="$node_ip"
done
```

The above command first retrieves the IP address and the Pod subnet IP address range of each node in your cluster.

These parameters are retrieved from the Node in Kubernetes, in particular, from the `node.status.addresses` field (IP address) and the `node.spec.podCIDR` field (Pod subnet IP address range).

The command then creates the appropriate routes through the GCP API with `gcloud`.

Once you have done that, the routes will stay there and will remain valid as long as the Kubernetes cluster exists.

> The GitHub repository of this course contains a script named `inter-node-routes.sh` that automates the creation and deletion of these routes.

There's one last, less obvious, thing that has to be done in the one-time setup.

Linux contains a packet filtering framework named netfilter that can be configured with a tool named `iptables`.

This framework allows defining rules for dropping and altering packets as they travel through the network stack.

> Netfilter is typically used to implement firewalls and network address translation (NAT).

The design of Kubernetes requires you to make some adjustments to the default Netfilter rules for your Pod network to work properly.

To do so, add the following code to the critical section of your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
```

```
  ADD)
    # ...
    {
      flock 100
      # ...
      ensure() {
        eval "$(sed 's/-A/-C/' <<<"$@")" &>/dev/null || eval "$@"
      }

      ensure iptables -A FORWARD -s "$pod_network" -j ACCEPT
      ensure iptables -A FORWARD -d "$pod_network" -j ACCEPT

      iptables -t nat -N MY_CNI_MASQUERADE &>/dev/null
      ensure iptables -t nat -A MY_CNI_MASQUERADE -d "$pod_network" -j RETURN
      ensure iptables -t nat -A MY_CNI_MASQUERADE -d "$host_network" -j RETURN
      ensure iptables -t nat -A MY_CNI_MASQUERADE -j MASQUERADE
      ensure iptables -t nat -A POSTROUTING -s "$pod_subnet" -j MY_CNI_MASQUERADE
    } 100>/tmp/my-cni-plugin.lock
```

So what does this code do?

First of all, the `ensure` function checks whether a given Netfilter rule already exists and creates it if it doesn't — this is to ensure that rules are created only once.

The remaining code consists of two sets of Netfilter rules that solve two different problems.

The first set consists of two rules:

```
ensure iptables -A FORWARD -s "$pod_network" -j ACCEPT
ensure iptables -A FORWARD -d "$pod_network" -j ACCEPT
```

This solves the problem that Linux hosts, by default, drop any packets that are neither destined to them nor originating from them (in other words, Linux hosts, by default, don't *forward* packets).

Imagine that a Pod sends a packet to another Pod on the

same host — the packet passes through the default network namespace of the node, but neither the source nor the destination IP address of the packet matches the IP address of the node, and so the default network namespace would just drop this packet.

The above two rules instruct the default network namespace to *not* drop such third-party packets *if* they either originate from a Pod or are destined to a Pod.

Without these rules, the Pods in your cluster can't communicate with each other, neither with Pods on the same node nor with Pods on different nodes.

The next set of netfilter rules is a bit longer:

```
iptables -t nat -N MY_CNI_MASQUERADE &>/dev/null
ensure iptables -t nat -A MY_CNI_MASQUERADE -d "$pod_network" -j RETURN
ensure iptables -t nat -A MY_CNI_MASQUERADE -d "$host_network" -j RETURN
ensure iptables -t nat -A MY_CNI_MASQUERADE -j MASQUERADE
ensure iptables -t nat -A POSTROUTING -s "$pod_subnet" -j MY_CNI_MASQUERADE
```

These rules set up network address translation (NAT) that allows your Pods to access destinations outside the cluster (such as the public Internet).

Again, this is necessary because Pods have a different IP address than the nodes that they're running on.

Imagine, a Pod sends a packet to a destination outside the cluster, for example, 104.31.71.82 — the packet is routed to the default gateway of the GCP subnet, but since the packet

has the Pod's IP address in the source field, the gateway will drop it, because it doesn't recognise Pod IP addresses.

The above rules fix this by setting up NAT in the default network namespace that replaces the Pod's IP address in the source field with the node's IP address for every packet that leaves the cluster — in that way, gateway recognises and processes the packet, which allows it to reach its destination.

The above code works by creating a custom Netfilter chain named `MY_CNI_MASQUERADE` which performs NAT (masquerading) on all packets to a destination *outside* the cluster, which is expressed as all packets that are *not* destined to the Pod network and *not* destined to the host network.

Without the above set of rules, your Pods can't reach destinations outside the cluster.

That's it for the one-time setup!

The next thing to do is the Pod-specific setup.

The Pod-specific setup

The Pod-specific setup is the heart of the `ADD` operation of your CNI plugin.

It consists of those configurations that are executed for every new Pod, which is the connection of the Pod network

namespace to the bridge:

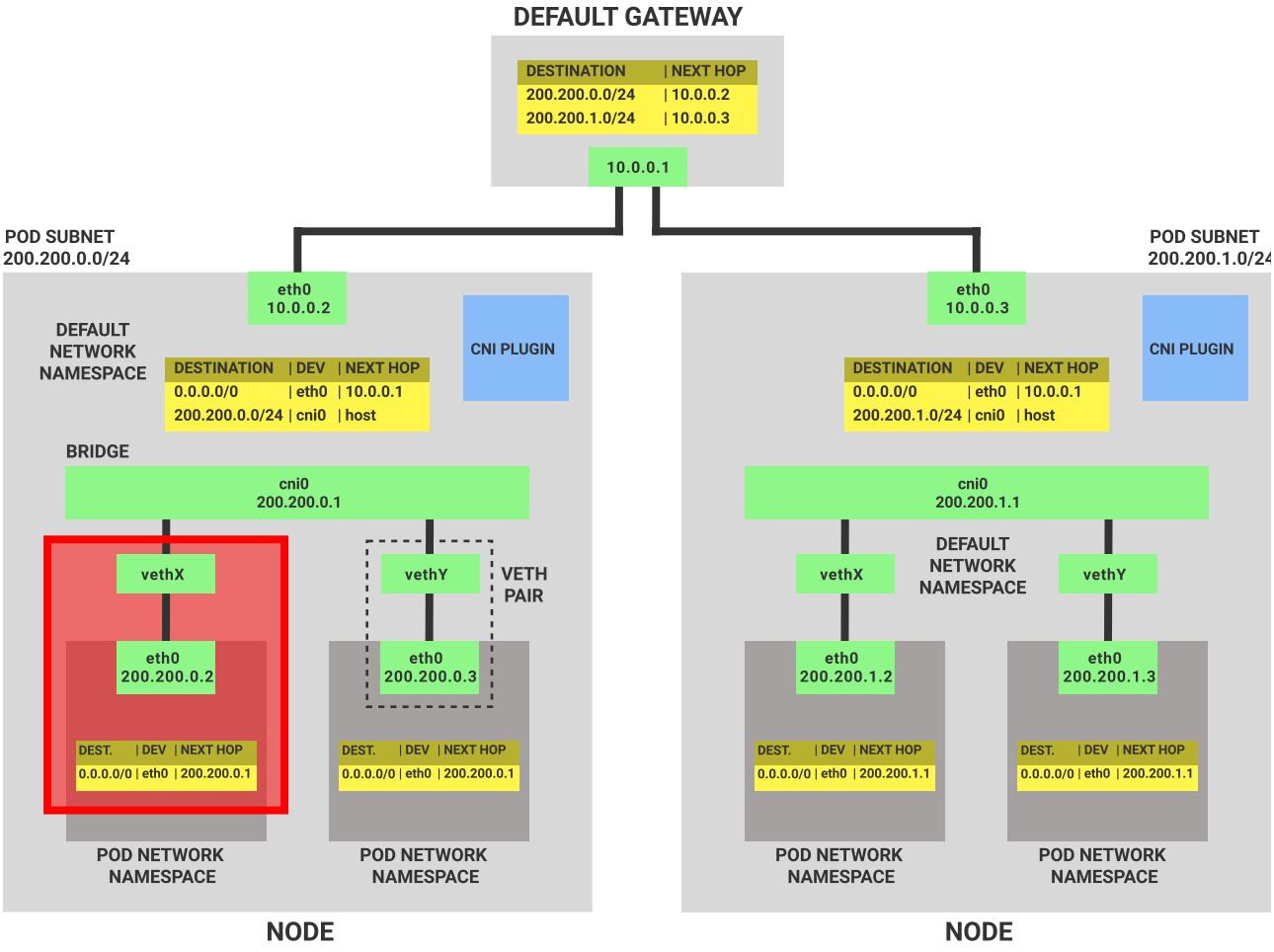


Fig. CNI plugin — Pod-specific setup

To implement this, you will use some the following CNI environment variables that the kubelet passes to your CNI plugin:

- `CNI_NETNS` is a reference to the Pod network namespace as a path in the form of `/proc/<PID>/ns/net`
- `CNI_CONTAINERID` is the ID of the Pod that is being

- created
- `CNI_IFNAME` is the name for the network interface to create in the Pod network namespace

The first thing that you have to do is to create a named link to the Pod network namespace.

This is necessary so that you can refer to this network namespace with the `ip` command, which you will use for the rest of the setup.

To do so, add the following code to your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    mkdir -p /var/run/netns/
    ln -sf "$CNI_NETNS" /var/run/netns/"$CNI_CONTAINERID"
```

The above code allows you to use the following construct to execute arbitrary commands in the Pod network namespace:

bash

```
$ ip netns exec "$CNI_CONTAINERID" <cmd>
```

If you, for example, execute `ip netns exec "$CNI_CONTAINERID" ip link show`, you get the network interfaces of the Pod network namespace and *not* the network interfaces of the default network namespace.

The remainder of the Pod-specific setup can be divided into four tasks:

1. Creating a veth pair
2. Setting up the host-end of the veth pair
3. Setting up the Pod-end of the veth pair
4. Creating the default route in the Pod network namespace

Let's go through these tasks one by one.

Creating a veth pair

First of all, you have to create the raw veth pair that you will later set up to connect the Pod network namespace to the bridge.

One end of the veth pair, the Pod-end, is supposed to be in the Pod network namespace and must be named according to the `CNI_IFNAME` environment variable (which by default is `eth0` in Kubernetes).

The other end of the veth pair, the host-end, is supposed to be in the default network namespace and must be connected to the bridge.

You could either create the veth pair in the default network namespace (and then move the Pod-end to the Pod network namespace) or in the Pod network namespace (and them move the host-end to the default network namespace).

You will use the second approach, because creating the veth pair in the default network namespace may lead to name clashes with the Pod-ends if multiple instances of the CNI plugin run concurrently.

To do so, add the following code to your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    host_ifname=veth$RANDOM
    ip netns exec "$CNI_CONTAINERID" ip link add "$CNI_IFNAME" type veth peer name "$host_ifname"
```

The first line in the above code chooses a random name for the host-end of the veth pair, and the second line creates the veth pair in the Pod network namespace.

At this point, your configuration looks like this:

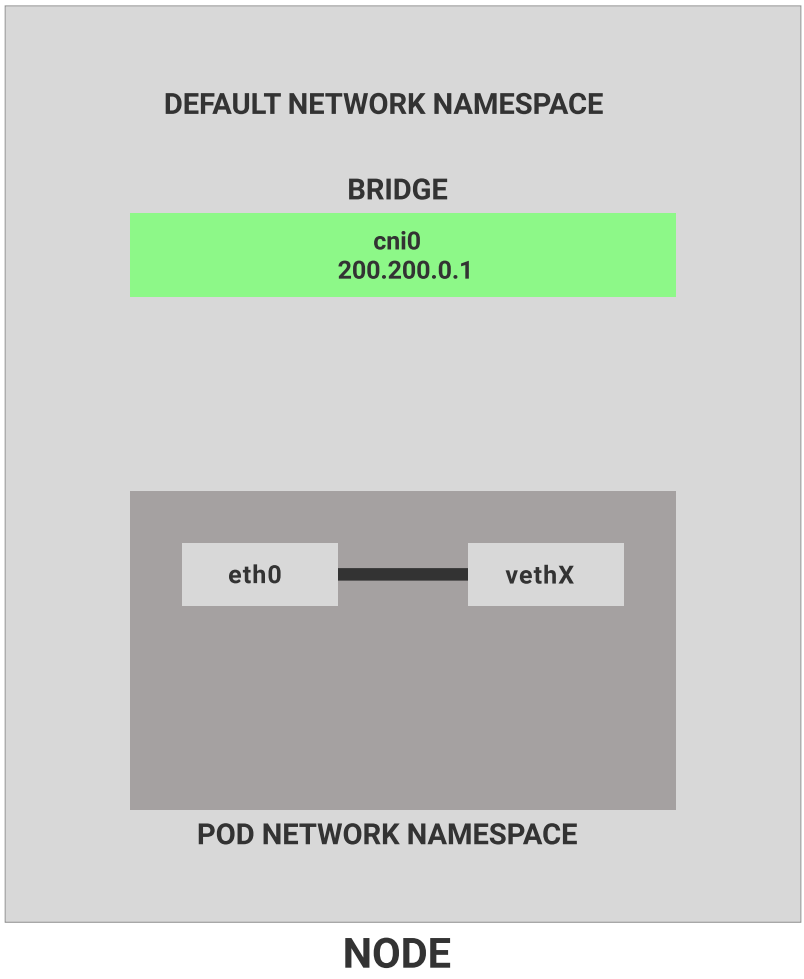


Fig. CNI plugin — Pod-specific setup

You have two network interfaces in the Pod network namespace, one named `eth0` (since this is the default value of `CNI_IFNAME`) and the other with a random name.

However, these network interfaces don't do anything yet because they are neither configured nor enabled.

The next step is thus to set them up.

Setting up the host network interface

The host-end of the veth pair should be in the default network namespace and it should be connected to the bridge.

You can do that with the following code:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    ip netns exec "$CNI_CONTAINERID" ip link set "$host_ifname" netns 1
    ip link set "$host_ifname" master cni0 up
```

The first line in the above code moves the host network interface to the default network namespace, and the second line connects it to the bridge and enables it.

At this point, your configuration looks like this:

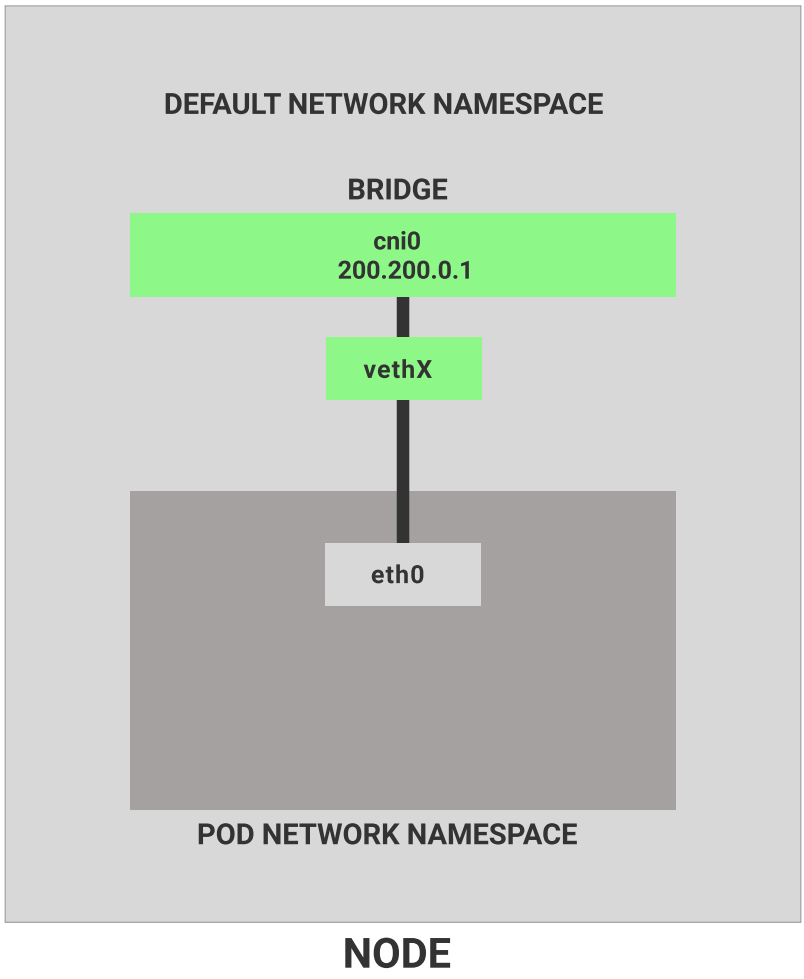


Fig. CNI plugin — Pod-specific setup

The host network interface is now correctly set up, but its counterpart in the Pod network namespace is still untouched.

Let's set up that one next.

Setting up the Pod network interface

The Pod-end of the veth pair should remain in the Pod

network namespace and it should be assigned an IP address.

This should be the IP address that was previously returned by the IPAM plugin because the Pod-end of the veth pair will be the Pod's primary network interface.

You can implement with the following code:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    ip netns exec "$CNI_CONTAINERID" ip address add "$pod_ip" dev "$CNI_IFNAME"
    ip netns exec "$CNI_CONTAINERID" ip link set "$CNI_IFNAME" up
```

The first line in the above code assigns an IP address to the Pod-end of the veth pair.

Note that this is the IP address that was previously returned by the `host-local` IPAM plugin and that you saved in the `pod_ip` variable.

The second line finally enables the network interface.

Your configuration looks now like this:

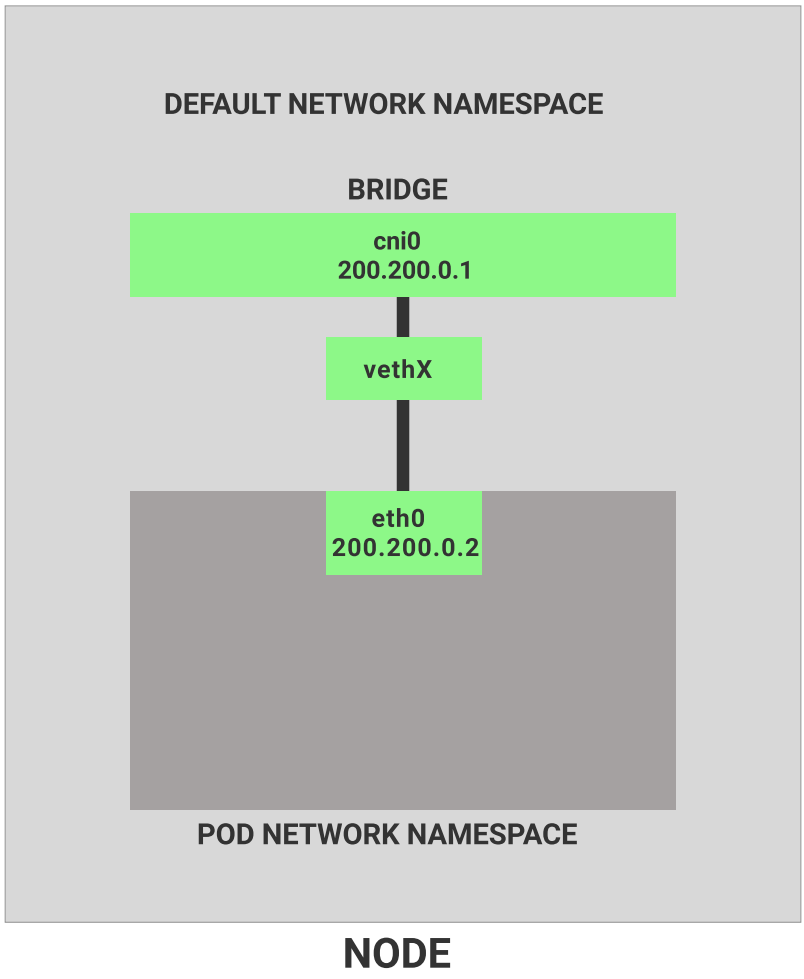


Fig. CNI plugin — Pod-specific setup

Both network interfaces of the veth pair are now correctly set up and enabled — ready for communication.

However, something is still missing.

The Pod network namespace does not yet have a default route.

Let's fix this next.

Creating a default route in the Pod network namespace

By default, a new Linux network namespace does not have a default route configured.

However, a default route is required, because, in your CNI plugin design, the bridge in the default network namespace acts as the default gateway for all the Pod network namespaces.

Thus, you have to add a default route to the bridge's IP address in the Pod network namespace.

You can do that with the following code:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    ip netns exec "$CNI_CONTAINERID" ip route add default via "$bridge_ip" dev "$CNI_IFNAME"
```

At this point, your configuration looks like this:

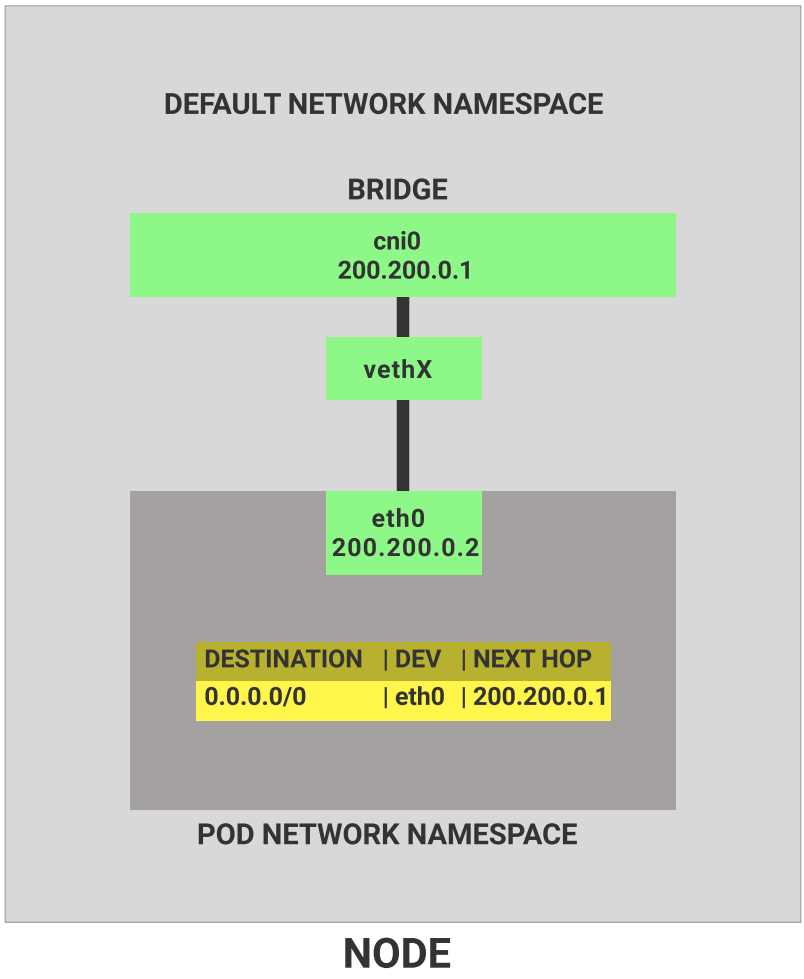


Fig. CNI plugin — Pod-specific setup

This is the final state of the setup!

The Pod network namespace is now correctly wired up so that the Pod can communicate with all other Pods (as well as with any other destination inside and outside the cluster).

The only thing that your CNI plugin has left to do is returning a response to the kubelet.

Returning the response

The response that the `ADD` operation of the CNI plugin is supposed to return to the kubelet is defined in the CNI specification.

The main purpose of the response is to let the kubelet now about the network interface that was created in the Pod network namespace including its IP address.

Here is a typical example response:

```
{
  "cniVersion": "0.3.1",
  "ips": [
    {
      "version": "4",
      "address": "200.200.0.2/24",
      "gateway": "200.200.0.1",
      "interface": 0
    }
  ],
  "interfaces": [
    {
      "name": "eth0",
      "sandbox": "/proc/7182/ns/net"
    }
  ]
}
```

The `interfaces` field contains the name of the network interface that the CNI plugin created in the Pod network

namespace, and the `ips` field contains the IP address that was assigned to it.

> Both the `interfaces` and `ips` fields are arrays, because a CNI plugin may create more than one network interface in a Pod network namespace (for example, to connect the Pod to multiple Pod networks). The IP addresses are mapped to the network interfaces with an index into the `interfaces` array in the `ips.interface` field.

If this JSON object looks familiar to you, then you are right — you already received a similar response from the `host-local` IPAM plugin that contained the `ips` field with the IP address for the Pod.

This is because an IPAM plugin is just a CNI plugin, and every CNI plugin returns the same type of response.

This comes in handy for you now, because you can reuse the response you got from `host-local` and just augment it with the missing information.

The following code demonstrates that:

my-cni-plugin

```
case "$CNI_COMMAND" in
  ADD)
    # ...
    response=$(jq ". += {interfaces:[{name:\"$CNI_IFNAME\",sandbox:\"$CNI_NETNS\"}]} | \
      .ips[0] += {interface:0}" <<<"$ipam_response")
```

```
    log "Response:\n$response"
    echo "$response" >&3
    ;;
```

The first line in the above code constructs the final response of your CNI plugin.

In particular, it takes the response from the `host-local` IPAM plugin (that you saved in the `ipam_response` variable) and adds the `interfaces` field to it.

> This is necessary because responses from IPAM plugins never contain the `interfaces` field.

Since the `host-local` response contains the IP address of the Pod in the `ips` field and you added the `interfaces` field now, the response object is complete and should resemble the example shown above.

The second line in the code writes the response to the log file (which you will find useful for debugging) and the last line writes it to `stdout` where it will be read by the kubelet.

> The `>&3` in the last line causes the output to be written to file descriptor 3, which you directed to `stdout` at the beginning of your code.

That's the complete code for the `ADD` operation of your

CNI plugin!

There are two remaining operations, `DEL` and `VERSION` that you have to implement next.

The DEL operation

The `DEL` operation is invoked by the kubelet whenever a Pod is being deleted.

The purpose of the `DEL` operation is to disconnect the Pod network namespace from the Pod network and release any resources that are not needed anymore.

The implementation is simpler than you might think — here is the complete code:

my-cni-plugin

```
case "$CNI_COMMAND" in
  DEL)
    /opt/cni/bin/host-local <<<"$ipam_netconf"
    rm -f /var/run/netns/"$CNI_CONTAINERID"
    ;;
```

The first line invokes the `host-local` IPAM plugin in the same way that it is invoked in the `ADD` operation.

However, there is a crucial difference now — this time, the `CNI_COMMAND` environment variable is set to `DEL`, which

means that `host-local` will execute its `DEL` operation too.

The `DEL` operation of the `host-local` IPAM plugin releases the allocation of the Pod's IP address, which makes this IP address available again for other Pods.

The second line in the above code removes the named link to the Pod network namespace that was created by the `ADD` operation.

That's all you have to do for the `DEL` *operation.*

What about the other resources that the `ADD` operation created for the Pod, such as the veth pair and the default route?

The crux is that these resources are tied to the Pod's network namespace and the kubelet will delete the entire network namespace in the process of deleting the Pod.

On Linux, any resources associated with a network namespace will be automatically deleted when the network namespace is deleted, so there is no need for you to delete these resources manually.

The VERSION operation

All the `VERSION` operation does is returning the CNI version that this CNI plugin implements as well as the CNI

versions that it supports for its input.

The format of the response object is described in the CNI specification.

Here is the complete implementation for your CNI plugin:

my-cni-plugin

```
case "$CNI_COMMAND" in
  VERSION)
    echo '{"cniVersion":"0.3.1","supportedVersions":["0.1.0","0.2.0","0.3.0","0.3.1"]}' >&3
    ;;
esac
```

As you can see, your CNI plugin reports that it implements CNI version 0.3.1 and that it supports input from all CNI versions up to version 0.3.1.

Again, the output is written to file descriptor 3 which is directed to `stdout` — this is where the kubelet will read it.

That completes the implementation of your CNI plugin.

You made it to the end!

> As mentioned, you can find the complete source file of the CNI plugin in the lab's GitHub repository.

Take a break after this hard work and get ready for the next step.

In the next section, you will install and test your CNI plugin.

Chapter 7

Lab 4/4 — Installing and testing the CNI plugin

In this section, you will install and test the CNI plugin that you created in the previous section.

Remember that the cluster you created does not yet have a CNI plugin installed, and therefore all its nodes are `NotReady`.

After you install your CNI plugin, this will change and your cluster will become fully functional.

Once your CNI plugin is installed, you will verify that it works as expected.

Let's get started!

Installing the CNI plugin

To understand how a CNI plugin has to be installed, it helps to envision how the kubelet handles CNI plugins:

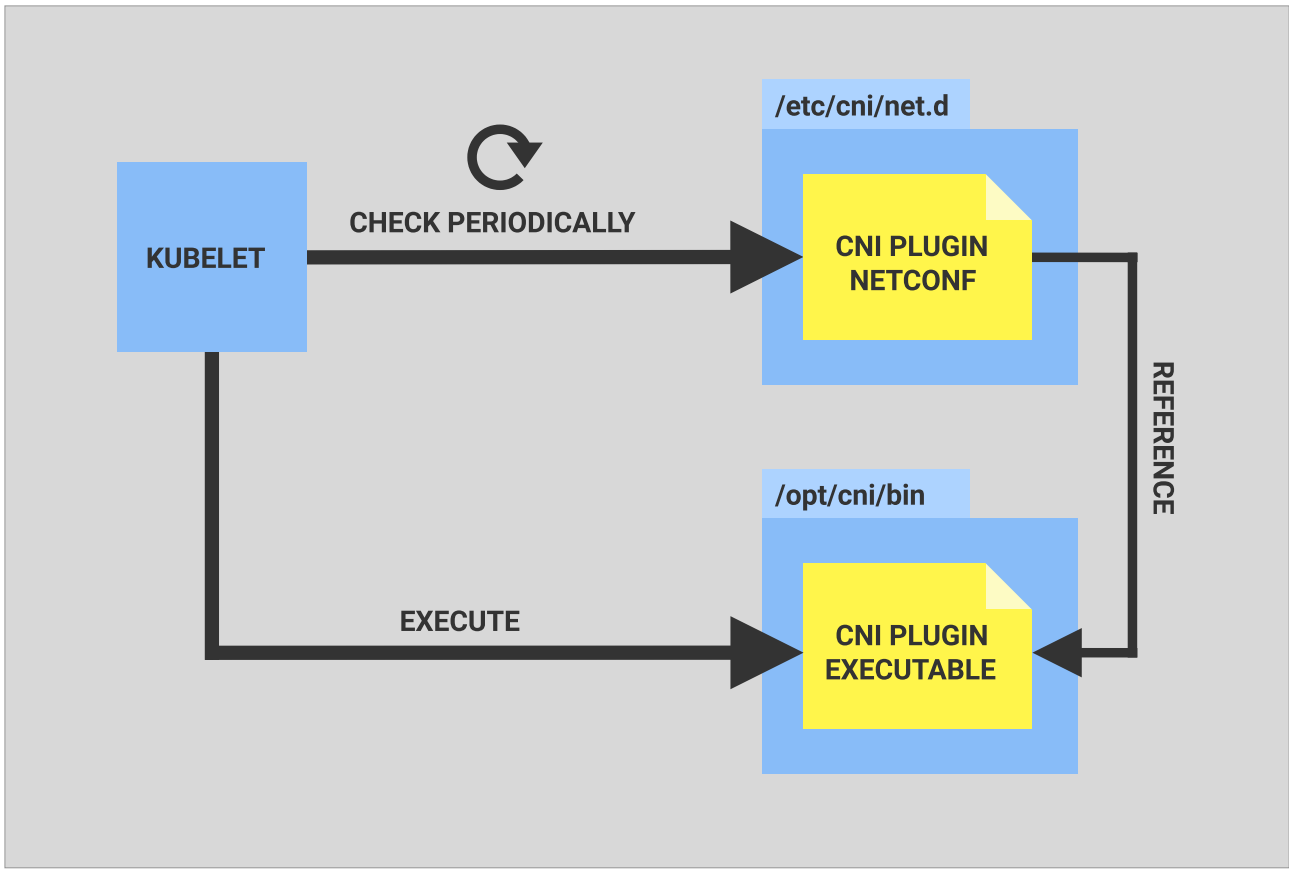


Fig. Installing a CNI plugin

The kubelet periodically checks for a network configuration (NetConf) file of a CNI plugin in a specific directory.
This directory is `/etc/cni/net.d` by default, but can be customised with the `--cni-conf-dir` kubelet flag.

> Note that this is the same NetConf that is passed to the CNI plugin executable on `stdin`.

If the kubelet finds a NetConf file there, it accepts this as the currently installed CNI plugin — if it doesn't, then it reports a `KubeletNotReady` status signalising that it didn't find a CNI plugin.

> This is what's currently happening in your cluster, which is the reason that your nodes are `NotReady`.

Every NetConf file contains the name of the executable file belonging to the CNI plugin (in the `type` field).
When it comes to executing the CNI plugin, the kubelet reads the name of this executable file from the NetConf and then searches this file in a specific directory.
This directory is `/opt/cni/bin` by default, but can be customised with the `--cni-bin-dir` kubelet flag.
If the kubelet can find the file in this directory, then it executes it, otherwise, it reports an error.
So, to install a CNI plugin to a specific node, you have to do two things:

1. **Put the CNI plugin executable in `/opt/cni/bin` of the node**
2. **Put a NetConf file in `/etc/cni/net.d` of the node**

And you have to do this on every node of your cluster.

> You can find a script named `cni-plugin.sh` on the course's GitHub repository that automates the installation and deinstallation of the CNI plugin in your Kubernetes cluster.

For the first step, installing the CNI plugin executable, you can use the following command:

bash

```
$ for node in my-k8s-master my-k8s-worker-1 my-k8s-worker-2; do
  gcloud compute scp my-cni-plugin root@"$node":/opt/cni/bin
done
```

The above command uploads your `my-cni-plugin` executable to the `/opt/cni/bin` directory of each node of the cluster.

The second step, installing the NetConf file, need some more work because you haven't created a NetConf yet.

Let's first recap how an example NetConf for your CNI plugin looks like:

```
{
  "cniVersion": "0.3.1",
  "name": "my-pod-network",
  "type": "my-cni-plugin",
  "myHostNetwork": "10.0.0.0/16",
  "myPodNetwork": "200.200.0.0/16",
```

```
  "myPodSubnet": "200.200.1.0/24"
}
```

Note the following about this NetConf:

- The `cniVersion` field is set to `0.3.1` because your CNI plugin implements CNI version 0.3.1
- The `name` field is the name of the Pod network created by the CNI plugin and it can be set to an arbitrary value
- The `type` field is the name of the CNI plugin executable
- The `myHostNetwork`, `myPodNetwork`, and `myPodSubnet` fields are the custom fields used by your CNI plugin

The crux about this NetConf is that the `myPodSubnet` field must be different for each node because each node has a different Pod subnet.

You could look up these values manually and then create a NetConf file for each node and upload them to the correct node — however, this is tedious and error-prone.

A better way is to automate the process with a template.

Save the following to file named `my-cni-plugin.conf.jsonnet`:

my-cni-plugin.conf.jsonnet

```
{
  "cniVersion": "0.3.1",
```

```
  "name": "my-pod-network",
  "type": "my-cni-plugin",
  "myHostNetwork": "10.0.0.0/16",
  "myPodNetwork": "200.200.0.0/16",
  "myPodSubnet": std.extVar("podSubnet")
}
```

The above is a Jsonnet template that parameterises the value of the `myPodSubnet` field so that it can be filled in dynamically.

Jsonnet is a JSON templating tool, and if you haven't yet, you should install it with:

bash

```
# macOS
$ brew install jsonnet
# Linux
$ sudo snap install jsonnet
```

Now, you can use the following command to automatically generate and install the NetConf file to each node of your cluster:

bash

```
$ for node in my-k8s-master my-k8s-worker-1 my-k8s-worker-2; do
  tmp=$(mktemp -d)/my-cni-plugin.conf
  pod_subnet=$(kubectl get node "$node" -o jsonpath='{.spec.podCIDR}')
  jsonnet -V podSubnet="$pod_subnet" my-cni-plugin.conf.jsonnet >$tmp
  gcloud compute ssh root@"$node" --command "mkdir -p /etc/cni/net.d"
  gcloud compute scp "$tmp" root@"$node":/etc/cni/net.d
done
```

The above command queries the Pod subnet IP address range from each node in Kubernetes, substitutes the variable in the template, and uploads the resulting NetConf file to the `/etc/cni/net.d` directory of the node.

Your CNI plugin is now fully installed.

Both the CNI plugin NetConf and the executable are now available in the correct locations on each node of the cluster, so the kubelets should now detect your CNI plugin.

If this is true, then the nodes of your cluster should become `Ready`.

Let's do the test:

bash

```
$ kubectl get nodes
my-k8s-master     Ready    master   47h   v1.17.0
my-k8s-worker-1   Ready    <none>   47h   v1.17.0
my-k8s-worker-2   Ready    <none>   47h   v1.17.0
```

Bingo!

The kubelets indeed detected your CNI plugin and all the nodes are `Ready` now.

That means, so far everything seems to work.

However, to be sure, you need to test that your CNI plugin really does what it is expected to do.

Testing the CNI plugin

The purpose of a CNI plugin is to connect Pods to the Pod network so that they can communicate with other Pods, as well as with other destinations on the nodes and outside the cluster.

To verify that your CNI plugin does that, you can create a set of Pods in your cluster and then make the following tests:

1. **Pods have IP addresses**
2. **Pods can communicate to Pods on the same node**
3. **Pods can communicate to Pods on different nodes**
4. **Pods can communicate to processes on the nodes**
5. **Processes on the nodes can communicate to Pods**
6. **Pods can communicate to destinations outside the cluster**

Let's get started.

You can create a set of Pods for doing the tests by saving the following manifest in a file named `pods.yaml`:

pods.yaml

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: pod-1
spec:
  containers:
  - name: ubuntu
    image: weibeld/ubuntu-networking
    command: ["sleep", "infinity"]
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-2
spec:
  containers:
  - name: ubuntu
    image: weibeld/ubuntu-networking
    command: ["sleep", "infinity"]
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-3
spec:
  containers:
  - name: ubuntu
    image: weibeld/ubuntu-networking
    command: ["sleep", "infinity"]
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-4
spec:
  containers:
  - name: ubuntu
    image: weibeld/ubuntu-networking
    command: ["sleep", "infinity"]
```

And then applying the file with:

bash

```
$ kubectl apply -f pods.yaml
```

This creates four standalone Pods in your cluster with each Pod running a `weibeld/ubuntu-networking` container.

> The `weibeld/ubuntu-networking` container consists of Ubuntu 18.04 with some additional networking tools installed. These additional networking tools can come in handy if you want to make more advanced networking experiments in your cluster.

Now, let's start with the tests!

Pods have IP addresses

Let's start by verifying that all Pods got an IP address.
To do so, list the Pods as follows:

bash

```
$ kubectl get pods -o wide
NAME    READY   STATUS    RESTARTS   AGE   IP             NODE
pod-1   1/1     Running   0          5s    200.200.1.4    my-k8s-worker-1
pod-2   1/1     Running   0          5s    200.200.2.4    my-k8s-worker-2
pod-3   1/1     Running   0          5s    200.200.1.5    my-k8s-worker-1
pod-4   1/1     Running   0          5s    200.200.2.5    my-k8s-worker-2
```

All Pods got assigned an IP address.

Note how the IP address of each Pod belongs to the Pod subnet of the node that the Pod is running on.

For example, `pod-1` is running on node `my-k8s-worker-1` and has IP address 200.200.1.4, because node `my-k8s-worker-1` has the Pod subnet IP address range 200.200.1.0/24.

You can check the Pod subnet IP address range assigned to each node with the following command:

bash

```
$ kubectl get node <node> -o jsonpath='{.spec.podCIDR}'
```

So far, your CNI plugin did a good job.

But now let's see if the Pods can actually communicate with each other.

Pods can communicate to Pods on the same node

First, let's test if Pods can communicate with other Pods on the same node.

To do so, log into one of the Pods, for example, `pod-1` :

bash

```
$ kubectl exec -ti pod-1 -- bash
```

Then, pick a Pod that runs on the same node as the Pod you just logged into and note down its IP address.

> In the above example, this is `pod-3` with IP address 200.200.1.5.

Now, try to ping the IP address of this other Pod from the Pod you're logged into:

```bash
$ ping  200.200.1.5
PING 200.200.1.5 (200.200.1.5) 56(84) bytes of data.
64 bytes from 200.200.1.5: icmp_seq=1 ttl=64 time=0.061 ms
64 bytes from 200.200.1.5: icmp_seq=2 ttl=64 time=0.079 ms
```

Yout get a response!

Communication between Pods on the same node works.

Pods can communicate to Pods on different nodes

Now, let's test if Pods can communicate with Pods on different nodes.

Pick a Pod that runs on a different node than the Pod you're logged into and note down its IP address.

> In the above example, this may be `pod-2` with IP address 200.200.2.4.

Then try to ping the IP address of this other Pod from the

Pod you're logged into:

```bash
$ ping  200.200.2.4
PING 200.200.2.4 (200.200.2.4) 56(84) bytes of data.
64 bytes from 200.200.2.4: icmp_seq=1 ttl=62 time=1.52 ms
64 bytes from 200.200.2.4: icmp_seq=2 ttl=62 time=0.333 ms
```

You get a response as well!

Communication between Pods on different nodes works.

Pods can communicate to processes on the nodes

Besides communicating with other Pods, the Pods also need to be able to communicate with processes running in the host network of the cluster.

This includes processes such as the kubelet or the Kubernetes API server.

These processes use the network interface and IP address of the node that they're running on for their network communication.

You can list the IP addresses of all the nodes of your cluster with the following command:

```bash
$ kubectl get nodes -o wide
NAME              STATUS   ROLES    AGE   VERSION
```

```
 INTERNAL-IP
my-k8s-master     Ready    master   46m   v1.17.2   10.0.0.2
my-k8s-worker-1   Ready    <none>   46m   v1.17.2   10.0.0.3
my-k8s-worker-2   Ready    <none>   46m   v1.17.2   10.0.0.4
```

Let's first test if a Pod can communicate with the node it's running on.
To do so, note down the IP address of the node that the Pod you're currently logged into is running on.

> In the above example, this is node `my-k8s-worker-1` with IP address 10.0.0.3.

Then, from the Pod you're logged into, try to ping the IP address of that node:

bash

```
$ ping 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.067 ms
```

It works — your ping requests are answered by the network stack of the default network namespace of the node.
If your packets are received by the network stack of the default network namespace, it means that they can also be received by any process running in the default network

namespace, such as, for example, the kubelet.

That means, Pods can communicate to processes running on the same node.

Now let's test if a Pod can also communicate with processes running on a *different* node.

Pick one of the other nodes in your cluster and note down its IP address.

> In the above example, this may be node `my-k8s-worker-2` with IP address 10.0.0.4.

Then, from the Pod you're logged into, try to ping the IP address of that node:

bash

```
$ ping 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=63 time=1.24 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=63 time=0.252 ms
```

It works just the same as in the previous test with the same node.

Pods can communicate to processes running on different nodes.

Processes on the nodes can communicate to Pods

The communication between Pods and nodes must also work in the other direction.

That means, processes running in the host network on some node must be able to reach a Pod by its IP address.

For example, the kubelet must be able to make requests to the Pods on its node to perform the regular health checks.

To test this, log into one of the nodes of your cluster, say, `my-k8s-worker-1`, with the following command:

bash

```
$ gcloud compute ssh my-k8s-worker-1
```

Let's first test if a process in the host network can communicate to a Pod that runs on the same node.

To do so, pick a Pod that runs on the same node that you just logged into and note down its IP address.

> In the above example, this may be `pod-1` with IP address 200.200.1.4.

Now, try to ping the IP address of this Pod from the node you're currently logged into:

bash

```
$ ping 200.200.1.4
```

```
PING 200.200.1.4 (200.200.1.4) 56(84) bytes of data.
64 bytes from 200.200.1.4: icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from 200.200.1.4: icmp_seq=2 ttl=64 time=0.069 ms
```

Your requests indeed reach the Pod!

That means, processes in the host network can communicate to Pods on the same node

Now, let's test if a process in the host network can also reach a Pod that runs on a different node.

To do so, pick a Pod that runs on a different node than the one you're currently logged into, and note down its IP address.

> In the above example, this may be `pod-2` with IP address 200.200.2.4.

Then, try to ping the IP address of this Pod from the node you're logged into:

bash

```
$ ping 200.200.2.4
PING 200.200.2.4 (200.200.2.4) 56(84) bytes of data.
64 bytes from 200.200.2.4: icmp_seq=1 ttl=63 time=1.52 ms
64 bytes from 200.200.2.4: icmp_seq=2 ttl=63 time=0.253 ms
```

It works just the same as in the previous test with a Pod on the same node.

Processes in the host network can communicate to Pods running on different nodes.

So far, you have tested all possible communication pairings inside the cluster — and all of them work.

But there is one last thing to test.

Pods can communicate to destinations outside the cluster

Most applications in Pods also need to be able to communicate with destinations outside the cluster — in particular, the Internet.

Let's test if the Pods in your cluster can communicate with a destination on the Internet.

To do so, log again into one of the Pods, for example, `pod-1`:

bash

```
$ kubectl exec -ti pod-1 -- bash
```

Then, choose an IP address from a location on the Internet — for example, 104.31.70.82, which is the IP address of `learnk8s.io`.

Now, try to ping this IP address from the Pod you're currently logged into:

bash

```
$ ping 104.31.70.82
PING 104.31.70.82 (104.31.70.82) 56(84) bytes of data.
64 bytes from 104.31.70.82: icmp_seq=1 ttl=54 time=34.8 ms
64 bytes from 104.31.70.82: icmp_seq=2 ttl=54 time=34.7 ms
```

It works — the `learnk8s.io` server on the Internet receives the packets from your Pod!

If you want to make this more explicit, you can of course also use the domain name of a destination on the Internet rather than its IP address:

bash

```
$ ping learnk8s.io
PING learnk8s.io (104.31.70.82) 56(84) bytes of data.
64 bytes from 104.31.70.82 (104.31.70.82): icmp_seq=1 ttl=54 time=34.4 ms
64 bytes from 104.31.70.82 (104.31.70.82): icmp_seq=2 ttl=54 time=34.8 ms
```

This works just the same.

Pods can communicate to destinations outside the cluster.

All tests passed!

You have now tested all possible communication scenarios — and all of them work.

That means, your CNI plugin as a whole works.

You developed and deployed a valid CNI plugin from scratch — congratulations!

You have a fully functional Kubernetes cluster now.

If you want, you can deploy your applications to this cluster.

Or you may make further networking experiments on the Pods of this cluster (the container image used in the Pods has many networking tools installed).

However, if you don't need your cluster anymore, you should delete all the GCP resources used for your cluster to **prevent any unwanted charges on your next monthly GCP bill.**

Below are the instructions to do so.

Cleaning up

You can delete all the GCP resources that you created for this lab with the following sequence of commands:

bash

```
$ gcloud -q compute routes delete my-k8s-master my-k8s-worker-1 my-k8s-worker-2
$ gcloud -q compute instances delete my-k8s-master my-k8s-worker-1 my-k8s-worker-2
$ gcloud -q compute firewall-rules delete my-k8s-internal my-k8s-ingress
$ gcloud -q compute networks subnets delete my-k8s-subnet
$ gcloud -q compute networks delete my-k8s-vpc
```

> In case you created a new GCP project for this lab, you can also just delete the project to remove all the resources. Use `gcloud compute delete <Project-ID>` to delete the project.

After doing this, you can be sure that you're not using any paid services anymore and will not have any bad surprises on your next GCP bill.

To remove all the traces of the cluster on your local machine too, you can remove the kubeconfig file that you used to access the cluster:

```bash
$ rm ~/my-kubeconfig
```

And, you can also unset the `KUBECONFIG` variable:

```
unset KUBECONFIG
```

That's the end of this lab.

You successfully designed, implemented, deployed, and tested your own CNI plugin on Kubernetes.

That's a great achievement!